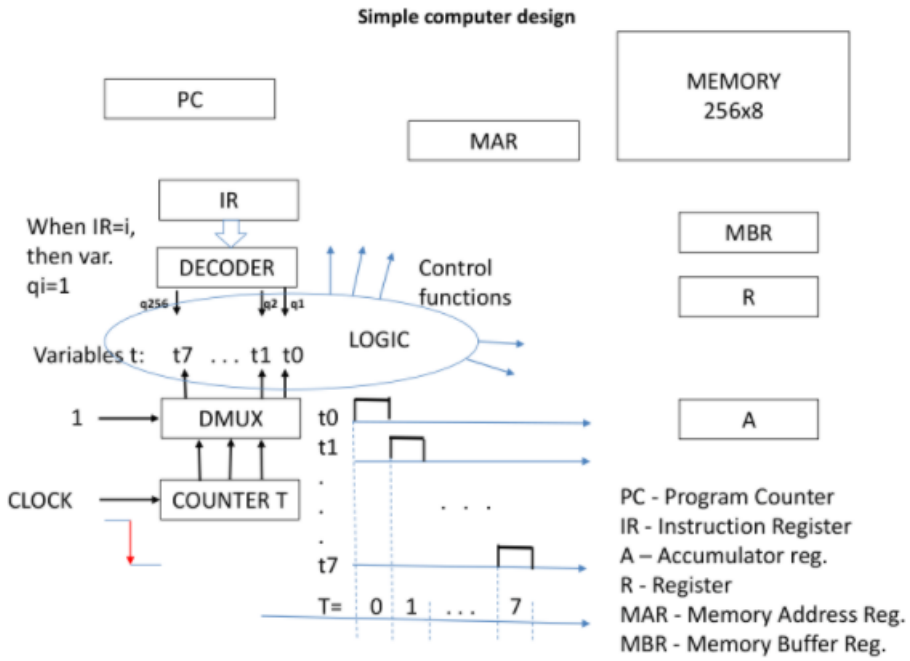
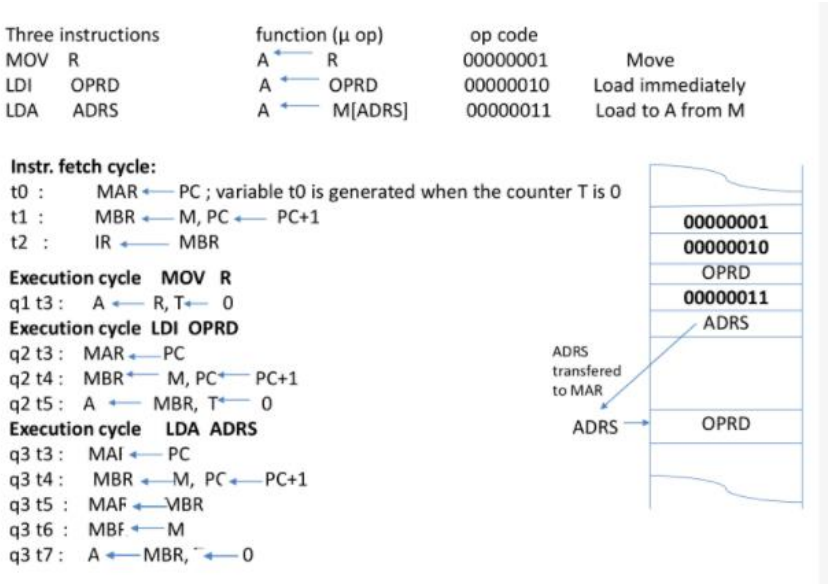


Schemat blokowy mikroprocesora który chcemy zaprojektować. Mamy rejestr PC, IR oraz pamięć, i MAR – rejestr adresowy pamięci i rejestr bufora tego co wychodzi z pamięci.

Grupa rejestrów R-A.

Kod operacji trafia do IR jest następnie deokodowany. Ponieważ mamy 8 bitów to może być 256 rozkazów. Dalej masz DMUX.

W momencie kiedy licznik liczy to na wyjściach(DMUX) pojawiają się sygnały taktujące.



Mikroprocesor musi wykonywać jakieś rozkazy.

Może on wykonywać rozkazy takie jak:

*MOV R to znaczy zawartość R prześlij do akumulatora. Jeżeli ten rozkaz pojawi się na IR to tam będzie wtedy wpisana wartość 1.

*LDI OPRD – załaduj w sposób natychmiastowy. Drugi bajt OPRD ma zostać załadowana do akumulatora.

*LDA ADRS – ładuj adres – podajesz tutaj adres komórki pamięci której wartość ma zostać załadowana do akumulatora.

Po prawej stronie widzisz kawałek pamięci, możesz zauważyć użyte operacje. I jak masz instrukcje LDI to po niej następuję wartość. W przypadku LDA widzisz adres do którego musisz przeskoczyć, żeby odczytać wartość.

Wykonanie instrukcji:

-pobierasz kod operacji z pamięci – Aby pobrać z pamięci kod operacji i wpisać go do IR a następnie podać go na decoder aby ten wymusił na układzie logicznym określonej funkcji musisz:

***PC wpisać do MAR. (PC-adres pamięci)**

***Z pamięci przesyłasz do MBR komórkę która jest wskazywana przez PC. Następnie zwiększasz PC na następny rozkaz.**

***Następnie zawartość z MBR przesyłasz do IR.**

To całę to jest pobranie kodu operacji. Nie zależnie od tego jaki kod operacj pobierasz, to zawsze wykonuje się tak samo.

Jak mikroprocesor będzie chciał wykonać rozkaz MOV R to trzeba:

*Zawatość R przesyłamy do A po czym do T wpisujesz 0 bo nie potrzebujesz kolejnych impulsów.

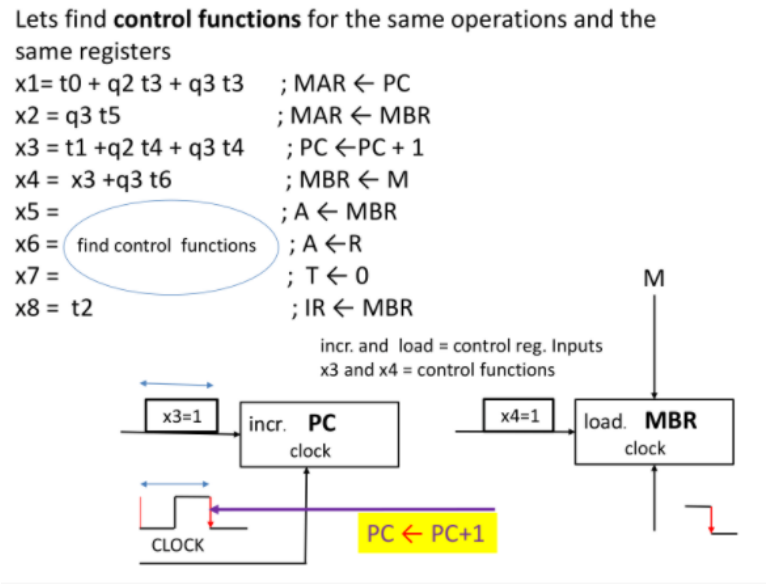
Jak będzie chiał wykonać rozkaz LDI OPRD:

- PC ładujesz do MAR // PC będzie wsazywało na OPRD
- Następnie z pamięci ładujesz do MBR i zwiększasz PC o jeden
- Z MBR ładujesz do A i zmieniasz T na 0 bo nie musisz mieć więcej taktów na tą operacje

Jak będziesz chciał wykonać LDA ADRS:

- Z PC przesyłasz do MAR // adres komórki pamięci został przesłany do MAR
- W następnym takcie z pamięci przesyłasz do MBR i zwiększasz PC
- ten adres mus zaadresować pamięć czyli z MBR przesyłasz do MAR
- następnie ładujesz do MBR to co wskazywało MAR
- i z bufora wysyłasz do akumulatora po czym T=0 bo już więcej operacji nie ma.

Czyli każdy rozkaz składa się z pobrania rozkazu (pogrubione) a potem w zależności od używanej instrukcji robisz coś innego.



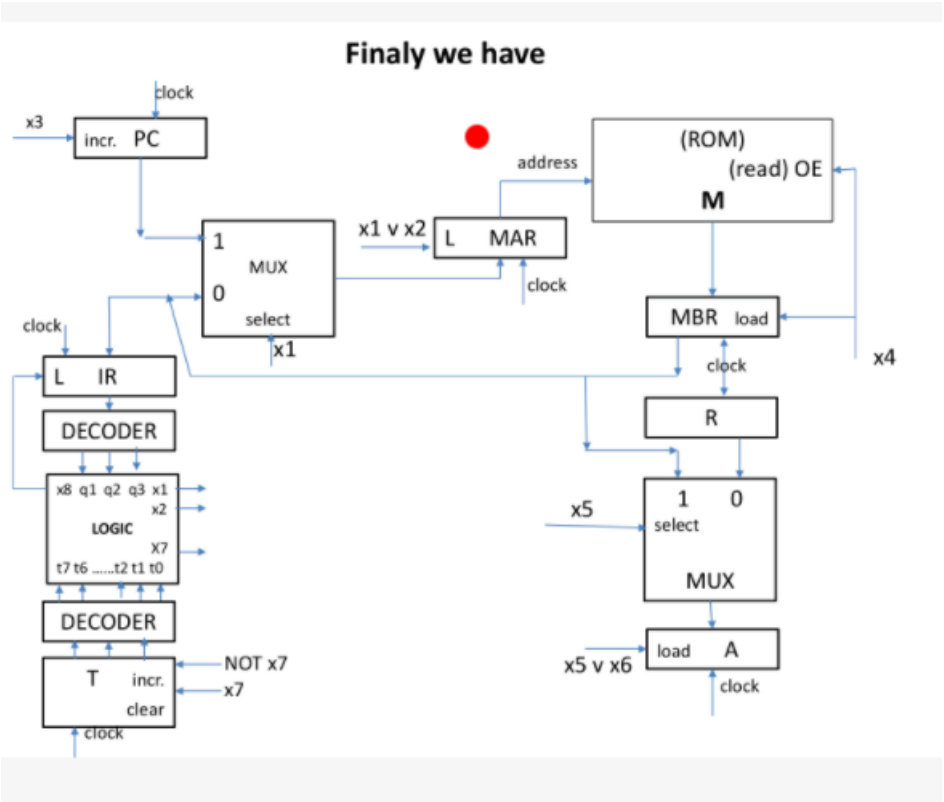
Chodzi o to że aby powiększyć PC o 1 to musisz wykonywać operacje (patrz wyżej)

albo t1 z pobrania, q2*t3 w LDI OPRD oraz q3*t4 w LDA ADDR

Sa to funkcje sterujące.

Dolny rysunek pokazuje o co chodzi. Zakładasz że PC ma wzrosnoąć o 1. I stanie się to tylko wtedy kiedy x3=1.

Drugi rysunek przedstawia kiedy z pamięci prześlesz dane do MBR wtedy kiedy x4=1



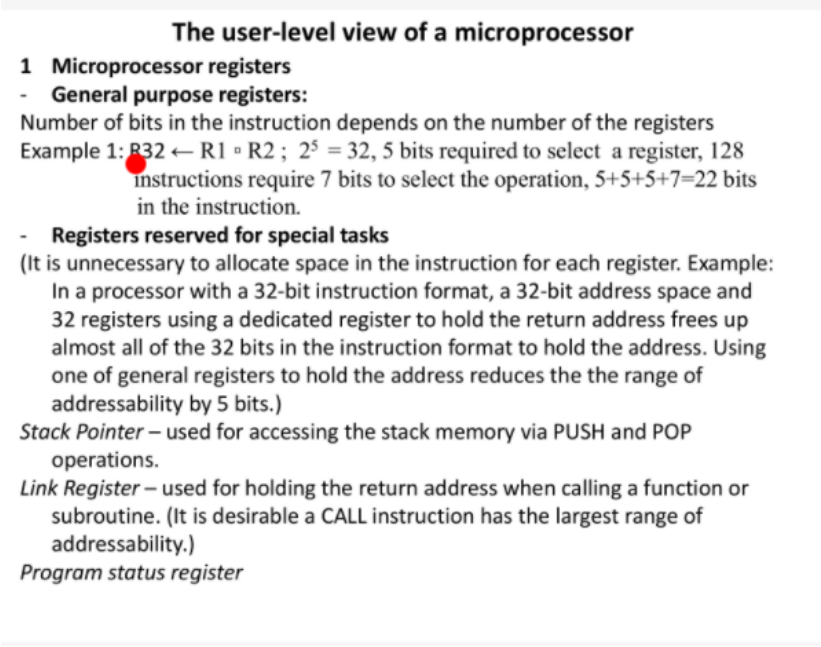
Końcowo możesz ten nasz komputer zrealizować następująco.

Na dole masz układ logiczny który generuje sumę funkcji sterujących.

Z MBR masz połączenie do IR. Jak coś jest w MBR to czeka to na wejścu IR. Jak x8 będize równe jeden i pojawi się takt zegara to zostanie SStam zapisane.

Po co multipleksery. (Pierw ten na dole) Do akumulatora możesz wpisać coś z R czy z MBR. Jeżeli x5=0 to R jest połączone z akumulatorem. Ale aby coś zapisać do akumulatora to x5 lub x6 musi być równe 1, no bo inaczej się to nie ma dokonać.

Drugi MUX decyduje czy do MAR ładujesz PC czy też MBR, I jak poprzednio aby coś załadować do MAR musi być spełniona albo funkcja x1 albo x2.



The user-level view of a microprocessor

1 Microprocessor registers

- **General purpose registers:**

Number of bits in the instruction depends on the number of the registers

Example 1: R32 ← R1 ▫ R2 ; $2^5 = 32$, 5 bits required to select a register, 128 instructions require 7 bits to select the operation, 5+5+5+7=22 bits in the instruction.

- **Registers reserved for special tasks**

(It is unnecessary to allocate space in the instruction for each register. Example: In a processor with a 32-bit instruction format, a 32-bit address space and 32 registers using a dedicated register to hold the return address frees up almost all of the 32 bits in the instruction format to hold the address. Using one of general registers to hold the address reduces the the range of addressability by 5 bits.)

Stack Pointer – used for accessing the stack memory via PUSH and POP operations.

Link Register – used for holding the return address when calling a function or subroutine. (It is desirable a CALL instruction has the largest range of addressability.)

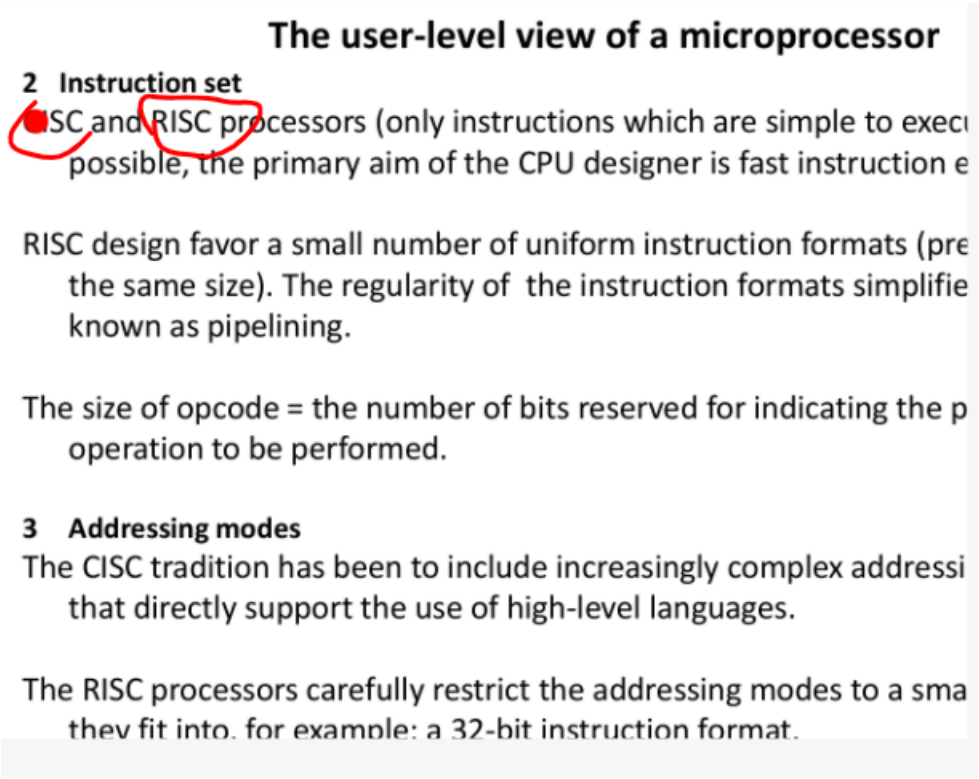
Program status register

W mikroporocesorze masz z reguły pewne rejestry ogólnego przeznaczania.

Liczba rejestrów ogólnego przeznaczenia ma wpływ na liczbę rozkazów.

Rejestry specjalne:

*Służą do zapamiętywania np. miejsca powrotnego
Np., Stack Pointer, Oraz Link Register.



The user-level view of a microprocessor

2 Instruction set

SC and RISC processors (only instructions which are simple to execu possible, the primary aim of the CPU designer is fast instruction e

RISC design favor a small number of uniform instruction formats (pre the same size). The regularity of the instruction formats simplifie known as pipelining.

The size of opcode = the number of bits reserved for indicating the p operation to be performed.

3 Addressing modes

The CISC tradition has been to include increasingly complex addressi that directly support the use of high-level languages.

The RISC processors carefully restrict the addressing modes to a sma they fit into, for example: a 32-bit instruction format.

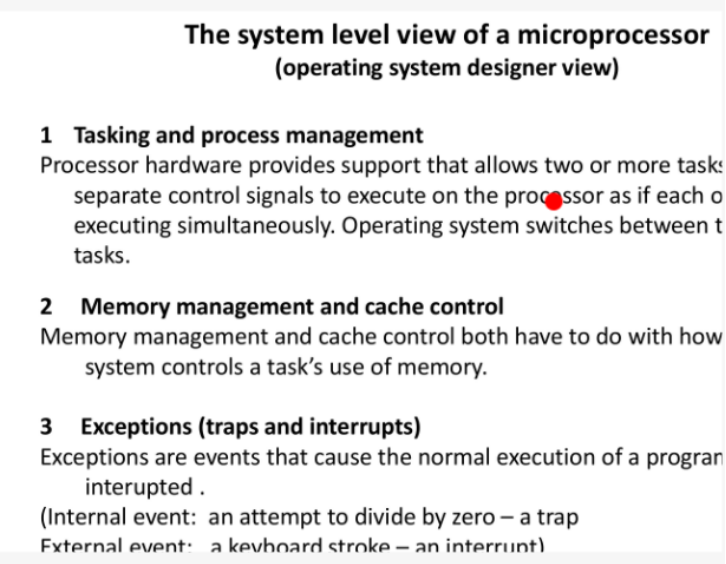
Mamy rozróżnienie na mikro CISC oraz RSC.

Na początku jak powstawały mikroprocesory, to wydawało się konstruktorom, że te rozkazy w języku maszynowym powinny odpowiadać rozkazom języków wyższego rzędu np. C. I to było dosyć skomplikowane, było trudno np. o procesory o tej samej długości rozkazów. A to utrudniało przetwarzanie potokowe. Chodzi w tym o to że aby wykonać jakiś rozkaz to pobierasz go, dekodujesz a następnie wykonujesz. Do każdej z tych części angażujesz inną część.

To wpadnięto na pomysł że w cyklu np. 3 pobierasz nowy rozkaz, dekodujesz rozkaz który był pobrany w cyklu 2 a wykonujesz rozkaz który był pobrany w cyklu 1.

Ale jeżeli każdy rozkaz miałby różną długość to to wyżej byłoby ciężkie do zrealizowania. W związku z tym zrezygnowano z mikroprocesorów CISC.

I wprowadzono RISC. W RISC bardziej skomplikowane rozkazy wykonujesz jako ciąg kilku prostszych obrazów.



The system level view of a microprocessor
(operating system designer view)

1 Tasking and process management
Processor hardware provides support that allows two or more tasks
separate control signals to execute on the processor as if each o
executing simultaneously. Operating system switches between t
tasks.

2 Memory management and cache control
Memory management and cache control both have to do with how
system controls a task's use of memory.

3 Exceptions (traps and interrupts)
Exceptions are events that cause the normal execution of a progran
interupted .
(Internal event: an attempt to divide by zero – a trap
External event: a keyboard stroke – an interrupt)

Co musimy wiedzieć, aby prawidłowo go używać? Jeżeli piszesz w C to prawie nic nie musisz o tym wiedzieć :P

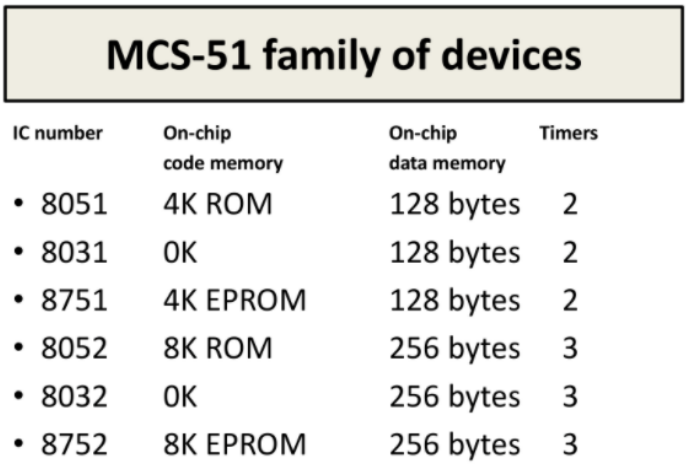
Jak chcesz pisać w assemblerze to wtedy musisz wiedzieć więcej o procesorze:

*znać rejestry

*liste rozkazów

*tryby adresów

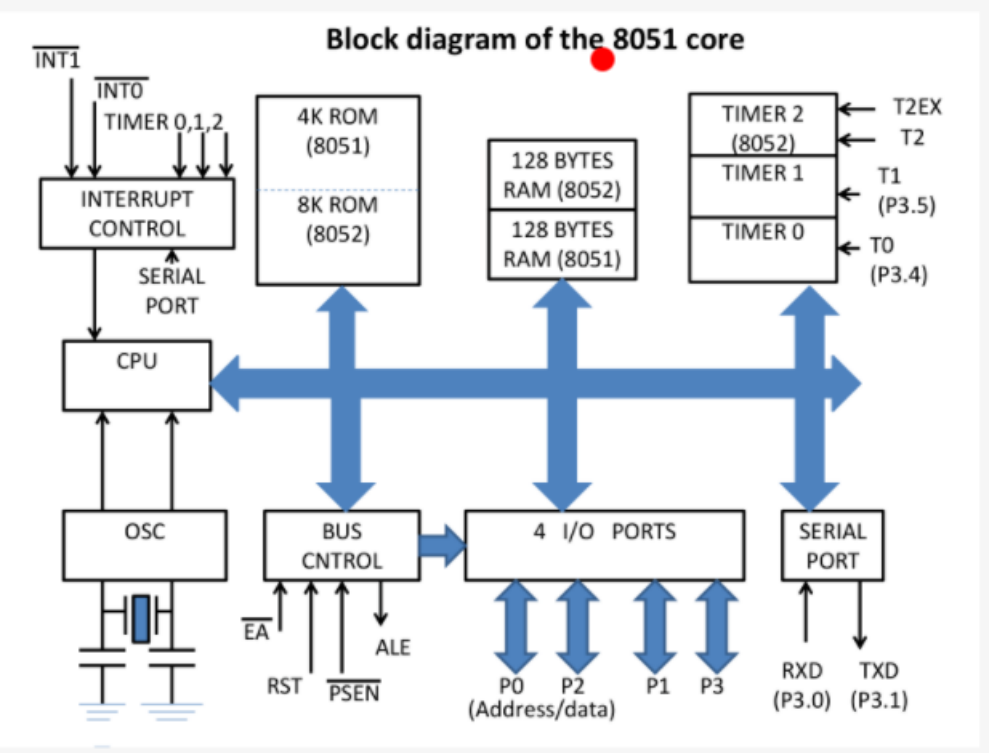
*przerwania



MCS-51 family of devices

IC number	On-chip code memory	On-chip data memory	Timers
• 8051	4K ROM	128 bytes	2
• 8031	0K	128 bytes	2
• 8751	4K EPROM	128 bytes	2
• 8052	8K ROM	256 bytes	3
• 8032	0K	256 bytes	3
• 8752	8K EPROM	256 bytes	3

8051 różni się od 8052 wielkością ROMU oraz w 8052 mamy o 1 więcej timer i w 8052 mamy większy RAM również.



Masz rdzeń CPU oraz oscylator do którego dołączasz kwarz przez kondensatory. Zakładamy że kwartz taktuje 12MHZ i po zastosowaniu dzielnika mamy 1MHZ.

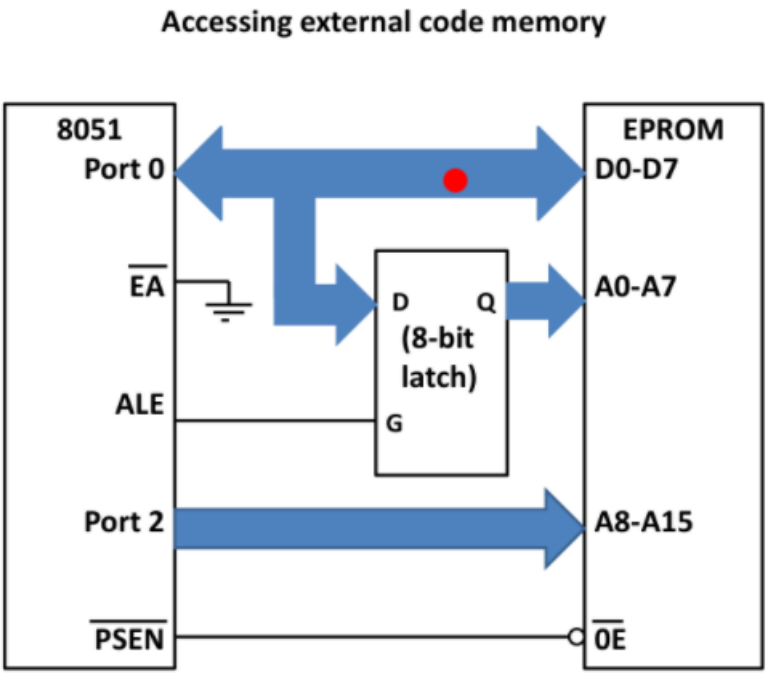
Są 3 Timery(8052). Liczniki które mogą zliczać sygnal z zewnątrz lub mogą również zliczać wewnętrzny zegar. Mamy wejście szeregowe oraz wyjście.

Masz też 4 porty 8 bitowe. Dalej w 8051 mamy wewnętrzny RAM który się składa z 1 * 128, w 8052 masz go dwa razy więcej.

Masz również sterowanie przerwaniami. INt1-0 są to przerwania zewnętrzne. Skok do przerwania następuje po zakończeniu bieżącego rozkazu. Kolejne przerwania pochodż z timerów. Możesz też mieć przerwanie od portu szeregowego

Na dole są też sygnały sterujące. RST – powoduje ponowne uruchomienie programu. Zawsze jest on włączony w momencie włączenia zasilania. RST resetuje licznik PC. (Nie jest tak w każdym mikrokontrolerze)

Kreska oznacza że sygnał jest aktywny gdy na wejście podasz 0.



EA jest podpięte do 0. To wówczas zewnętrzny dostęp mówi nam że nie używamy wewnętrznego ROMU. Używamy wtedy tylko EPROM.

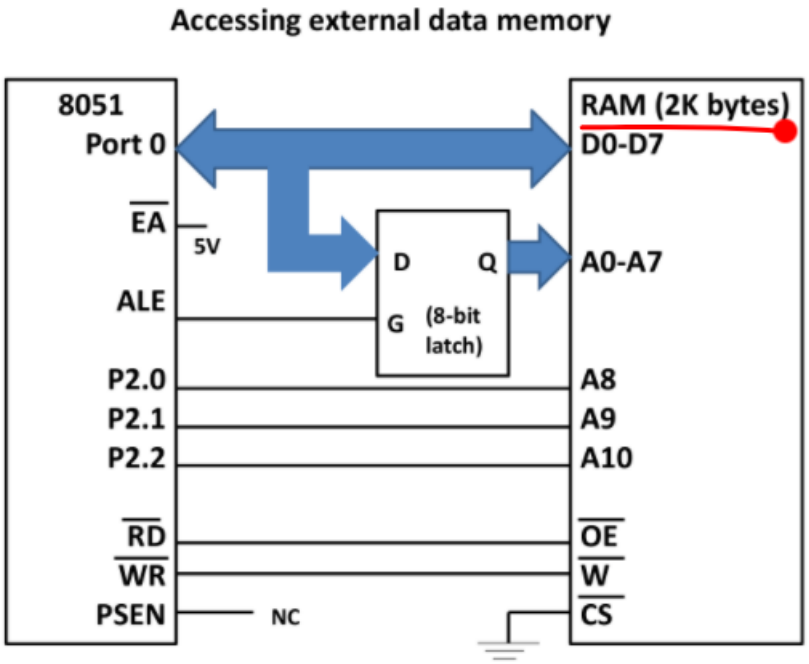
ALE –

w eprom masz po 8 bitów adresów. Te mniej znaczące A0-A7 pojawiają się na wyjściu E0.(PAMIETAJ ŻE DO EPROM NIE MOŻESZ NIC ZAPISAĆ!!!). Cała pamięć EPROM ma 64Kbajtów.Do zaadresowania całej pamięci potrzebujesz 16 bitów.

Po podaniu pierwszych 8 bitów używasz ALE po to żeby zatrzasnąć od A0 do A7. Bo tam do góry masz port wejściowy dla danych.

Na P2 masz zawsze adres, natomiast na P0 albo generujesz adres, albo może być portem wejściowym/wyjściowym dla danych.

PSEN – jak masz 0 i ustawiłeś adresy to eeprom generuje wartość jakiejś komórki pamięci danych.



Pamięć maksymalna RAM może wynosi 64K.

Przez to że używamy tu 2K to używasz 11 bitów.

I znowu od A0-A7 masz do adresu, oraz Rejestr portu 2 wyjście 0 (P2.0) oraz pozostałe.

ALE działa tak samo

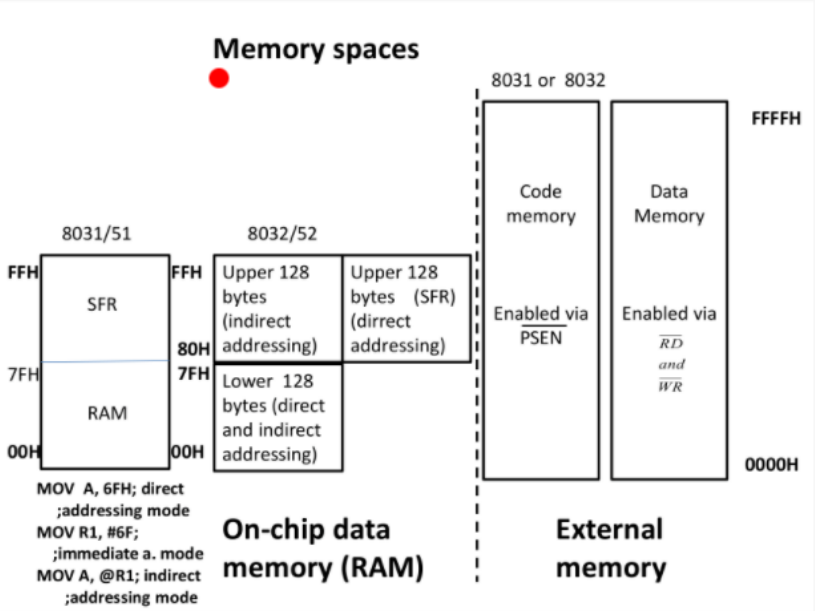
EA – korzystasz z programu który jest w EPROMIE wewnętrznym.

PSEN – też jest wyzwalany 0. NC – no connection.

RD – służy do odczytu

WR-zapis.

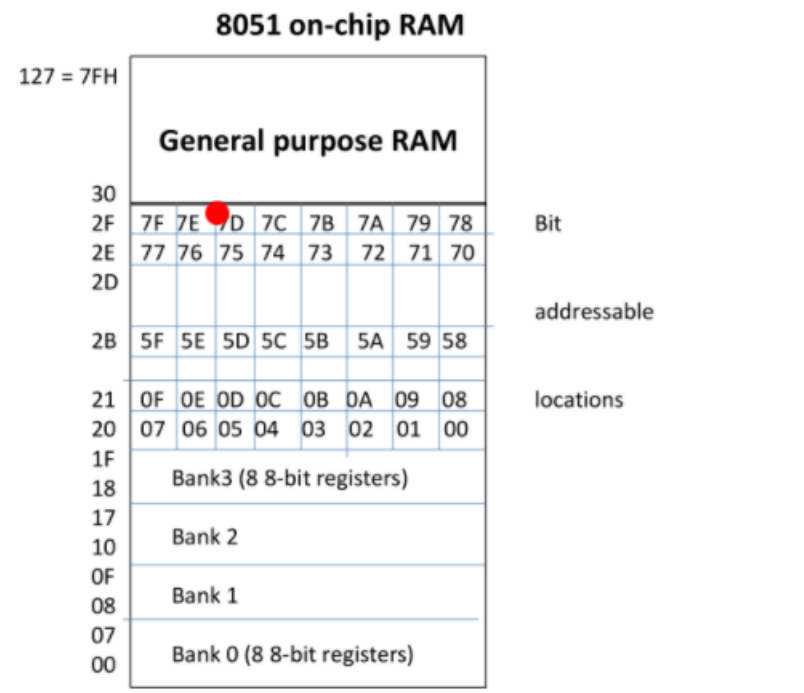
CS- służy do wyboru której pamięci używamy. Bo możesz mieć wybór z jakiego układu pamięci korzystasz z połączonych.



Masz pamięć do której możesz podłączyć pamięć zewnętrzną. Odpalasz je za pomocą odpowiednich sygnałów. RAM będzie użyty jeżeli użyjesz sygnału RD lub WR.

Pamięć wewnętrzna którą mieliśmy poprzednio (skrajnie lewa) jest zaznaczona poniżej. To RAM jest od 00H do 7FH. W zależności od mikroprocesora to masz RAM 128 bajtów i SFR.

I wtedy przy 8052 masz RAM i SFR od 80H do FFH? JAk to jest możliwe? Tajemnica leży w trybie adresowania.



Tutaj masz tą dolną część RAMU. Ta cała pamięć jest podzielona na 3 części. Na dole masz od 0 do 7 rejestry (R0-R7). I to jest BANK 0

tak samo wygląda Bank 1, 2, 3. Po co to?

Jak z programu głównego skaczesz do jakiegoś podprogramu i coś tam robisz, i potem wracasz to w programie głównym musisz mieć dokładnie te same info w rejestrach.



Nie ma straty czasu, z zapisem danych na stos.

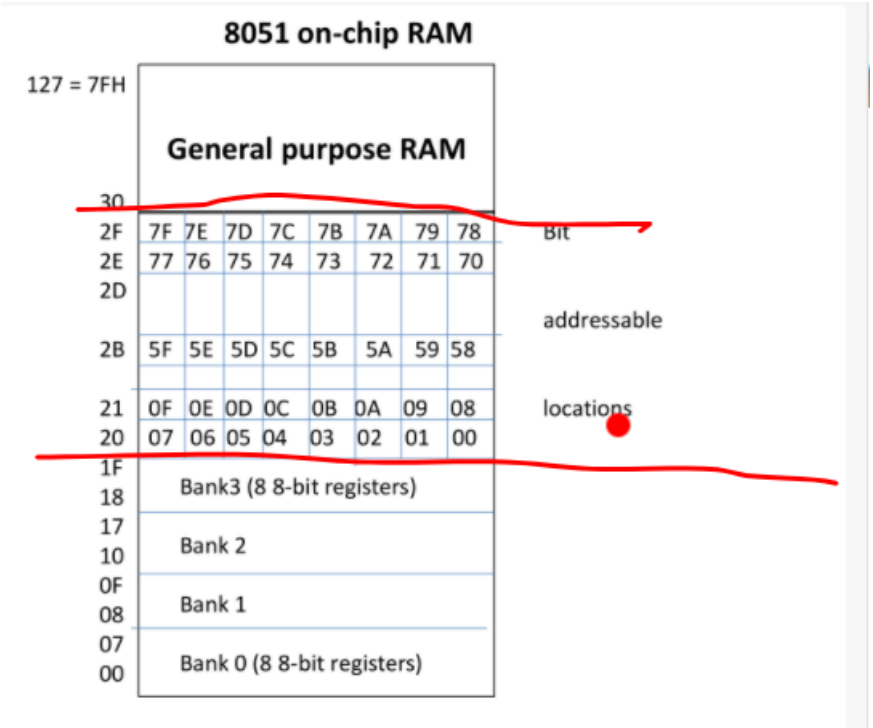
Skąd wiadomo w jakim rejestrze jesteśmy? Wynika to z użycia PSW.

Symbol	Name	Address
*ACC	Accumulator	OEOH
*B	B Register	OFOH
*PSW	Program Status Word	ODOH
SP	Stack Pointer	81H
DPTR	Data Pointer 2 Bytes	
DPL	Low Byte	82H
DPH	High Byte	83H
*PO	Port 0	80H
*P1	Port 1	90H
IP2	Port 2	OAOH
*P3	Port 3	OBOH
*IP	Interrupt Priority Control	OB8H
*IE	InterruptEnable Control	OA8H
TMOD	Timer/Counter Mode Control	89H
*TCON	Timer/Counter Control	88H
*+ T2CON	Timer/Counter 2 Control	OC8H
THO	Timer/Counter 0 High Byte	8CH
TLO	Timer/Counter 0 Low Byte	8AH

* = Bit addressable
+ = 8052only

SPECIAL FUNCTION REGISTERS

Jest on 8 bitowy. Jest pod adresem 0d0H. Jest to rejestr wskaźników. Oznacza on z którego banku korzystamy.



Kolejna część (każdą komórkę możesz zaadresować). Taka przestrzeń jest w każdym mikrokontrolerze. Możemy się np. umówić że bit 00 steruje ruchem silnika.

Pobranie czegoś z tego będzie wyglądało następująco:

MOV A, 2CH //wpisuje **bajt** pod adresem

ORL A, #100000000

MOV 2ch, A //przesylasz akumulator

zamiast tych wszystkich operacji możesz dać SETB 67H

Co oznacza adresowanie dirrect i indirrect?

MOV A, 5FH; to jest przykład, gdzie używasz dirrect. Do akumulatora przesyłasz zawartość komórki 5FH.

MOV R0, #5FH ;ADRESOWANIE NATYCHMIASTOWE, TZN ŻE TĄ LICZBĘ W SPOSÓB NATYCHMIASTOWY PRZERZUCASZ DO R0.

//NOTATKI Z ZAJEC!!!

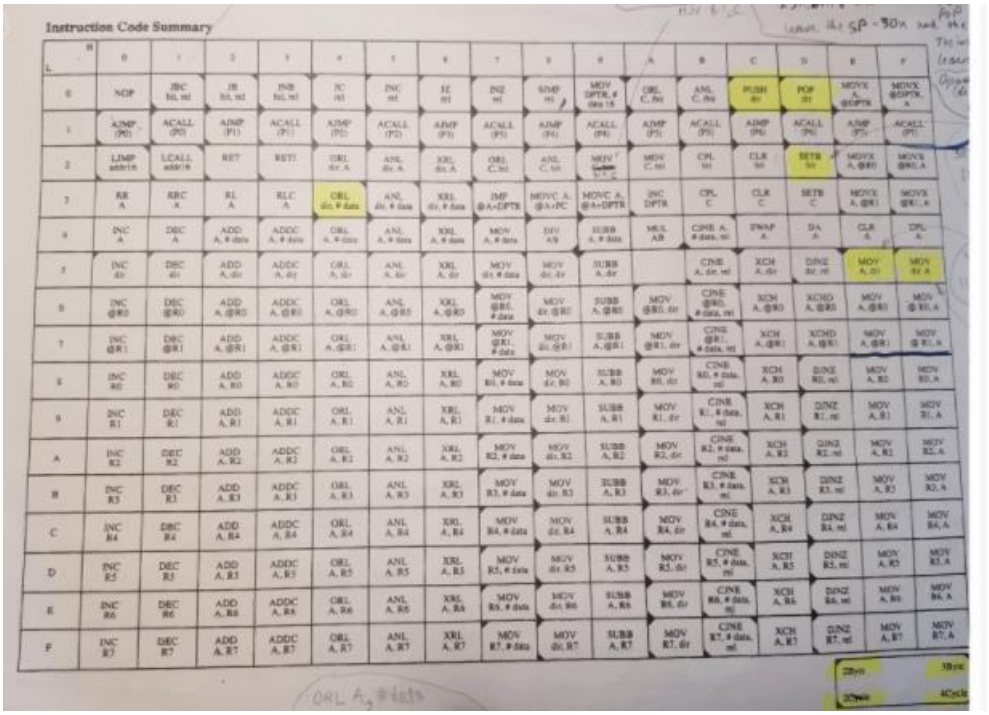
Symbol	Name	Address
*ACC	Accumulator	OEOH
*B	B Register	OFOH
*PSW	Program Status Word	ODOH
SP	Stack Pointer	81H
DPTR	Data Pointer 2 Bytes	
DPL	Low Byte	82H
DPH	High Byte	83H
*PO	Port 0	80H
*P1	Port 1	90H
IP2	Port 2	OAOH
*P3	Port 3	OBOH
*IP	Interrupt Priority Control	OB8H
*IE	InterruptEnable Control	OA8H
TMOD	Timer/Counter Mode Control	89H
*TCON	Timer/Counter Control	88H
*+ T2CON	Timer/Counter 2 Control	OC8H
THO	Timer/Counter 0 High Byte	8CH
TLO	Timer/Counter 0 Low Byte	8AH

* = Bit addressable
+ = 8052only

SPECIAL FUNCTION REGISTERS

Wszystkie rejestry specjalne. Tam gdzie masz * tzn że masz dostęp do każdego poszczególnego bitu. Możesz wtedy zapisać np.

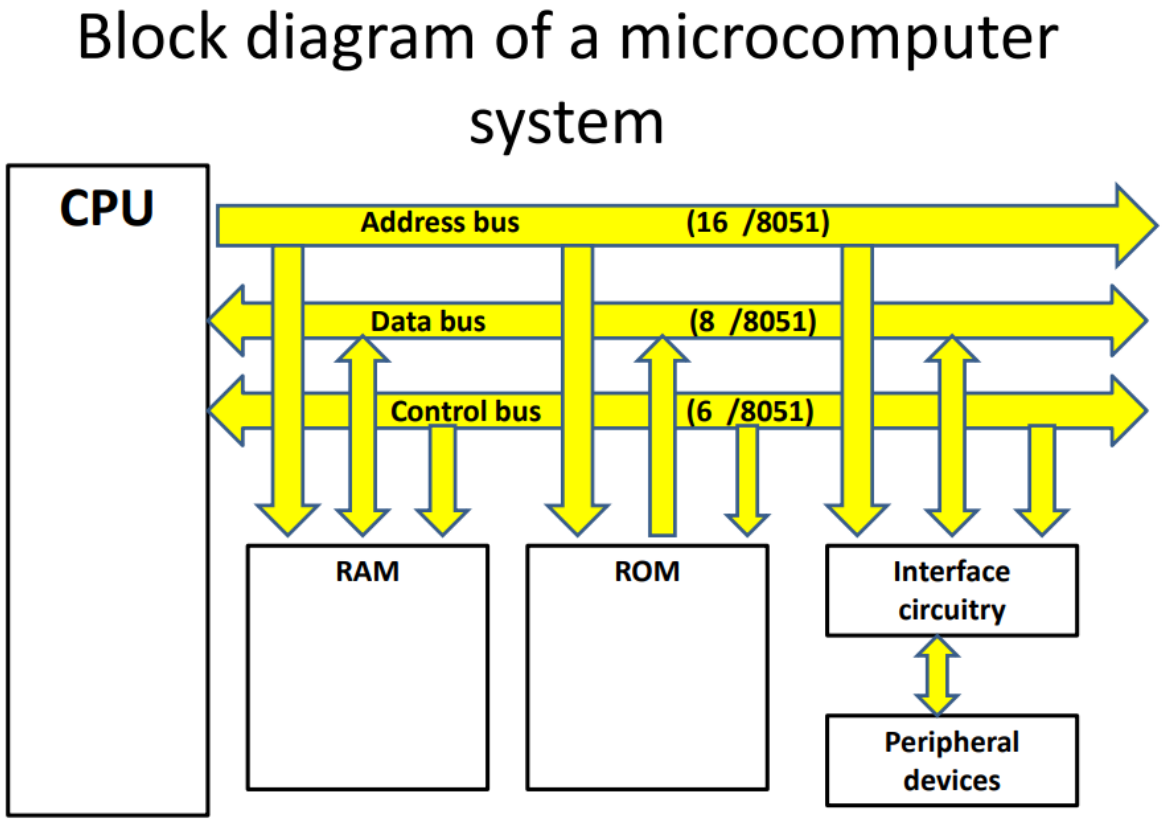
SETB P1.2



Masz listę rozkazów. Jesst ich łącznie 256. Poszczególne cyfry do góry i na dole pozwalają ustalić OPCODE danej operacji. I np. MOV A, DIR ma OPCODE E5.

Wykład 2 (Łukasz):

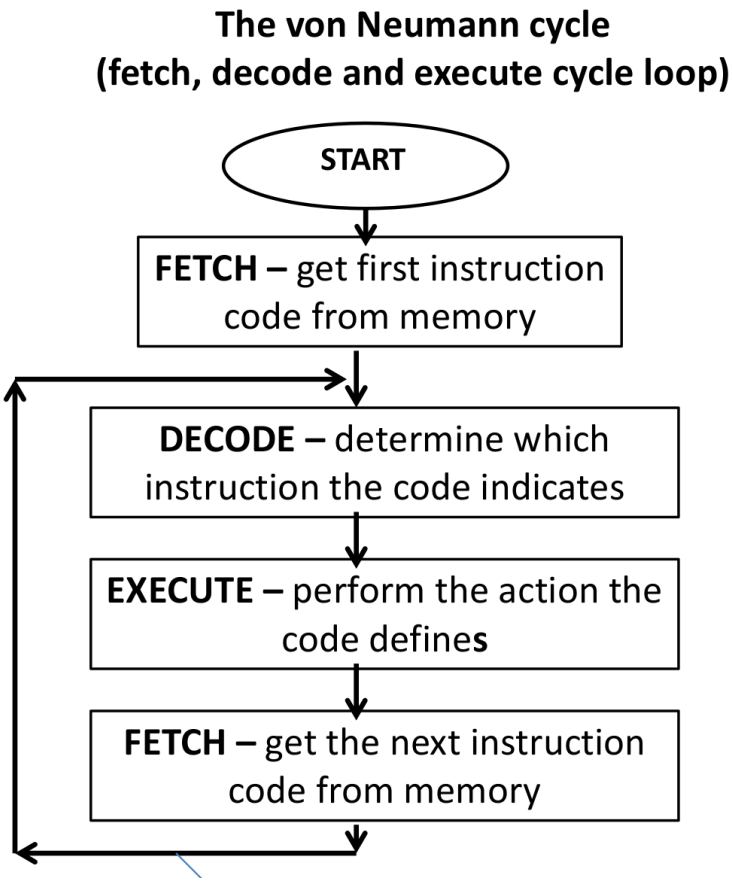
Diagram blokowy mikrokontrolera



W przypadku procesora 8051:

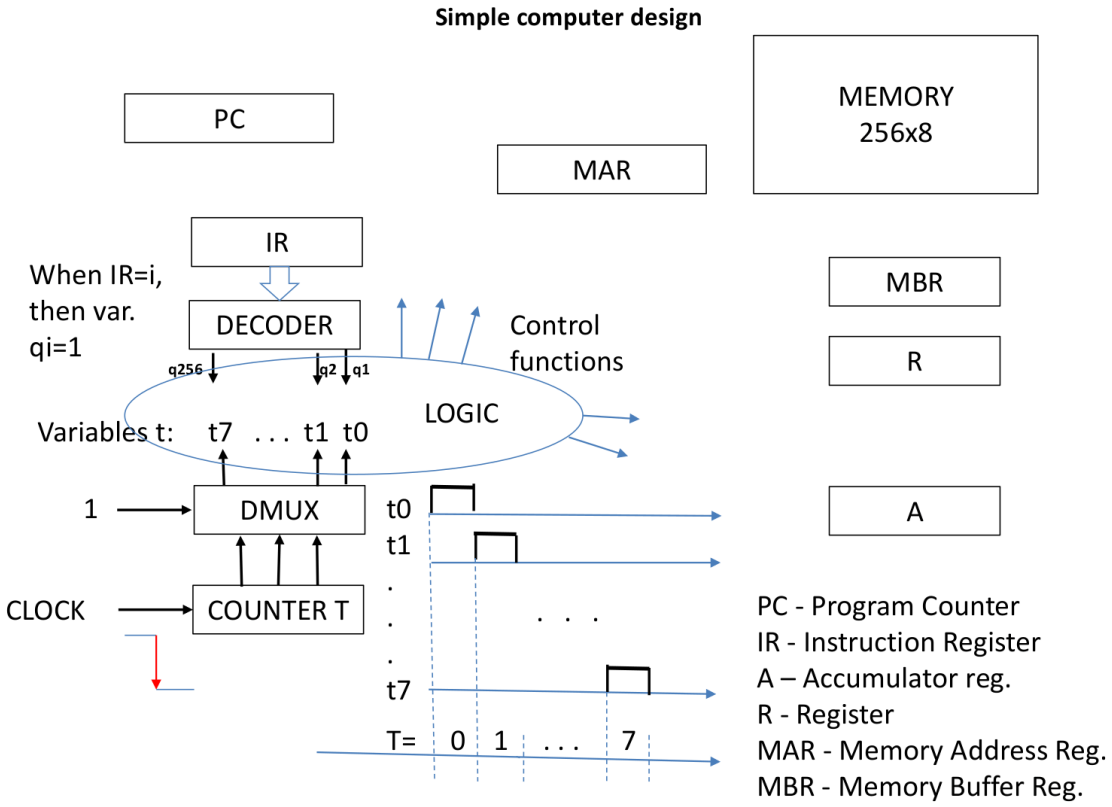
- Szyna adresowa
  - 16 bitowa,
  - przestrzeń adresowa pamięci RAM i ROM wynosi po 2^16=65536 Bajtów
- Szyna danych
  - 8 bitowa
- Szyna kontrolująca
  - 6 bitowa

Wykonywanie programu:



FETCH - pobranie rozkazu z pamięci (rozkaz może być jednym słowem 16,32 lub 64 bitowym)
DECODE - dekodowanie rozkazu
EXECUTE - wykonanie rozkazu

Przykład projektowania systemu mikrokontrolera:



PC - Program Counter - zawiera adres do pamięci (inkrementowany każdorazowo po wykonaniu instrukcji i jednocześnie w momencie opadającego zbocza zegara taktującego)

IR - Instruction Register - zawiera pobrany rozkaz z pamięci (256 wyjść, bo DSM51 może mieć maksymalnie 256 rozkazów)

MAR - Memory Address Register - zanim pamięć zostanie zaadresowana, to adres musi znaleźć się w tym rejestrze

MBR - Memory Buffer Register - w momencie zapisu/odczytu z pamięci dane przechodzą przez ten bufor

R - rejestr (np. R1 - w tym przykładzie jest tylko jeden)

Rozkaz z pamięci jest przepisywany do IR. Po zdekodowaniu na jednym z 256 wyjść dekodera pojawia się “1” - każdy rozkaz ma swoje wyjście, które jest połączone z układem logicznym.

Bloki DMUX i COUNTER_T tworzą generator impulsów taktujących.

Układ logiczny na podstawie zdekodowanego rozkazu i impulsów taktujących generuje funkcje sterujące, które zarządzają mikrooperacjami w całym układzie.

Analiza wykonywania przykładowego programu:

Three instructions	function (μ op)	op code	
MOV R	A ← R	00000001	Move
LDI OPRD	A ← OPRD	00000010	Load immediately
LDA ADRS	A ← M[ADRS]	00000011	Load to A from M

Instr. fetch cycle:

t0 : MAR ← PC ; variable t0 is generated when the counter T is 0

t1 : MBR ← M, PC ← PC+1

t2 : IR ← MBR

Execution cycle MOV R

q1 t3 : A ← R, T ← 0

Execution cycle LDI OPRD

q2 t3 : MAR ← PC

q2 t4 : MBR ← M, PC ← PC+1

q2 t5 : A ← MBR, T ← 0

Execution cycle LDA ADRS

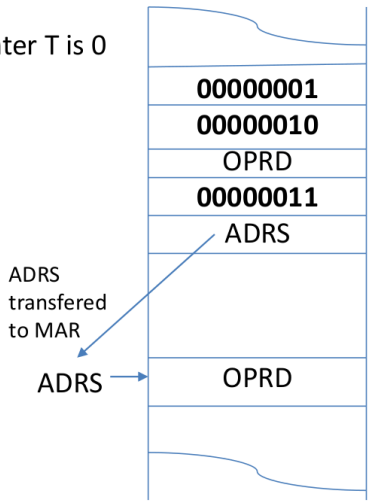
q3 t3 : MAR ← PC

q3 t4 : MBR ← M, PC ← PC+1

q3 t5 : MAR ← MBR

q3 t6 : MBR ← M

q3 t7 : A ← MBR, T ← 0



Instrukcje w kolejności wykonywania:

Instrukcja	Kod operacji	Opis
MOV R	00000001	zapisz wartość rejestru R do akumulatora
LDI OPRD	00000010	załaduj w sposób natychmiastowy liczbę ORPD (operand) do akumulatora
LDA ADRS	00000011	zawartość komórki pamięci o adresie M pobierz do akumulatora

Zawartość rejestru rozkazów:
00000001 - wykonaj 1 instrukcję
00000010 - wykonaj 2 instrukcję
OPRD - operand wpisany w pamięci
00000011 - wykonaj 3 instrukcję
ADRS - adres adresujący komórkę w pamięci, której zawartość ma być przypisana do A
Pobranie kodu operacji jest dla każdego rozkazu takie same.

Mikrooperacje:

Wykonywana jest 1 instrukcja na 1 impuls taktujący.

Instr. fetch cycle:

t0 : MAR ← PC ; variable t0 is generated when the counter T is 0
t1 : MBR ← M, PC ← PC+1
t2 : IR ← MBR

ti - kolejne impulsy taktujące. Zawsze mają wartość 1.
Wartość PC (patrz strona wyżej) jest przepisywana MAR. Następnie zawartość pamięci M jest przepisywana do MBR. W tym samym czasie wartość PC jest zwiększana o 1. Ostatecznie dane z MBR trafiają do IR.

Execution cycle MOV R

q1 t3 : A ← R, T ← 0 q1=1 - - wykonaj rozkaz 1

Execution cycle LDI OPRD

q2 t3 : MAR ← PC q2=0
q2 t4 : MBR ← M, PC ← PC+1 q2=0
q2 t5 : A ← MBR, T ← 0 q2=1 - wykonaj rozkaz 2

Execution cycle LDA ADRS

q3 t3 : MAR ← PC q3=0
q3 t4 : MBR ← M, PC ← PC+1 q3=0
q3 t5 : MAR ← MBR q3=1 - wykonaj rozkaz 3
q3 t6 : MBR ← M
q3 t7 : A ← MBR, T ← 0

qi ==1, gdy kod i-tej operacji został już pobrany. W przeciwnym wypadku qi=0.

Jeśli (qi and ti+3)==1, to wykonywana jest i-ta instrukcja (np. (q1 and t3)=1 -> wykonaj 1 instrukcje)

Bit qi oznacza, że poprzednie impulsy taktujące t zostały już wykorzystane - poprzednia instrukcja się w pełni wykonała.

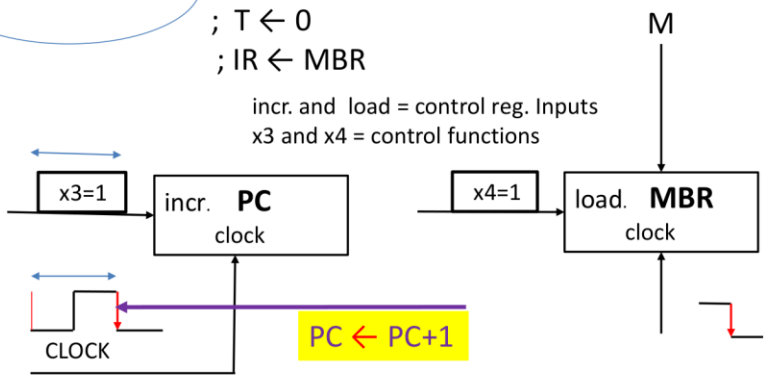
Po wykonaniu 1 instrukcji następuje pobranie kodu operacji 2 rozkazu. Następnie bit q2=1 itd.

Inkrementacja PC (Program Counter - patrz wyżej) odbywa się każdorazowo po wykonaniu instrukcji.

Generowanie funkcji kontrolnych x

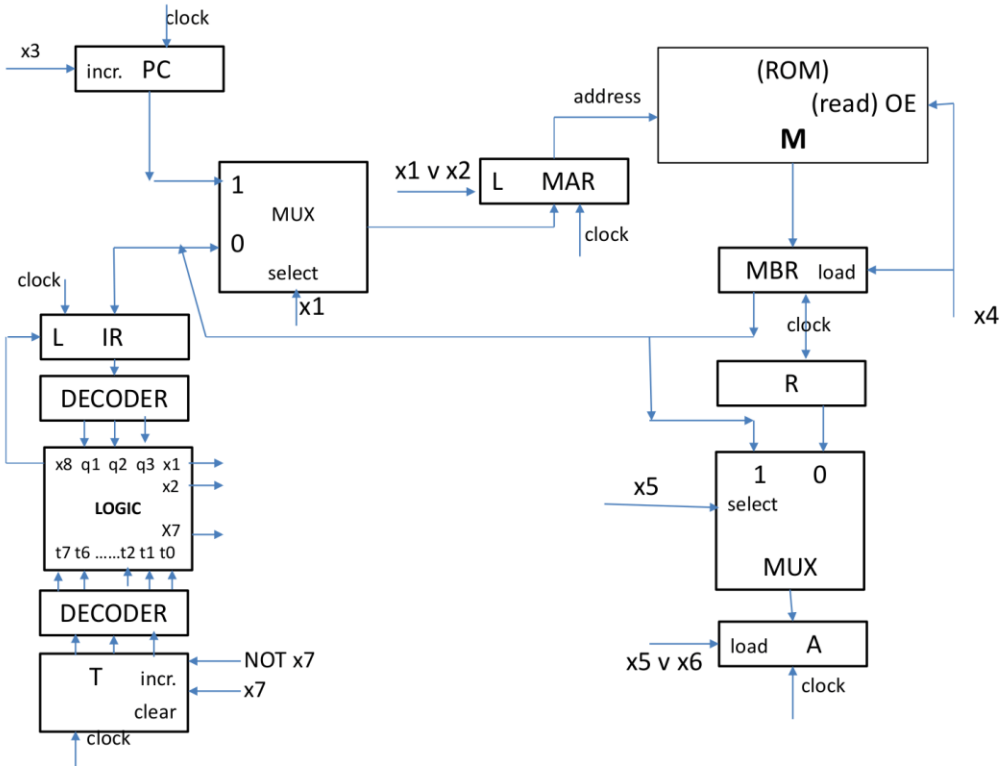
Lets find **control functions** for the same operations and the same registers

x1= t0 + q2 t3 + q3 t3 ; MAR ← PC
x2 = q3 t5 ; MAR ← MBR
x3 = t1 +q2 t4 + q3 t4 ; PC ←PC + 1
x4 = x3 +q3 t6 ; MBR ← M
x5 = ; A ← MBR
x6 = find control functions ; A ←R
x7 = ; T ← 0
x8 = t2 ; IR ← MBR



Każda funkcja kontrolna jest wyznaczana inaczej dla każdej mikrooperacji.
Jeśli bit x3=1, to następuje inkrementacja PC na opadającym zboczu sygnału zegara.
Bit xi i-tej funkcji kontrolnej steruje danym elementem układu.

Finaly we have



Np. Wejście x8 dekodera jest połączone z wejściem ładowania L rejestru rozkazów IR, ponieważ x8=t2, co oznacza, że tylko jeśli nastąpi 2 impuls taktujący, to ma nastąpić przepisanie zawartości rejestru MBR do IR.
Rejestry dzielą się na:

- Ogólnego przeznaczenia - liczba bitów instrukcji zależy od liczby rejestrów np. Jeśli jest 32 rejestry (potrzeba 5 bitów by zapamiętać jego adres) oraz 128 instrukcji (wymaga 7 bitów by wybrać operację) to 5+5+5+7=22 bity przypada na jedną instrukcję.
- Zarezerwowane tylko do specjalnych operacji np.
  Stack Pointer (SP) - wskaźnik stosu - adresowanie stosu (instrukcje PUSH,PULL)
  Link Register - służy do przechowywania adresu powrotnego podprogramu w momencie wywołania funkcji (instrukcja CALL). Wskazuje wierzchołek stosu.

  Rejestry Specjalne:

Symbol	Name	Address
TH1	Timer/Counter 1 High Byte	8DH
TL1	Timer/Counter 1 Low Byte	8BH
+TH2	Timer/Counter 2 High Byte	OCDH
+TL2	Timer/Counter 2 Low Byte	OCCH
+RCAP2H	T/C 2 Capture Reg. High Byte	OCBH
+RCAP2L	T/C 2 Capture Reg. Low Byte	OCAH
ISCON	Serial Control	98H
SBUF	Serial Data Buffer	99H
PCON	Power Control	87H
DPH		83H
DPL		82H
SP		81H
P0	Port 0	80H

*** = Bit addressable**
+ = 8052 only

SPECIAL FUNCTION REGISTERS

Architektury rozkazów:
- CISC - instrukcje proste do wykonania - używane kiedy liczy się szybkość układu
- RISC - instrukcje krótkie ot tej samej długości bitowej.

Tryby adresowania:

- Pośrednie - np. MOV A,@R1 - zapis do A danych komórki pamięci o adresie R1 (wskaźnik)
- Bezpośrednie - np MOV A,R7
- Natychmiastowe
- Relatywne

The MCS-51 instruction set
Addressing modes

● Register addressing (e.g. , **ADD A,R2** or **MOV A,R5**)

Opcode (5 bits)	n	n	n

Another example : **ADD A,R7**; opcode is 2FH, three LSBits are 1 1 1

● Direct addressing (e.g., **MOV TH1,A** or **MOV 8DH,A**;

The direct address of TH1 (8DH) is the second byte of the instruction. The accumulator is specified implicitly in the opcode.

Opcode (8 bits)	Direct address (8 bits)

Another example : **ORL A, direct**; direct is an address in internal RAM

● Immediate addressing (e.g., **MOV A, #6FH** ; (A) ← #6FH)

Immediate operand, preceded by #, is loaded into the accumulator

0 1 1 1 0 1 0 0	**0 1 1 0** 1 1 1 1

● Indirect addressing – register R0 or R1 operates as pointer register indicating an address in RAM where data are written or read. (e.g., **MOV @R0,A**; ((R0)) ← A)

1 1 1 1 0 1 1 i	((Ri)) ← A

```
Example
        MOV   RO, #60H        ; immediate
LOOP:   MOV   @R0, #0         ; indirect
        INC   R0              ; register
        CJNE  R0, #80H, LOOP  ; R0, #data, rel=indirect address -128... +127,
; encoding 10111rrr ddddddddd eeeeeeee , the first operands are compared,
; the relative address (offset) is an 8bit signed value which added to the PC forms
; the address (label) of the next instruction executed.
        (continue)
```

Procesory z architekturą RISC ograniczają tryby adresacji do małych segmentów, by zmieściły się one do danego formatu instrukcji np 32 bitowego.

Adresowanie rejestru:
Bity n oznaczają numer rejestru.

Adresowanie bezpośrednie:
Kod operacji : 0 1 1 1 0 1 0 0
Adres bezpośredni: 0 1 1 0 1 1 1 1

W adresowaniu pośrednim można używać tylko rejestrów R0 i R1.

Ostatni bit kodu operacji dotyczy używanego rejestru (0-> R0, 1->R1).

Adresowanie relatywne (np. SJMP TABLE - 4 bajty):

The MCS-51 instruction set
Addressing modes

- Relative addressing - used with certain jump instructions

A relative address, an 8-bit signed value (+127 to -128) is added to the PC incremented to the address following the jump instruction.
(E.g., SJMP TABLE ; if the label TABLE represents location 100FH, and the instruction SJMP TABLE is in memory at locations 0FFEH and 0FFFH, the assembler will assign a relative offset equal 0FH as byte 2 of the instruction.

opcode	relative offset = 0FH

- Absolute addressing – used with the ACALL and AJMP instructions. It allows branching within the current 2K page of code memory.

A10 –A8	opcode(5 bits)	ADDR7 – ADDR0

- Long addressing – used with the LCALL and LJMP instructions

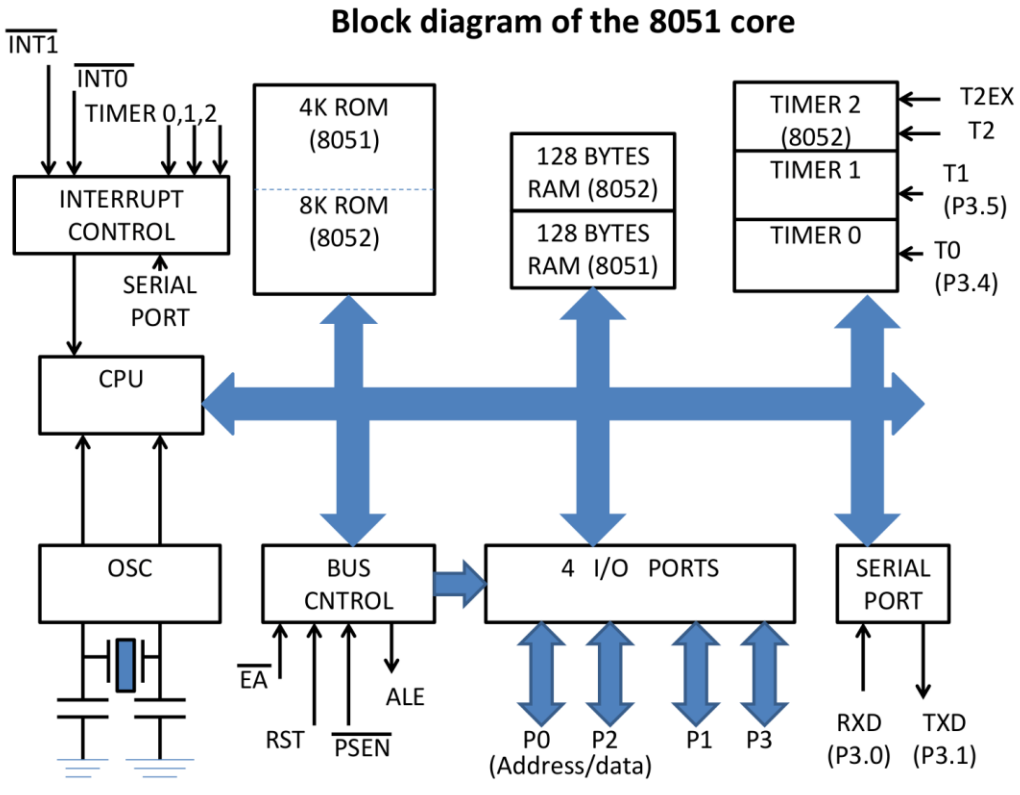
The 3-byte instruction contains one-byte opcode and two-byte destination address.

- Indexed instruction – either the PC or DPTR and the accumulator form the effective address for a JMP or MOVC instruction

E.g., JMP @A+DPTR ; (PC) ← (A) + (DPTR) , add the 8-bit unsigned contents of A ; with the 16-bit pointer, and load the resulting sum to the PC

MOVC A, @A+PC ; (PC) ← (PC)+1 , 1 byte, 2 cycles
(A) ← ((A) +(PC))

Diagram blokowy układu 8051:



Sygnał EA - External Access

!EA=0, gdy chcemy skorzystać z pamięci zewnętrznej (! - negacja). W momencie zapisu/odczytu z pamięci wewnętrznej !EA=1.

!PSEN - sygnał połączony z pamięcią odczytujący coś z pamięci EPROM.

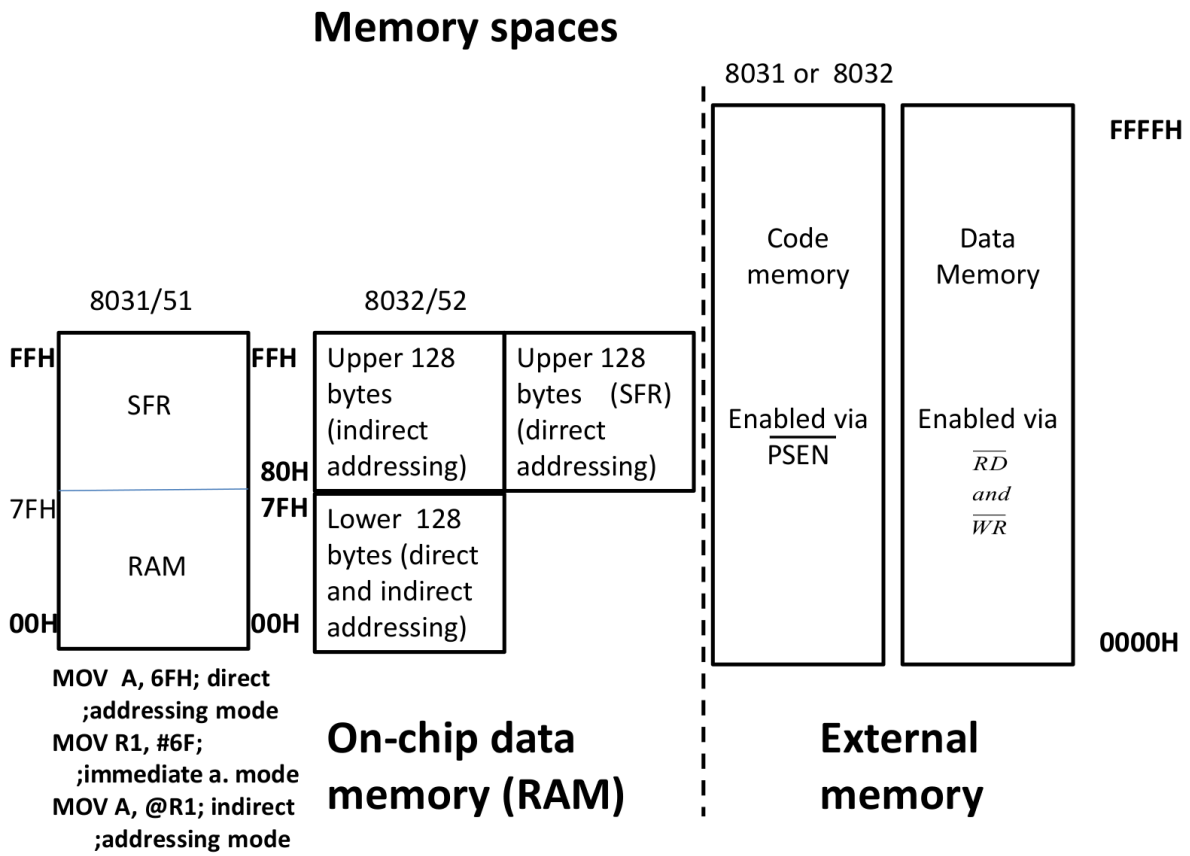
RST - RESET - może resetować PC (Program Counter) w momencie włączenia zasilania (wówczas rozpoczyna się wykonywanie programu z pamięci zewnętrznej ROM od adresu pamięci równym 0).

P0,P1,P2,P3 - porty 8 bitowe
Port P2 generuje potrzebny adres do wykonania programu
Port P0 jest multipleksowany (albo generuje adres jak P2, albo jest portem wejściowym/wyjściowym dla danych).

INTERRUPT CONTROL:
!INT0, !INT1 - przerwania zewnętrzne
SERIAL_PORT - przerwanie z portu szeregowego

OSC - oscylator - zazwyczaj 12MHz

Pamięć mikroprocesora:



Pamięć zewnętrzna:

Zakres pamięci zewnętrznej = 16 bitów (0000 - FFFF).
Przestrzeń adresowa = 2^16=65536 B

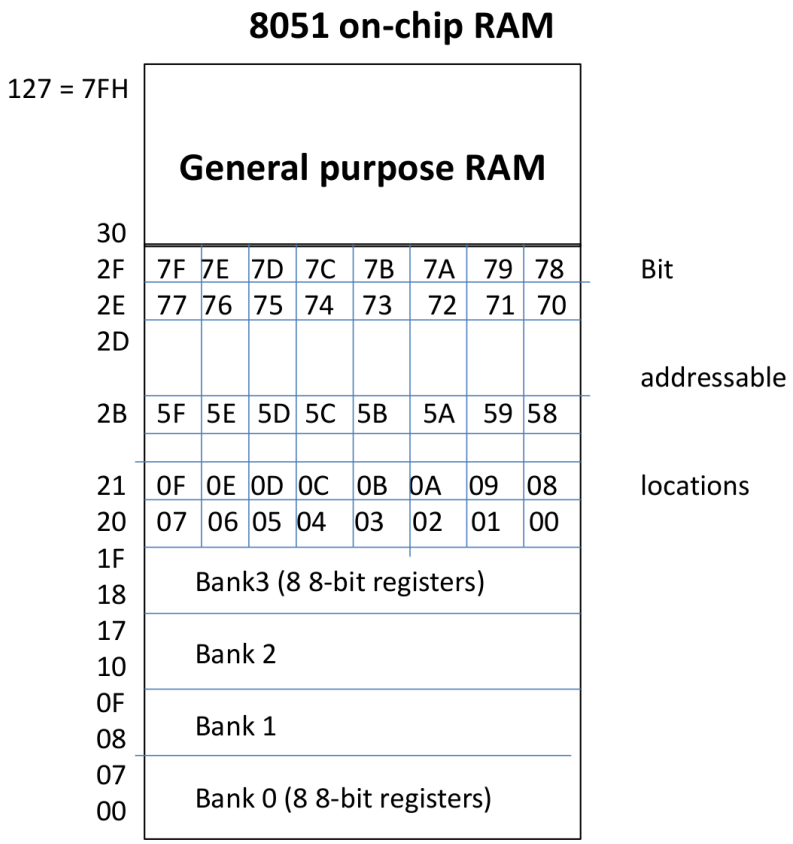
!PSEN - Program Store Enable - sygnał odczytujący coś z pamięci EPROM (Code Memory).

!RD,!WR - sygnały używanie w momencie wykonania operacji na pamięci danych.

Pamięć wewnętrzna:

W zakresie 80 - FF jest albo pamięć RAM albo rejestry specjalne (SFR).

Pamięć RAM (zakres 00 - 7F):



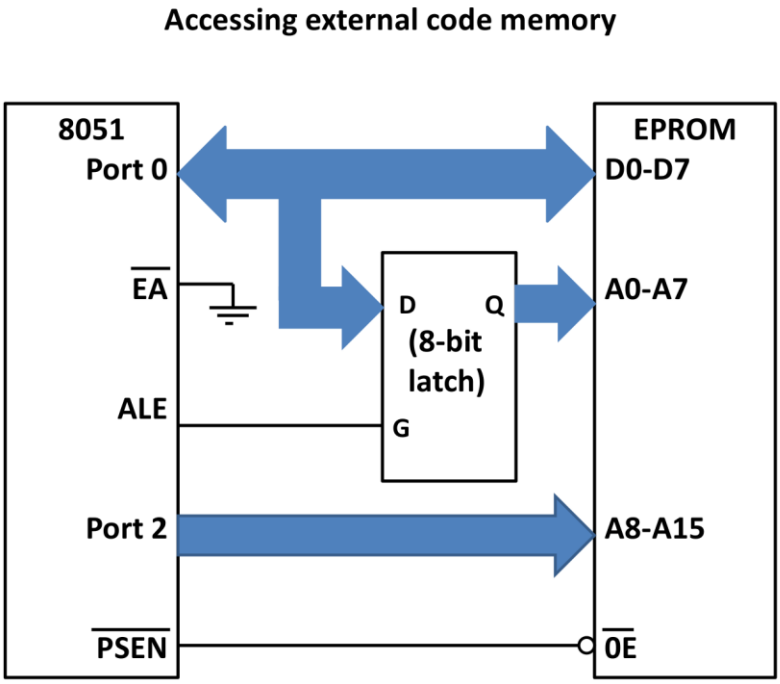
Rejestr PSW korzysta z 2 bitów RS1,RS0 przy wyborze konkretnego banku rejestrów do użytku. Jest rejestrem specjalnym.

RS1/RS0	0	1
0	Bank 0	Bank 1
1	Bank 2	Bank 3

Każdy bank składa się z 8 rejestrów 8 bitowych (R0-R7).

Banki rejestrów służą do efektywnego podziału pamięci.
Overflow (przepełnienie) występuje, gdy flaga C==1 (carry) na najbardziej znaczącym bicie. Wówczas wynik może wyjść przekłamany.
Bit P - Parity - bit parzystości.

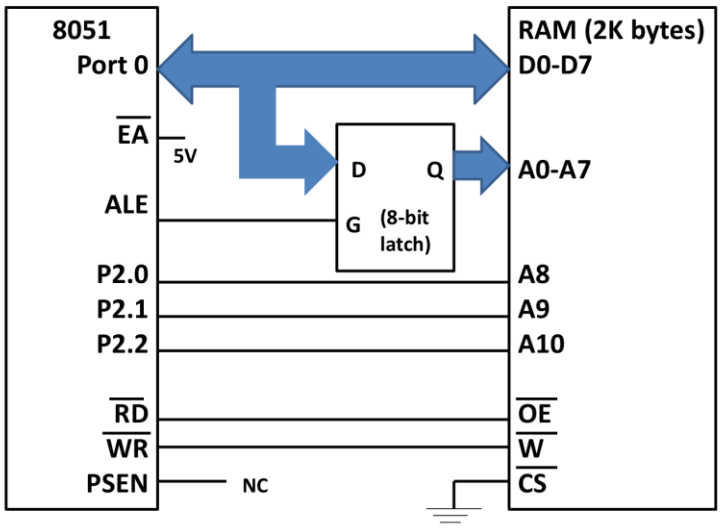
Dostęp do pamięci zewnętrznej EPROM:



Adres na porcie 0 musi być zapamiętany w rejestrze 8 bitowym (przerzutnik D), który jest wyzwalany sygnałem ALE (Address Large Enable).
A0 - A7 - 7 bitów mniej znaczących z portu 0.
A8 - A15 - bardziej znaczące bity z portu 2.
Gdy zapis bitów A0 - A15 adresu zostanie wykonany, to port 0 przechodzi w tryb zapisu/odczytu danych z pamięci EPROM.

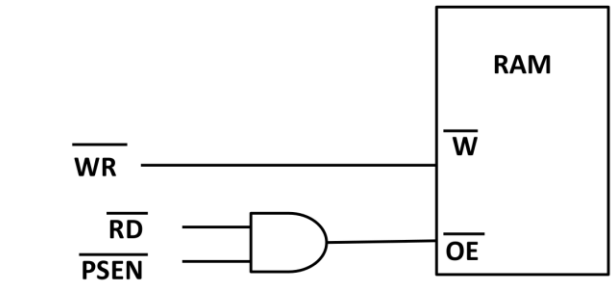
Dostęp do pamięci zewnętrznej RAM:

Accessing external data memory



!CS - Chip Select - wybór kości pamięci RAM (w przypadku gdy jest ich więcej). Gdy tak jak w tym przypadku mamy 1 kość pamięci RAM to wejście !CP jest dołączone do uziemienia (zawsze pojawi się 1 na tym wejściu co będzie sygnalizować użycie tej pamięci RAM).

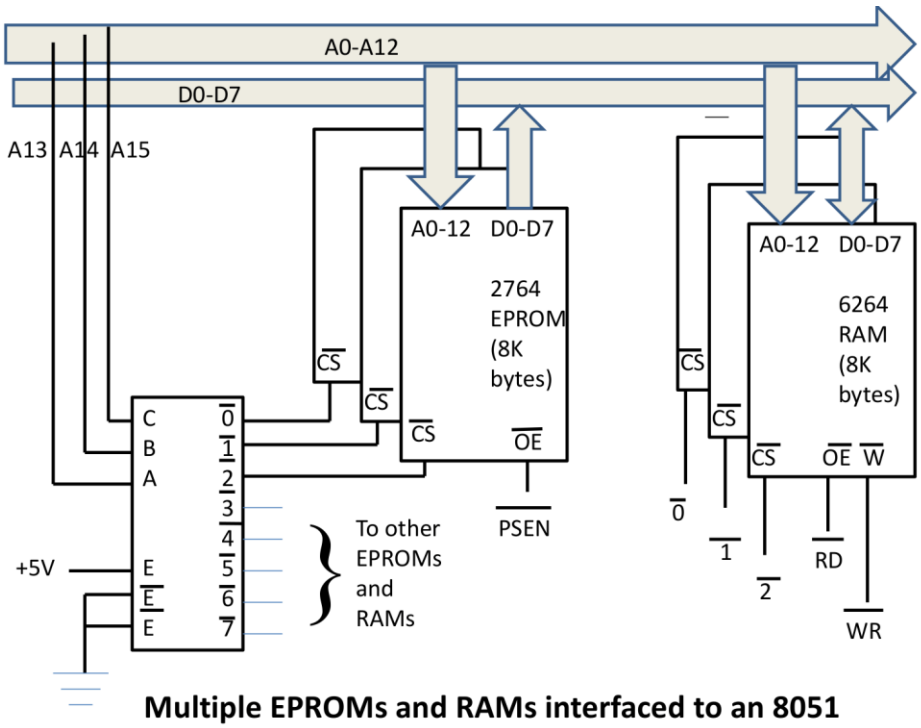
Overlapping the external code and data spaces



PSEN is used to read code memory and RD is used to read data memory. RAM can occupy code and data memory space by connecting its OE line to the logical AND of PSEN and RD. The RAM can be written to as data memory, and read as data or code memory. Thus a program can be loaded into the RAM and executed.

Program jest trzymany w pamięci RAM. Odczytywanie jego instrukcji odbywa się poprzez wyjście PSEN (Program Store Enable).
Jeśli na !OE (Output Enable) jest 1, to następuje odczyt danych.

Wiele pamięci zewnętrznych podłączonych do mikroprocesora:



Multiple EPROMs and RAMs interfaced to an 8051

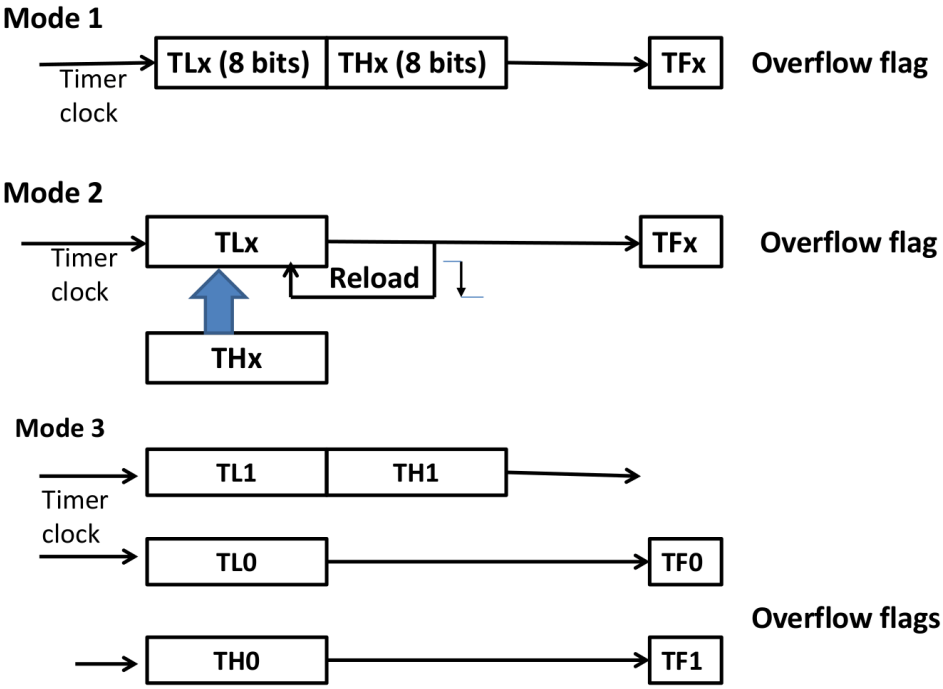
Wejścia CBA decydują, które wyjście jest aktywne.

Rodzaje instrukcji:

- Arytmetyczne np. ADD itp
- Logiczne np. ANL,ORL itp
- Transferu danych np. MOV
- Boolowskie np. CLR, CLP,SET itp.
- Program brenching

Timery:

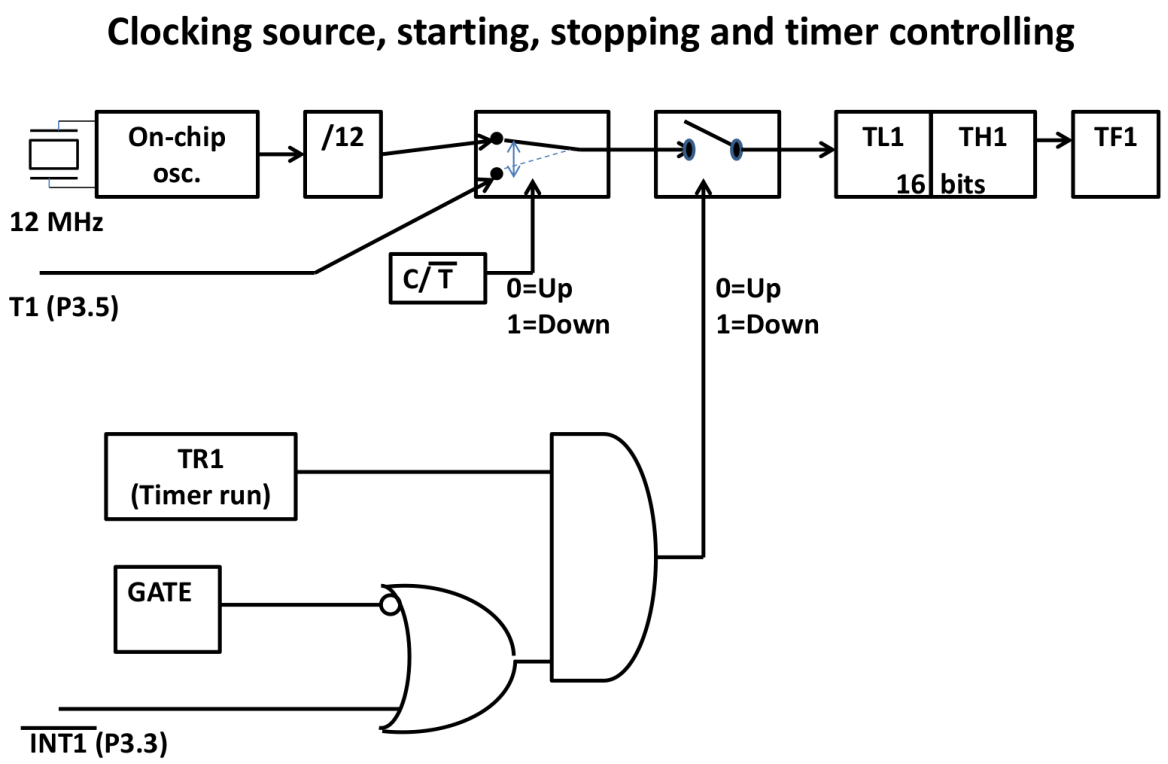
Timer modes and the overflow flag



Tryby pracy:

- Mode 1 - Jeden timer 0 lub 1 tworzy 16 bitowy licznik podzielony na 2 części TLx i THx. Gdy licznik TLx będzie zawierał same 1, to inkrementowany jest licznik THx i zerowany TLx. Gdy THx zawiera same 1, to na następnym zboczu opadającym flaga overflow=1.
- Mode 2 - w momencie, gdy na TLx są same 1, to następuje załadowanie TLx wartością obecną w THx i TLx liczy od tej wartości THx od nowa aż do samych 1 itd (reload).
- Mode 3 - Wykorzystywane są oba timery. Timer 1 jest pozbawiony flagi przepełnienia, natomiast mamy 2x 8 bitowy licznik TL0 i TL1 każdy ze swoją flagą przepełnienia TF.

Zegar timera:



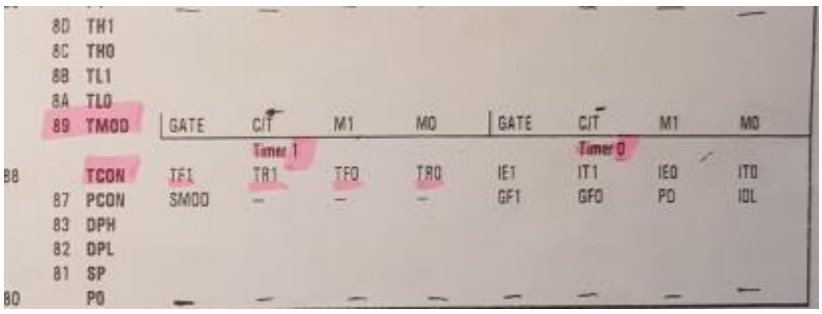
Timer 1 operating in mode 1

Sygnał z licznika C/!T steruje przełącznikiem.

Zegar timera może być jako:

- Sygnał 12MHz jest dzielony na 12 części po 1MHz. Sygnał 1MHz wchodzi na przełącznik a potem na timer.
- W tej sytuacji każdy timer ma swoje wejście zegarowe.

Elementy zegara (patrz rys. wyżej) znajdują się w komórkach pamięci w zakresie adresów 89 - 8D.



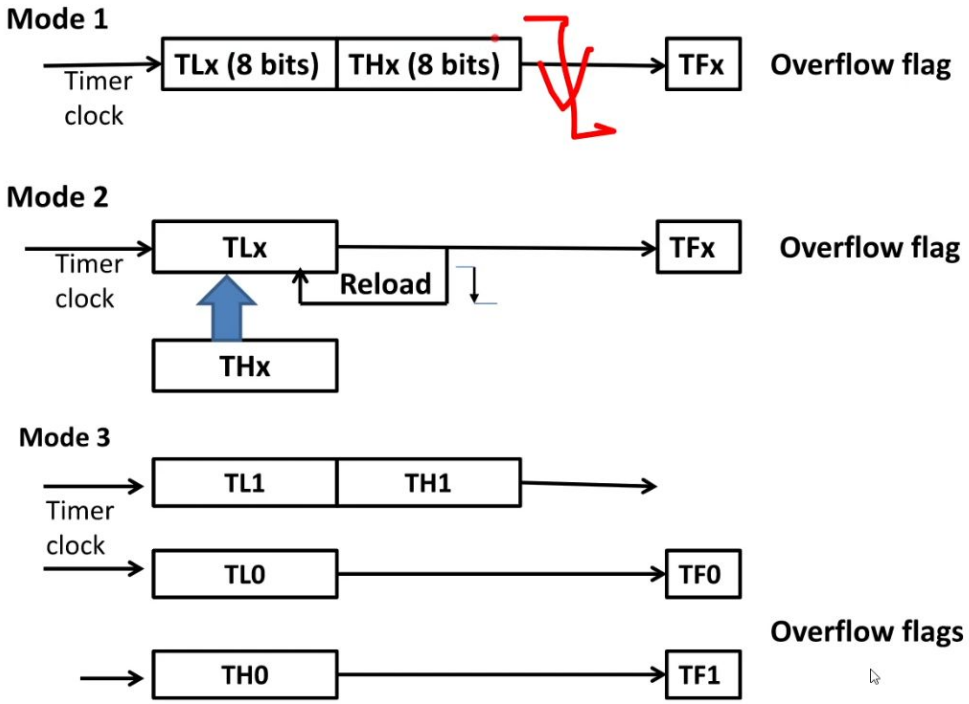
Adres	Timer

8D	TH1
8C	TH0
8B	TL1
8A	TL0
89	TMOD - służy do inicjalizacji timera

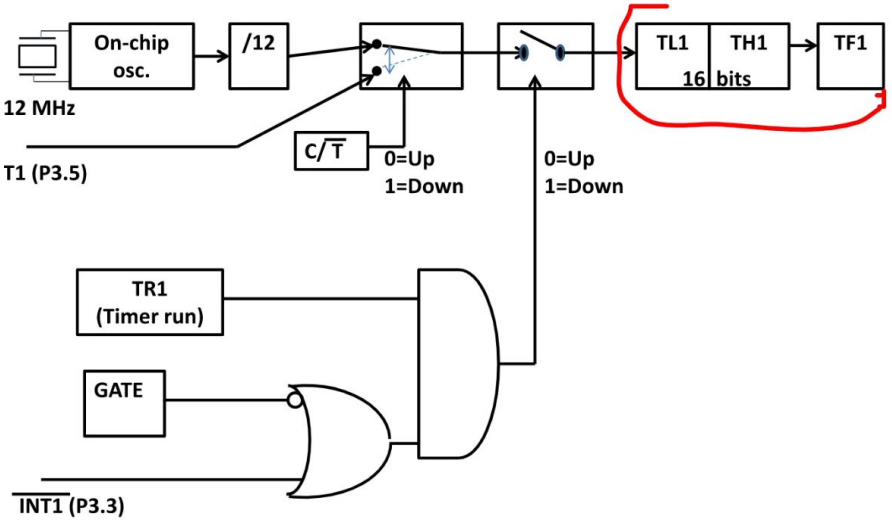
Inicjalizacja timera może wyglądać np tak: MOV TMOD, #00010000B

Bity w kolejności od prawej:
0 - GATE = 0
0 - wartość C/!T = 0
01 - tryb pracy = mode 1
Itd.

Timer modes and the overflow flag



W modzie 1 z samych jedynek przechodzi w stan 0 to powoduje ustawienie flagi overflow.
Mod 2 jest z przeładowaniem i działa na opadające zbocze powoduje to że z rejestr THx wpisuje się na TLx i zlicza od nowa.
Mod 3 niby działa tak że TL0 jest podłączony do TF0, TH0 jest podłączony do flagi TF1 ,a TL1 i TH1 nie jest w ogóle podłączone.



Timer 1 operating in mode 1

W tym przypadku jest w trybie 1. Przełącznik umożliwia sygnał wejściowy do licznika.Pochodzi on albo z wew. oscylatora w f=12Mhz ,czyli co 1 mikrosekundę albo sygnał pochodzi z GATE i TR1 i INT1.

Initializing and accessing timer registers

```
MOV     TMOD, #0001000B,  timer 1, 16-bit timer (mode 1), C/notT=0, GATE=0 internal clocking
MOV      TL1, #9C
MOV     TH1, #0FFH;          FF9CH = -100D
SETB    TR1;                 overflow flag is set after 100µs
```

To stop the timer and clear the flag:

```
CLR     TR1
CLR     TF1
```

Pulse wave generation:

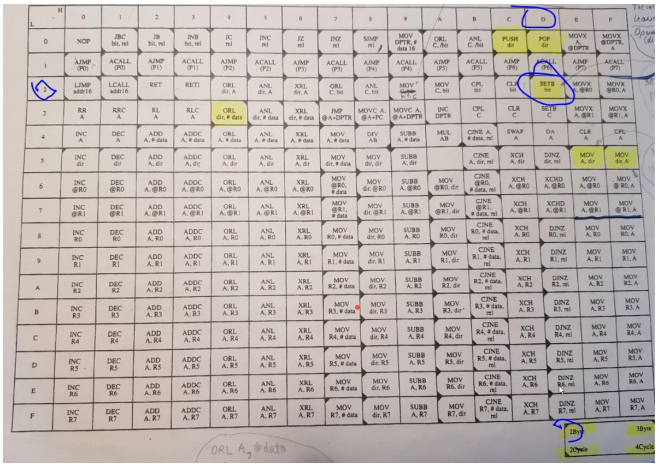
```
        ORG     8300H
LOOP:   SETB    P1.0
        CLR     P1.0
        SJMP    LOOP
        END
```

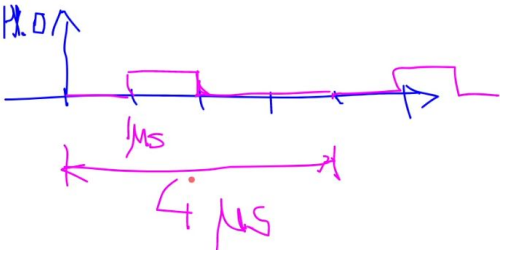
10 kHz square wave on P1.0

```
        ORG     8300H
        MOV     TMOD,  #02H;        8-bit auto-reload mode, Timer 0
        MOV     TH0,  # -50;  TH0 = -50
        SETB    TR0
LOOP:   JNB     TF0,LOOP; Jump if Bit Not set
        CLR     TF0
        CPL     P1.0
        SJMP    LOOP
        END
```

1 program inicjuje i dostęp do rej timera
MOV TMOD, #0001000B, timer 1 #tak powinno być ten adres nawiązuje do wart. TMOD
MOV TL1, #9C bo 3 najmniej znaczące bity zostawiamy a resztę na przeciwny zmieniamy.
SETB TR1 po 100mikros flaga bd ustawiona

2 program zeruje flagi
3 program
ORG 8300H - mjejsce w pamięci gdzie ma być początek programu
SETB P1.0 - ustawiamy adresowi P1(adresowi P1) najmniej znaczący bit na przeciwny-1us
CLR1.0 - 1us
SJMP LOOP - 2us
END - ma byc przetłumaczony na język maszynowy

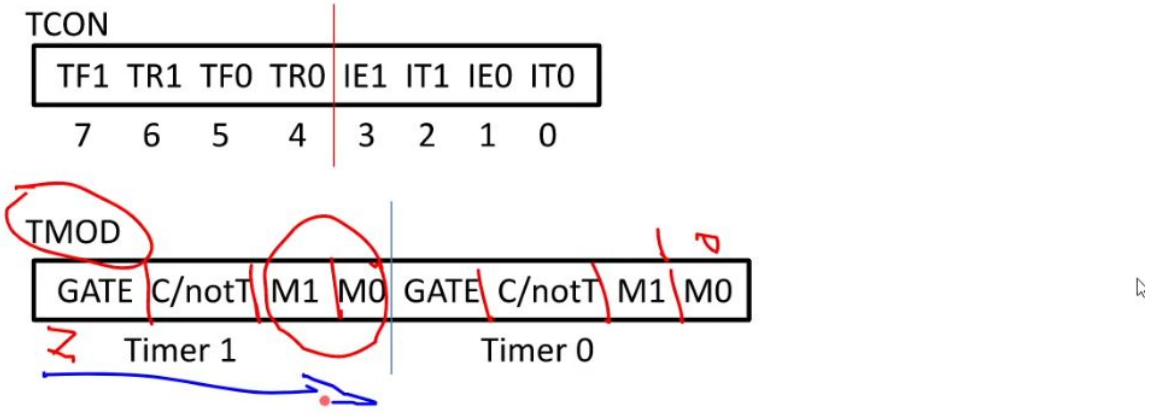




4 program - 10 kHz prostokątny na P1.0
ORG 8300H
MOV TMOD, 02H; 00000010-tryb pracy autoroload
MOV TH0, #-50;10kHz->0.01ms=100us // co 50us zmieniamy Stan TR0
SETB TR0;ustaw na 1
LOOP:
JNB TF0.LOOP ;TF0 co 50us ustawione na 1
CLR TF0
CPL P1.0; zmiana na znak przeciwny
SJMP LOOP
END

1 kHz square wave

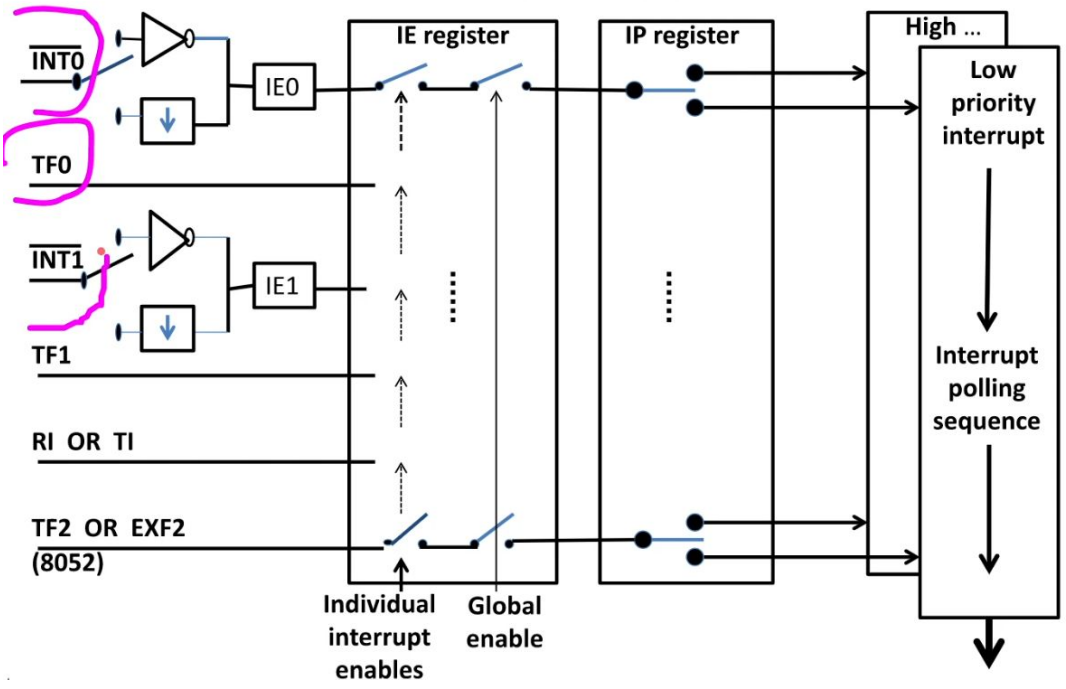
```
        ORG     8300H
        MOV     TMOD, #01H      ;16-bit t. mode, mode 2 cannot be used
                                ;500µs > 256µs, mode 1 is requested
LOOP:   MOV     TH0, #0FEH      ; high byte of -500
        MOV     TL0, #0CH       ;  low byte of -500
        SETB    TR0
WAIT:   JNB     TF0, WAIT       ; wait for overflow
        CLR     TR0             ; stop timer 0
        CLR     TF0             ; clear TF0
        CPL     P1.0
        SJMP    LOOP            ; repeat
        END
```



5 program - prostokątny co 1khz -> 1ms tu potrzebujemy 500 powtórzeń więc nie zmieści się w jednym adresie z autoreload(użyty tryb 1)
MOV TH0,#0FEH
MOV TL0,#OCH ;tu wpisane w 16 bitowym -500
CLR TR0 ; wył. zliczania
Ten program nie liczy dokładnie 1 kHz

Interrupts

Overview of 8051 interrupt structure



Przerwanie INT0 zewnętrzne
TF0 - flaga przerwania timera 0
Przerwanie INT1 zewnętrzne
TF0 - flaga przerwania timera 1
RI OR TI - przerwanie z portu szeregowego
TF2 OR EXF2 - przerwania od flagi timera 2 i czegoś jeszcze.
Individual interrupt enable - IE zezwala na wł. danego przerwania
Global enable - sygnał globalny zezwalający na przerwanie
IP register - rejestr priorytetów domyślnie ustala priorytet przerwania od góry.IP register przegląda Low i High priority interrupt w poszukiwaniu priorytetów ważnych i mniej ważnych.

Interrupt enable register IE

EA	-	ET2	ES	ET1	EX1	ET0	EX0

EA , IE7, bit address=AFH – enable/disable all interrupts
ES - e/d serial port interrupt
ET1 -/d timer1 overflow interrupt
EX0, IE0, bit address=A8H – e/d external interrupt 0

TCON

				IE1	IT1	IE0	IT0

7 6 5 4 3 2 1 0
IE1 - External interrupt 1 edge flag, cleared by hardware when interrupt is processed
IT1 - Set/cleared by software to specify falling-edge/low-level triggered external interrup
IE0 - (see IE1) ... 0 ...
IT0 – (see IT1)

IT1 i IT0 - określają jaki sygnał jest syg. przerwania, jeśli =1 to zbocze opadające robi przerwanie , jeśli =0 zbocze wznoszące.
IE1 i IE0 - to flaga która pamięta że było żądanie przerwania

Interrupts

Interrupt vectors:

When an interrupt is accepted, the value loaded into the PC is called the interrupt vector.

Interrupt	Flag	Interrupt vector address
Reset	RST	0000H
External 0	IE0	0003H
Timer 0	TF0	000BH
External 1	IE1	0013H
Timer 1	TF1	001BH
Serial port	RI OR TI	0023H
Timer 2	TF2 OR EXF2	002BH

Co się dzieje z akceptowanym przerwaniem? Skok do programu posiadającego przerwanie. Jeśli mamy dany typ przerwania jest wysyłany wektor przerwania.

Program 5. Examine the program which creates a 10 kHz square wave on P1.0 using Timer 0. Notice that timer mode 2 (8-bit auto-reload mode) is used.

```
        ORG     8200H
        MOV     TMOD, #02H      ;8-bit auto-reload mode
        MOV     TH0, #-50       ; -50 reload value in TH0
        SETB    TR0             ;Start timer
LOOP:   JNB     TF0, LOOP       ;Wait for overflow (timer 0 overflow flag TF0=1)
        CLR     TF0             ;Clear timer 0 overflow flag TF0
        CPL     P1.0            ;Complement bit P1.0.
;Between each complement operation, a delay of 50 µs (half of 10 kHz square wave
 ;period) is programmed using Timer 0.
        SJMP    LOOP            ;Repeat
        END
```

Program 6. Examine the program using Timer0 **and Interrupts** to create a 10 kHz square wave on P1.0

```
        ORG     0
        LJMP    MAIN    ;Jump above interrupt vectors
        ORG     000BH   ;Timer 0 interrupt vector
ISR0:   CPL     P1.0    ;Complement port bit
        RETI            ; RETI pops high- and low-order bytes of the PC from the stack
        ORG     0030H   ;Main program entry point
MAIN:   MOV     TMOD, #02H ;Timer 0, mode 2
        MOV     THO, #-50 ;50 µs delay
        SETB    TR0     ;Start timer
        MOV     IE, #82H ; Enable timer 0 interrupt
HERE:   SJMP    HERE    ; „Do nothing" loop. Each 50 µs an interrupt occurs and the Timer 0
                        ; ISR0 executes.
        END
```

The timer flag, TF0, is not explicitly cleared by software. When interrupts are enabled, TF0 is automatically cleared by hardware when the CPU vectors to the interrupt.

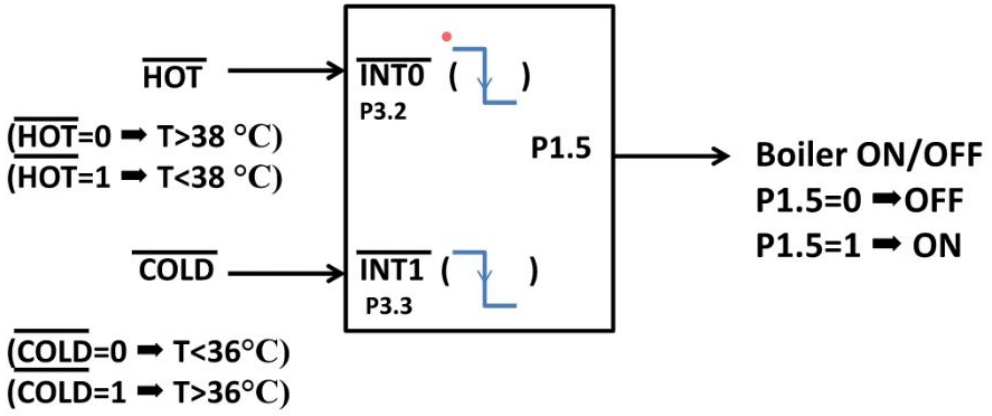
Program 5 był już wcześniej omówiony.Program 6 to ten sam program ale z wykorzystaniem przerwań.

```
ORG 0
LJMP MAIN ;skok do main
ORG 000BH ;Timer 0 interrupt vector
ISR0: CPL P1.0 ;Complement port bit
RETI ; RETI pops high- and low-order bytes of the PC from the stack
ORG 0030H ;Main program entry point
MAIN: MOV TMOD, #02H ;Timer 0, w trybie 2
MOV THO, #-50 ;50 μs delay
SETB TR0 ;Start timer
MOV IE, #82H ; Enable timer 0 interrupt ->82H->10000010 czyli  przerwanie od timera 0
HERE: SJMP HERE ; „Do nothing” loop. Each 50 μs an interrupt occurs and the Timer 0 ;
ISR0 executes.
```

Jaka jest różnica między użyciem przerwań a nie używanie ich?
Z przerwaniem możemy zamiast pętli “Do nothing” możemy dalej pisać program i może się on wykonywać i w razie wykonania on powróci do Maina.

Boiler controller

Using **interrupts,** write an 8051 controller program that keeps a water at 37 °C + 1 °C.



P1.5=0 nie ma grzania
P1.5 =1 grzanie
INT0=0 - gdy za wysoka temperatura
INT1=0 - gdy za niska temperatura

Program 4 Boiler temperature controller

```
;The program should turn on the water heater for T<36 C and turn off for T>38.
;(Signal HOT=0 if T>38 C, signal COLD=0 if T<36 C)
        ; Address of the location in memory wich contains the first byte od the instruction LJMP is 0.
        ORG     0       ;The ORG directive sets a new program origin for statements that follow.
        LJMP    MAIN    ;Jump above interrupt vectors. LJMP causes an unconditional branch to
                        ;label MAIN. The instruction loads the program counter with 0030H
ISR0:   CLR     P1.5    ;Turn heater off. The interrupt vector is 3.
                        ; Directive ORG 3 is not necessary, because the LJMP is 3-byte instruction
        RETI            ; Return from interrupt
        ORG     0013H   ;The Vector Address equal 13H is the address of the start of the ISR1.
ISR1:   SETB    P1.5    ;Turn heater on.
        RETI
        ; Above two Interrupt Service Routines ISR0 and ISR1
        ORG     30H     ;Main program entry point
MAIN:   MOV     IE, #85H    ;Main program begins. Enable external interrupts (TCON,TMOD sl.53,58)
        SETB    IT0     ;Negative edge triggered. IT0 is the interrupt 0 type control bit in TCON reg.
        SETB    IT1     ;Neg. edge triggered. IT1 is the interrupt 1 type control bit in TCON reg.
        SETB    P1.5    ;Turn heater on.
        JB    P3.2, SKIP  ;The HOT input is sampled. If HOT is high (1) (T<38C) the water heater is left ON.
                          ; If T>38 C turn heater off
        CLR     P1.5    ; Turn the water heater off.
SKIP:   SJMP    SKIP    ;Do nothing. Each time the temperaturę rises above 38C or falls below 36C,
                        ;an interrupt occurs.
        END             ; END directive is the last statement in the source program
```

ORG 0 ;The ORG directive sets a new program origin for statements that follow.
LJMP MAIN ;Skok do Maina

ISR0:

CLR P1.5 ;Turn heater off. Przerwanie wył. grzania
RETI ; Return from interrupt
ORG 0013H ;The Vector Address equal 13H is the address of the start of the ISR1.

ISR1:

SETB P1.5 ;Turn heater on. Przerwanie wł. grzania
RETI
; Above two Interrupt Service Routines ISR0 and ISR1
ORG 30H ;Main program entry point

MAIN:

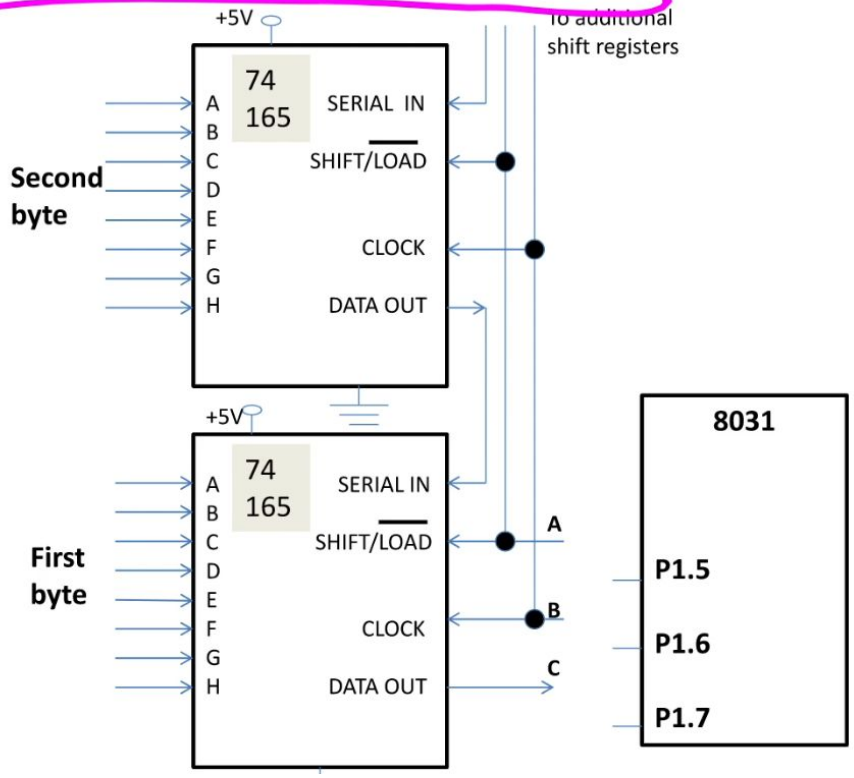
MOV IE, **#85H** ; 85H=10000101B(patrz str. 4)
SETB IT0 ;ustawianie przerwania zboczem ujemnym
SETB IT1 ;ustawianie przerwania zboczem ujemnym
SETB P1.5 ;Turn heater on. - wł. piec
JB P3.2, SKIP ; P3.2 to jest INT0 , czyli jeśli int0=1 to wł. grzanie
CLR P1.5 ; Turn the water heater off. jeśli int0 =1

SKIP:

SJMP SKIP ;Do nothing. Each time the temperaturę rises above 38C or falls below 36C, ;an interrupt occurs.
END ; END directive is the last statement in the source program
I jak już jest w skipie to przerwania działają tak że jeśli P3.2 = 1, to leci do ISR1, a jak P3.3 to ISR0.

Example of input/output expansion

1) Connect the microcontroller pins **P1.5, P1.6 and P1.7** to **A,B,C**
2) Write a subroutine that copies the **First byte** and the **Second byte** to 8051 internal RAM locations **30H** and **31H**



Jak shift =1 zegar przesuwa zawartość o jeden bit gdy będzie na zegarze zbocze narastające(Czyli ogólnie rejestr przesuwający), ale jeśli bd shift=0 czyli coś ma być załadowane a dokładnie te pierwszy i drugi bajt.
C- to wyjście 16 bitowe które miało być wpisane do RAMu.

Problem 1
Dotyczy do slajdu 44 – Example of input/output expansion

Podprogram GET_BYTES ładuje równolegle 2 bajty **First byte** oraz **Second byte** (gdy na wejściu sterującym SHIFT/$\overline{\text{LOAD}}$ pojawi się zero) do dwóch rejestrów przesuwających 74165 (paralel-in serial-out).
Następnie (gdy już SHIFT/$\overline{\text{LOAD}}$ = 1) narastające zbocza impulsów zegara na wejściu CLOCK przesuwają zawartość rejestrów 74165 (bit po bicie) w kierunku DATA OUT dolnego rejestru 74165.
Kolejne bity z wyjścia DATA OUT (z punktu C na rysunku, nie mylić z carry flag w PSW) dolnego rejestru 74165 są zapisywane do rejestru A mikrokontrolera. Kolejne bajty z rejestru A są zapisywane do bufora w wewnętrznej pamięci RAM.

Zapoznaj się z programem GET_BYTES i połącz na rysunku wyprowadzenia mikrokontrolera (P1.5, P1.6 i P1.7) z rejestrami 74165 (Clock –punkt B, SHIFT/$\overline{\text{LOAD}}$ punkt A i DATA OUT – punkt C).
Wpisz komentarze do kolejnych rozkazów podprogramu.

```
GET_BYTES:
        MOV   R7, #2
        MOV   R0, #30H
        CRL   P1.7
        SETB  P1.7
AGAIN: MOV  R6, #8
LOOP:  MOV  C, P1.6 ; get a bit, put it in the carry flag in the PSW
        RRC   A
        CLR   P1.5
        SETB  P1.5
        DJNZ  R6, LOOP
        MOV   @R0, A
        INC   R0
        DJNZ  R7, AGAIN
        RET
```

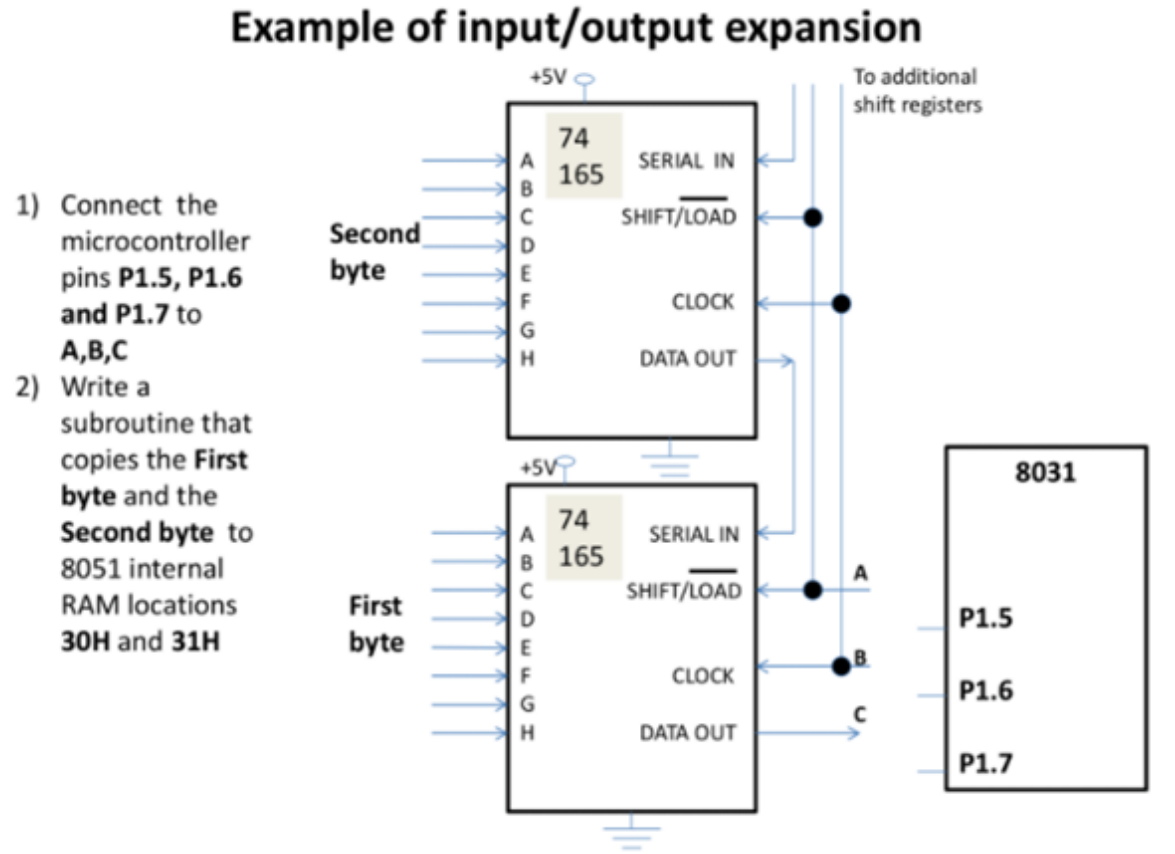
Problem 1 (in English)
See Example of input/output expansion - slide 44
The subroutine GET_BYTES copies the state of the 16 inputs (First Byte and Second Byte) to internal RAM locations 30H and 31H.
The **First Byte** and the **Second Byte** are sampled by the SHIFT/$\overline{\text{LOAD}}$ line low. Next (when SHIFT/$\overline{\text{LOAD}}$=1) each pulse on the CLOCK line shifts the data inside the registers 74165 "down" to DATA OUT (point C in the picture).
The instruction **MOV C, P1.7** moves DATA OUT bit to the carry flag in PSW.

Read the subroutine and connect correctly the microcontroller pins P1.5, P1.6, P1.7 with points A,B,C
Write a comment in the comment field at the end of each line.

```
GET_BYTES:
        MOV R7, #2
        MOV R0, #30H
        CRL P1.7
        SETB P1.7
AGAIN: MOV R6, #8
LOOP: MOV C, P1.6 ; get a bit, put it in the carry flag in the PSW
        RRC A
        CLR P1.5
        SETB P1.5
        DJNZ R6, LOOP
        MOV @R0, A
        INC R0
        DJNZ R7, AGAIN
        RET
```

P1.6 -pobiera nam wartości 16 bitowe - C
P1.7 - ustawia nam zbocze narastające - B
P1.5 - Nam ustawia przesuwanie sterując shift - A

WYKŁAD 4



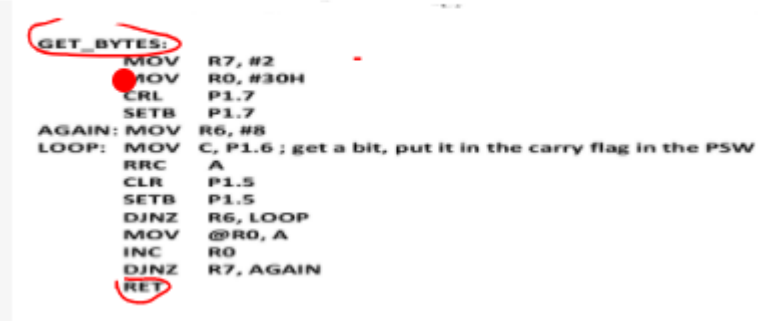
Rejestry przesuwne - zmiany tylko przy **zboczu narastającym**, wyjście górnego połączone z wejściem dolnego. Shift/Load jak jest **0**, to aktywne jest load, wtedy Bajt Pierwszy i drugi zostaną załadowane do rejestru, przełączeniu Shift/load na 1 zawartość górnego rejestru jest bit po bicie zapsywana do dolnego rejestru. Przesunięcie o jeden bit odbywa się na narastającym zboczu zegara.

Zadanie – dwa bajty mają być zapisane do pamięci RAM pod adresem 30H i 31H (do bufora). Robimy to za pośrednictwem akumulatora **(?)**.

Problem 1
Dotyczy do slajdu 44 – Example of input/output expansion

Podprogram GET_BYTES ładuje równolegle 2 bajty **First byte** oraz **Second byte** (gdy na wejściu sterującym SHIFT/$\overline{\text{LOAD}}$ pojawi się zero) do dwóch rejestrów przesuwających 74165 (paralel-in serial-out).
Następnie (gdy już SHIFT/$\overline{\text{LOAD}}$ = 1) narastające zbocza impulsów zegara na wejściu CLOCK przesuwają zawartość rejestrów 74165 (bit po bicie) w kierunku DATA OUT dolnego rejestru 74165.
Kolejne bity z wyjścia DATA OUT (z punktu C na rysunku, nie mylić z carry flag w PSW) dolnego rejestru 74165 są zapisywane do rejestru A mikrokontrolera. Kolejne bajty z rejestru A są zapisywane do bufora w wewnętrznej pamięci RAM.

Zapoznaj się z programem GET_BYTES i połącz na rysunku wyprowadzenia mikrokontrolera (P1.5, P1.6 i P1.7) z rejestrami 74165 (Clock –punkt B, SHIFT/$\overline{\text{LOAD}}$ punkt A i DATA OUT – punkt C).
Wpisz komentarze do kolejnych rozkazów podprogramu.



```
GET_BYTES:
        MOV   R7, #2
        MOV   R0, #30H
        CRL   P1.7
        SETB  P1.7
AGAIN:  MOV   R6, #8
LOOP:   MOV   C, P1.6 ; get a bit, put it in the carry flag in the PSW
        RRC   A
        CLR   P1.5
        SETB  P1.5
        DJNZ  R6, LOOP
        MOV   @R0, A
        INC   R0
        DJNZ  R7, AGAIN
        RET
```

RRC - Przesuwa zawartość akumulatora w prawo i wchodzi na miejsce bitu najbardziej znaczącego (7, numeracja od 0) bit przeniesienia z PSW czyli C.

Tu:

P1.7 -> shift/load

P1.5 -> clock

P1.6 -> data ou

Instrukcja MOV Instrukcja służąca do przekazywania danych pomiędzy rejestrami procesora i pamięcią wewnętrzną. W zależności od tego co i gdzie „przenosimy”, polecenie to może mieć kilkanaście różnych postaci w zależności od zastosowanych argumentów. Jednocześnie warto wiedzieć, że polecenie to w praktyce nie powoduje dosłownego ‘przemieszczenia’ danej, lub zawartości rejestru, ale jej skopiowanie za źródła do miejsca przeznaczenia. Przypomina to polecenie KOPIUJ - WKLEJ.

MOV A,Rn

- do akumulatora zostaje załadowana zawartość rejestru Rn
- kod: 1 1 1 0 1 n2 n1 n0 gdzie n2...n0 wskazują na R0...7 stąd: E8-EFh

MOV A,adres

- do akumulatora zastaje załadowana zawartość komórki wewnętrznej pamięci RAM o adresie: „adres”
- kod: 11100101 E5h

MOV A,@Ri

- do akumulatora zostaje załadowana zawartość komórki wew. RAM, której adres znajduje się w rejestrze R0 (i=0), lub R1 (i=1)
- kod: 1110011i stąd: E6h, E7h

MOV A,#dana

- instrukcja załadowania 8-bitowej liczby „dana” do akumulatora
- kod: 01110100 74h

CLR C

- wyzerowany zostaje znacznik (bit w rejestrze PSW - stanu) przeniesienia C
- kod: 11000011 C3h

SETB C

- ustawiona zostaje flaga przeniesienia
- kod: 11010011 D3h

CLR bit

- wyzerowany zostaje bit którego adres podany jest bezpośrednio
- kod: 11000010 C2h

SETB bit

- ustawiony zostaje bit którego adres podany jest bezpośrednio
- kod: 11010010 D2h

CPL C

- flaga C zostaje zanegowana
- kod: 10110011 B3h

CPL bit

- zanegowany zostaje bit którego adres podany jest bezpośrednio
- kod: 10110010 B2h

RRC A

- zawartość A zostaje przesunięta o 1 pozycję (1 bit) w prawo z uwzględnieniem znacznika C, to znaczy, że:
  bit 0 przyjmuje wartość bitu 1
  bit 1 przyjmuje wartość bitu 2
  itd...
  bit 6 przyjmuje wartość bitu 7
  znacznik C przyjmuje wartość bitu 0
  a
  bit 7 przyjmuje wartość znacznika C
- znaczniki: C jest ustawiany zgodnie z wynikiem operacji
- kod: 00010011 13h

Instrukcja DJNZ , rel

• w wyniku tej operacji od wskazanego argumentu „arg” odejmowana jest jedynka (jest on dekrementowany). Jeżeli w wyniku odjęcia wartość argumentu nie jest równa zero, to zostaje wykonany skok zgodnie z zasadami opisywanymi wcześniej w przypadku argumentu „rel”. Stan znaczników nie zmienia się. W zależności od typu argumentu rozróżnia się dwa typy instrukcji, oto one

DJNZ Rn, rel

• zmniejszona zostaje zawartość podanego rejestru Rn (R0...R7) o jeden, a następnie jeżeli nie jest równa zero, to następuje skok

• kod: 1 1 0 1 1 n2 n1 n0 n2...n0 wskazuje na R0...R7 stąd: D8-DFh

DJNZ adres, rel

• zmniejszona zostaje zawartość komórki pamięci wew. RAM o podanym bezpośrednio adresie o jeden, a następnie jeżeli nie jest równa zero, to następuje skok

• kod: 11010101 D5h

NOP

• nie rób nic

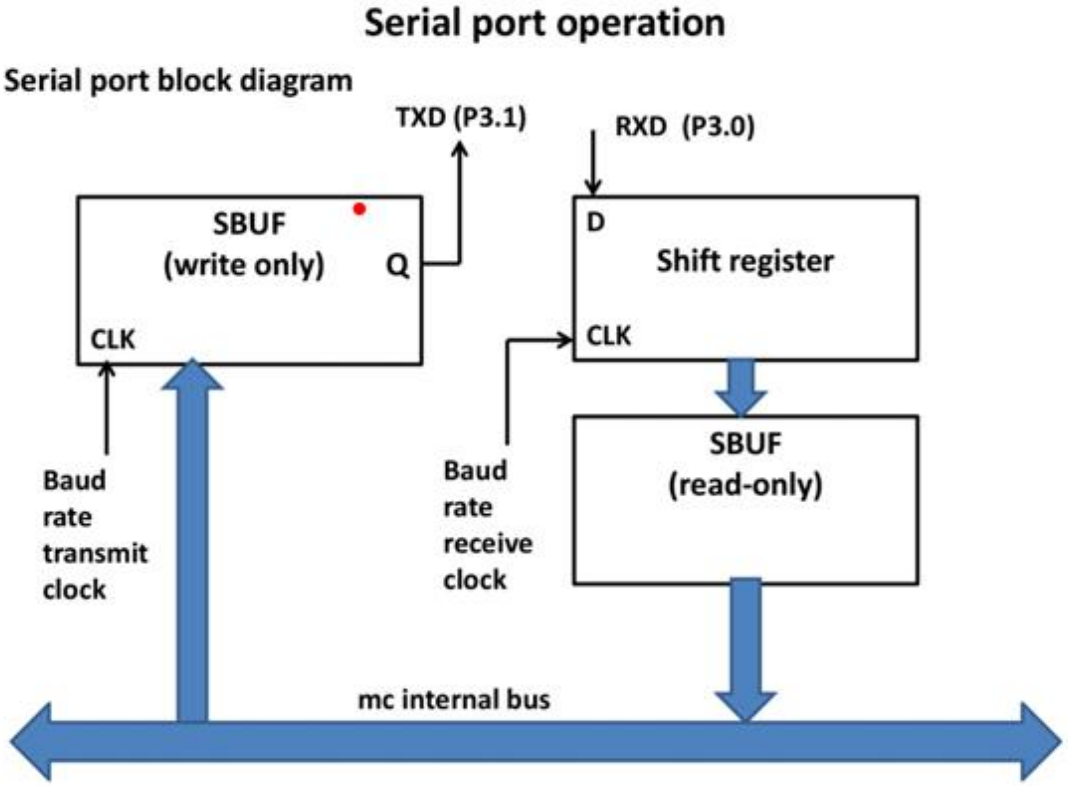
• w wyniku tej instrukcji nie zmienia się stan procesora, z wyjątkiem licznika rozkazów który po pobraniu tej instrukcji jest zwiększany o jeden

• kod: 00000000 00h

Instrukcja **INC**

Do wskazanego argumentu dodawana jest jedynka. Znaczniki nie ulegają zmianie.

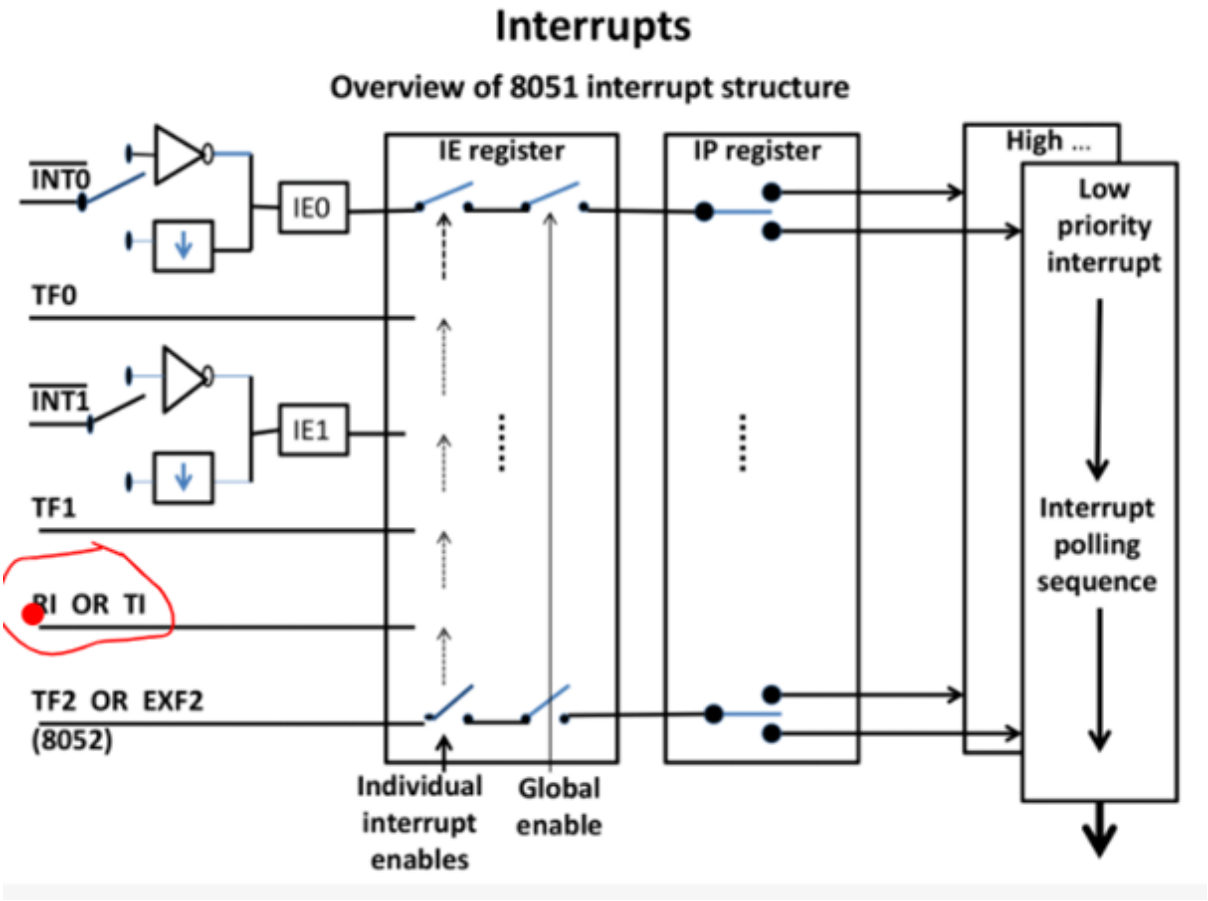
http://www.zstio-elektronika.pl/pliki_t_telekomunikacji/asembler_8051.pdf



TXD – wyjście danych

RXD – wejście danych

SBUF – bufor, pod adresem 99, jak zapisujemy coś to zapisujemy tu, następnie zegar odczytuje te bity i one tam są transmitowane. Jak jest odczytywany, to bity z częstotliwością zegara są zapisywane do rejestru przesuwającego, a następnie do tego nadajnika (chyba SBUF). Zarówno nadajnik jak i odbiornik wysyłają przerwanie do systemu przerwań.



Przerwanie z portu szeregowego mamy w tym zaznaczonym miejscu, przerwanie z odbiornika i nadajnika, po tym jakie to jest przerwanie (nadajnik/odbiornik) musi dopiero rozeznać w programie, wiedzieć jaki jest stan bitów.

Zapisujemy do bufora SBUF, wysyłamy, odebrane do Shift reister i potem do tego SBUF po prawo.

Serial port operation

Serial port modes

8-bit shift register, (mode 0, 0s in SM1 and SM2 of SCON)

(The RXD line is is used for both data input and ouput, the TXD line serves as the clock. Transmission is initiated by any instruction that writes data to SBUF. Reception is initializad when the receiver enable bit (REN) is 1 and the receive interrupt bit (RI) is 0. When RI is cleared, clock pulses are written out the TXD line. **Baud rate is fixed and equals (oscillator frequency)/12.**

8-bit UART with variable baud rate (mode 1)

(Ten bits are transmitted on TXD or received on RXD [a start bit (low), eight data bits (LSB first), a stop bit (high)].

9-bit UART with fixed baud rate (mode 2)

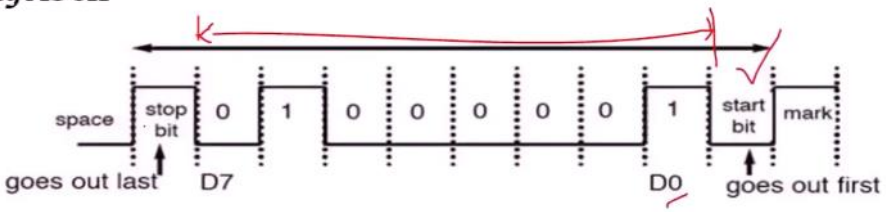
(The ninth data bit is a programmable data bit. On transmission, the ninth data bit equals TB8 in SCON reg. On reception, the ninth received data bit is placed in RB8. **The baud rate equals 1/32 or 1/64 the on-chip oscillator**.

9-bit UART with variable baud rate (mode 3)

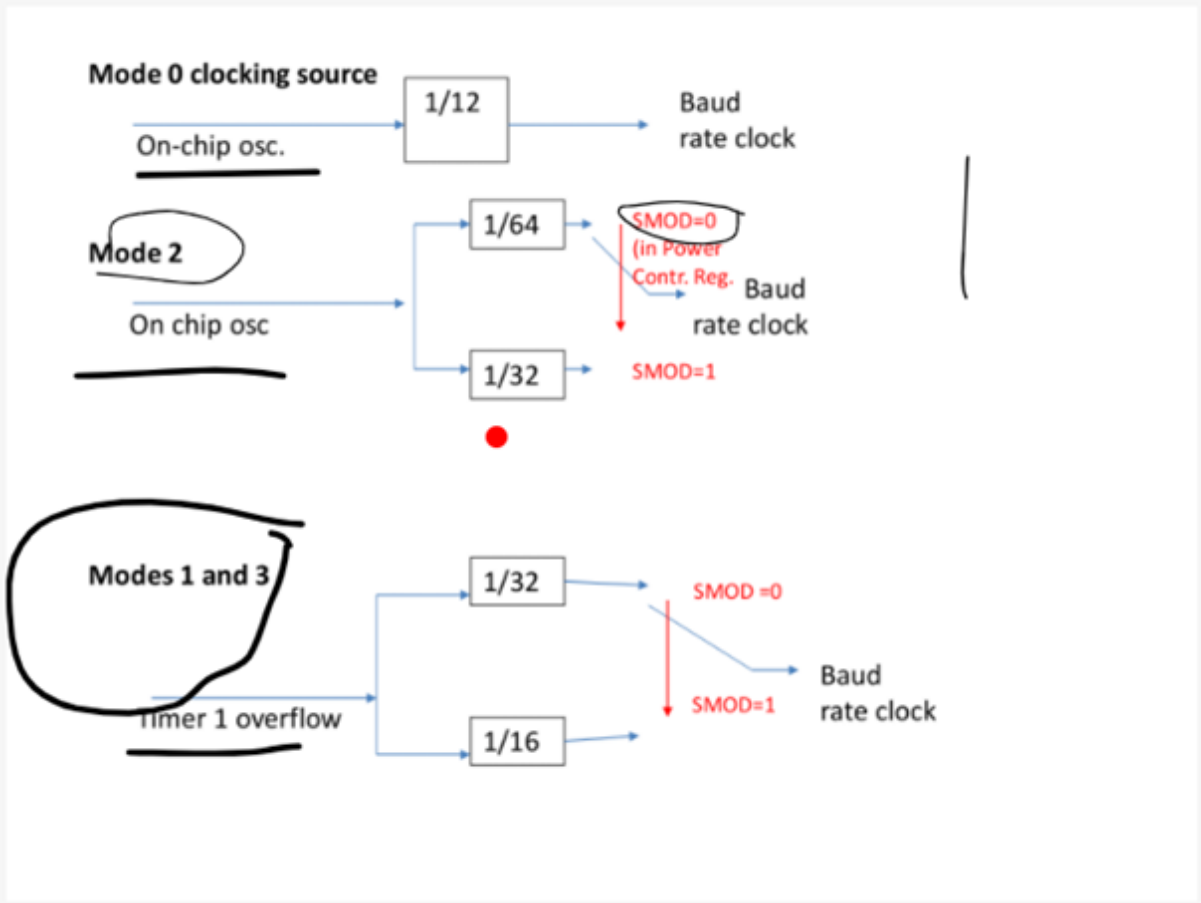
(The baud rate is programmable and provided by the timer: (timer 1 overflow) /32 or /64.)

Data Framing in Asynchronous Serial Communication

Framing ASCII



- The transmission begins with a start bit, followed by the LSB(D_0), then the rest of the bits until MSB (D_7), and finally, the one stop bit indicating the end of the character
- When there is no transfer, the signal is 1 (high), which is referred to as *mark*



Bit SMOD znajduje się w rejestrze PCR (Power Control Register)

Initialize the serial port

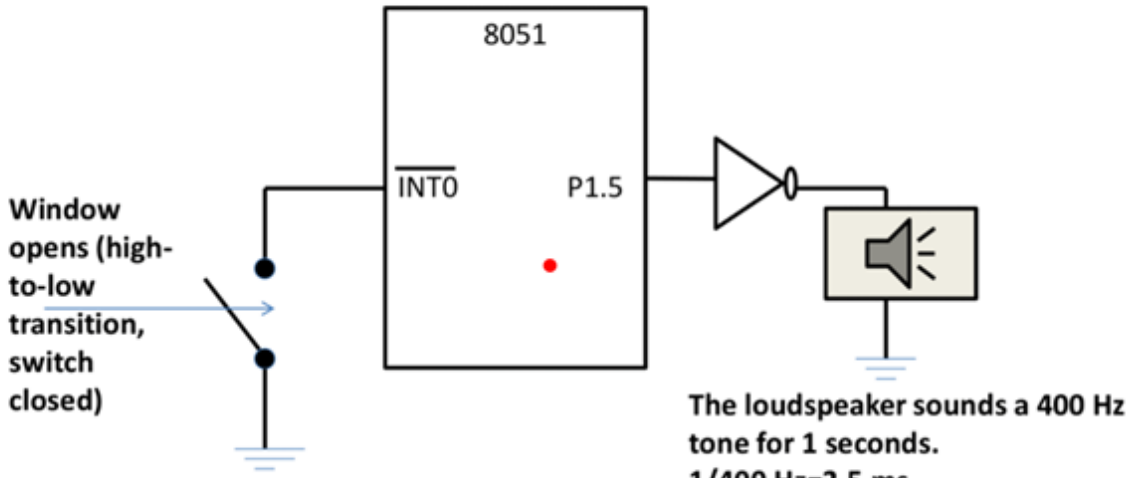
```
        ORG     8300H
INIT:   MOV     SCON, #52H      ;serial port, mode 1
        MOV     TMOD,#20H       ;timer 1, mode 2
        MOV     TH1, #-13       ; reload count for 2400 baud
        SETB    TR1             ; start timer 1
        END
```

SCON – rejestr sterujący portem szeregowym

Symbol	Nazwa	Symbol	Nazwa
* ACC	akumulator	* IP	rejestr priorytetu przerwań
* B	rejestr B	* IE	rejestr zezwoleń przerwań
* PSW	rejestr stanu	TMOD	rejestr trybu pracy timerów
SP	wskaźnik stosu	* TCON	rejestr sterujący timerów
DPTR	wskaźnik danych	TH0	Timer 0 bardziej znaczący bajt
DPL	mniej znaczący bajt	TL1	Timer 0 mniej znaczący bajt
DPH	bardziej znaczący bajt	TH1	Timer 1 bardziej znaczący bajt
* P0	port 0	TL1	Timer 1 mniej znaczący bajt
* P1	port 1	* SCON	rejestr sterujący portu szeregowego
* P2	port 2	SBUF	bufor portu szeregowego
* P3	port 3	PCON	rejestr sterujący zasilaniem mikrokontrolera

Intrusion warning system

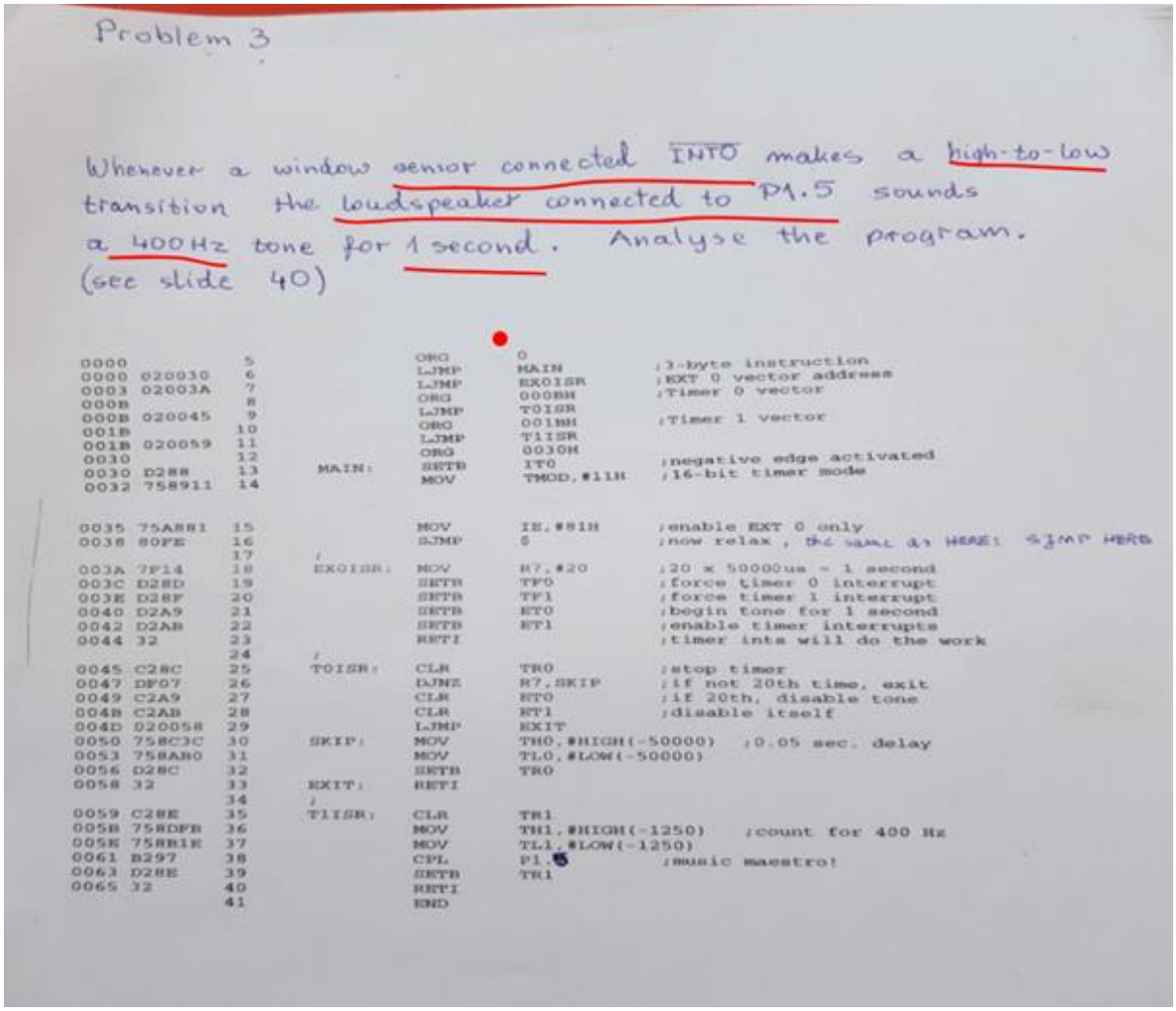
Using **interrupts,** write an 8051 program for an intrusion warning system



(see handout)

Piszemy kod realizujący buczenie głosnika (400MHz), jeżeli ktoś otwiera okno.

I tutaj kod:



Problem 3

Whenever a window sensor connected INT0 makes a high-to-low transition the loudspeaker connected to P1.5 sounds a 400Hz tone for 1 second. Analyse the program. (see slide 40)

```
0000                5             ORG     0
0000 020030         6             LJMP    MAIN            ;3-byte instruction
0003 02003A         7             LJMP    EX0ISR          ;EXT 0 vector address
000B                8             ORG     000BH           ;Timer 0 vector
000B 020045         9             LJMP    T0ISR
001B                10            ORG     001BH           ;Timer 1 vector
001B 020059         11            LJMP    T1ISR
0030                12            ORG     0030H
0030 D288           13  MAIN:     SETB    IT0             ;negative edge activated
0032 758911         14            MOV     TMOD,#11H       ;16-bit timer mode

0035 75A881         15            MOV     IE,#81H         ;enable EXT 0 only
0038 80FE           16            SJMP    $               ;now relax , the same as HERE: SJMP HERE
                    17  ;
003A 7F14           18  EX0ISR:   MOV     R7,#20          ;20 x 50000us = 1 second
003C D28D           19            SETB    TF0             ;force timer 0 interrupt
003E D28F           20            SETB    TF1             ;force timer 1 interrupt
0040 D2A9           21            SETB    ET0             ;begin tone for 1 second
0042 D2AB           22            SETB    ET1             ;enable timer interrupts
0044 32             23            RETI                    ;timer ints will do the work
                    24  ;
0045 C28C           25  T0ISR:    CLR     TR0             ;stop timer
0047 DF07           26            DJNZ    R7,SKIP         ;if not 20th time, exit
0049 C2A9           27            CLR     ET0             ;if 20th, disable tone
004B C2AB           28            CLR     ET1             ;disable itself
004D 020058         29            LJMP    EXIT
0050 758C3C         30  SKIP:     MOV     TH0,#HIGH(-50000)   ;0.05 sec. delay
0053 758AB0         31            MOV     TL0,#LOW(-50000)
0056 D28C           32            SETB    TR0
0058 32             33  EXIT:     RETI
                    34  ;
0059 C28E           35  T1ISR:    CLR     TR1
005B 758DFB         36            MOV     TH1,#HIGH(-1250)    ;count for 400 Hz
005E 758B1E         37            MOV     TL1,#LOW(-1250)
0061 B297           38            CPL     P1.5            ;music maestro!
0063 D28E           39            SETB    TR1
0065 32             40            RETI
                    41            END
```

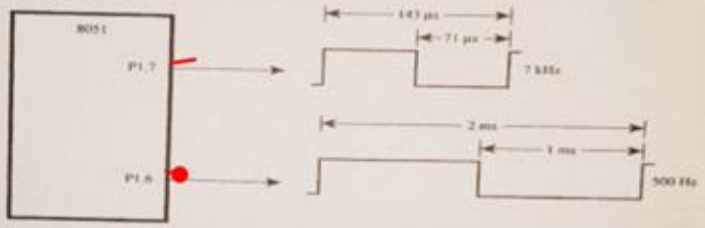
Na początku mamy skok do etykiety MAIN, poniżej mamy adresy od których mogą być programy obsługujące przerwanie.

Te trzy programy pod MAIN to obsługa różnych przerwań.

Kod generujący z określoną częstotliwością sygnał prostokątny:

Problem 2

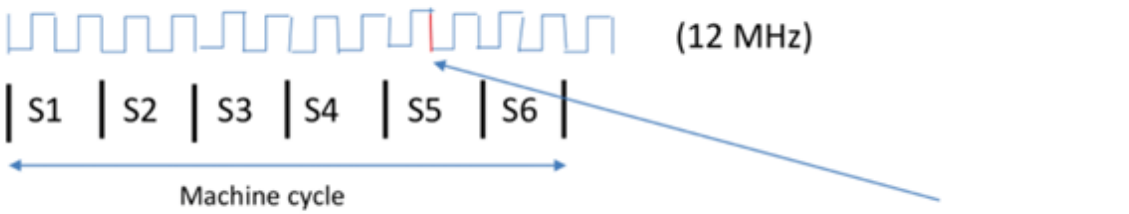
The program simultaneously creates two square waves on P1.7 and P1.6. Explain why timer 1 providing synchronization for the 500Hz signal has to operate in mode 1 not in mode 2. (see slides 34 and 35)




```
0000                5           ORG     0
0000 020030         6           LJMP    MAIN
000B                7           ORG     000BH           ;Timer 0 vector address
000B 02003F         8           LJMP    T0ISR
001B                9           ORG     001BH           ;Timer 1 vector address
001B 020043        10           LJMP    T1ISR
0030               11           ORG     0030H
0030 758912        12  MAIN:    MOV     TMOD,#12H       ;Timer 1 - mode 1
                   13                                   ;Timer 0 - mode 2
0033 758CB9        14           MOV     TH0,#-71        ;7 kHz using timer 0
0036 D28C          15           SETB    TR0
0038 D28F          16           SETB    TF1             ;force timer 1 interrupt
003A 75A88A        17           MOV     IE,#8AH         ;enable both timer intrrpts
003D 80FE          18           SJMP    $
                   19           ;
003F B297          20  T0ISR:   CPL     P1.7
0041 32            21           RETI
0042 C28E          22  T1ISR:   CLR     TR1
0044 758DFC        23           MOV     TH1,#HIGH(-1000)  ;1 ms high time &
0047 758B18        24           MOV     TL1,#LOW(-1000)   ; low time
004A D28E          25           SETB    TR1
004C B296          26           CPL     P1.6
004E 32            27           RETI
                   28           END
```

SJMP $ powoduje, że stoimy w miejscu.

Interrupt timing



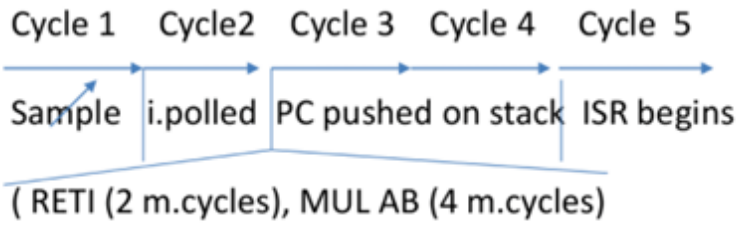
One machine cycle is 12 oscillator periods.
Interrupt samples are polled in the next m.cycle.
Interrupt is accepted if:

- no other interrupt of equal or higher priority is in progres,
- the polling cycle is the last cycle in an instruction,
- the current instruction is not a RETI or an access to IE or IP.
  (After RETI at least one instruction has to execute.)

Sample&latch
an interrupt signal

During the next two cycles, the procesor pushes the PC on the stack and loads the PC with the interrupt vector address and next the ISR begins.

Interrupt latency:



Timing związany w przerwaniami – jeżeli zegar jest 12MHz, to mikrosekunde mamy 12 impulsów zegara. Mikroporecesor to dzieli na 6 części. Rządanie przerwania pojawia się gdzieś miedzy S1 a S6, ten sygnał przerwania jest próbkowany i zapamiętywany, w następnym cyklu rządania zapamiętane są przeglądane.

<u>Kiedy przerwanie jest zaakceptowane?</u>

- Inne przerwanie o tym samym lub wyzszym priorytecie nie jest obsługiwane.

- Cykl przeglądania jest ostanim cyklem maszynowym rozkazu, dany rozkaz musi być wykonany do konca, a nastepnie dopiero może nastapic skok.

- Bieżący rozkaz to nie jest np. rozkaz RETI, lub to nie jest to dostęp do IE lub IP.

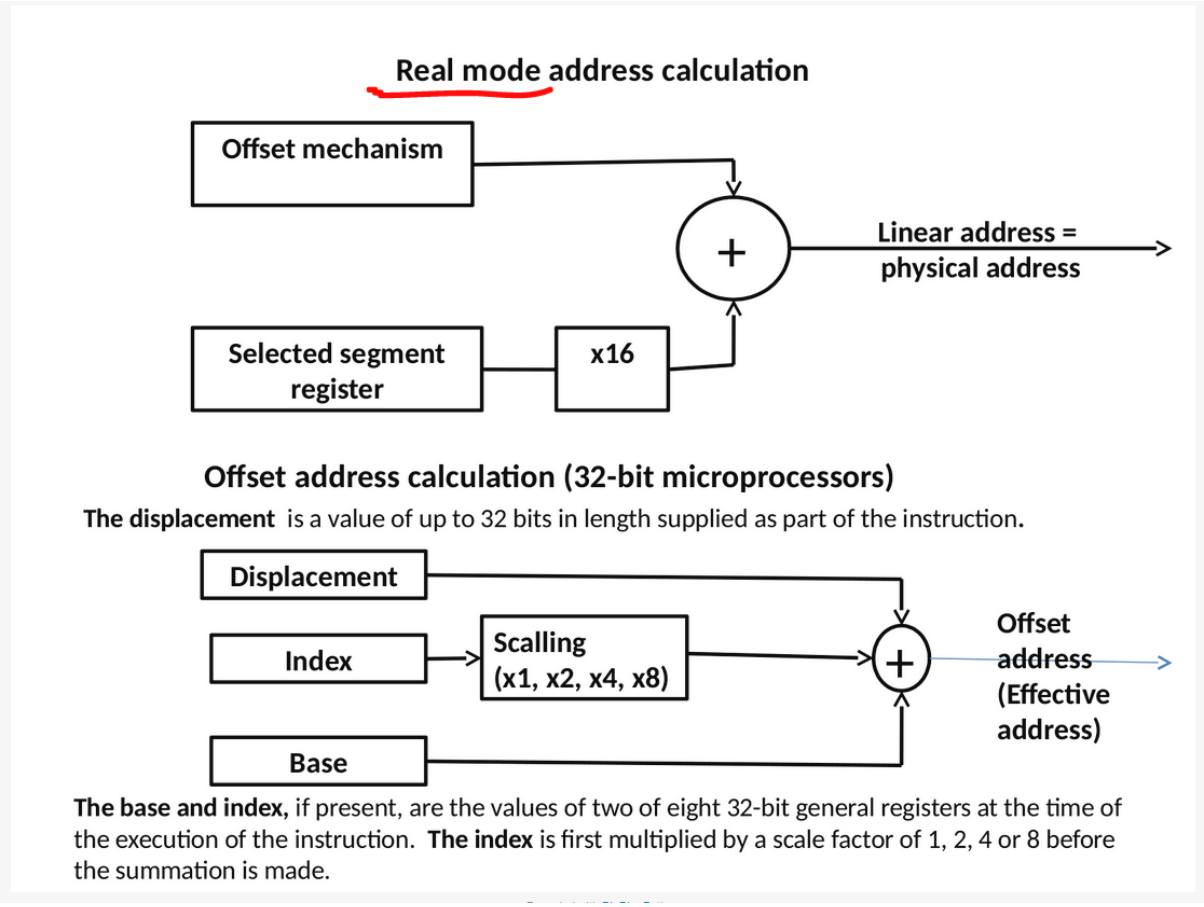
Na górze widzimy jeden cykl kiedy nastąpiło przerwanie, a **na dole widzimy co się dzieje w kolejnych cyklach** po przerwaniu.

Wykład 5 (Jarek)

Wykład 6 (Damian)

Początek wykładu 6

1. **Real mode address calculation**

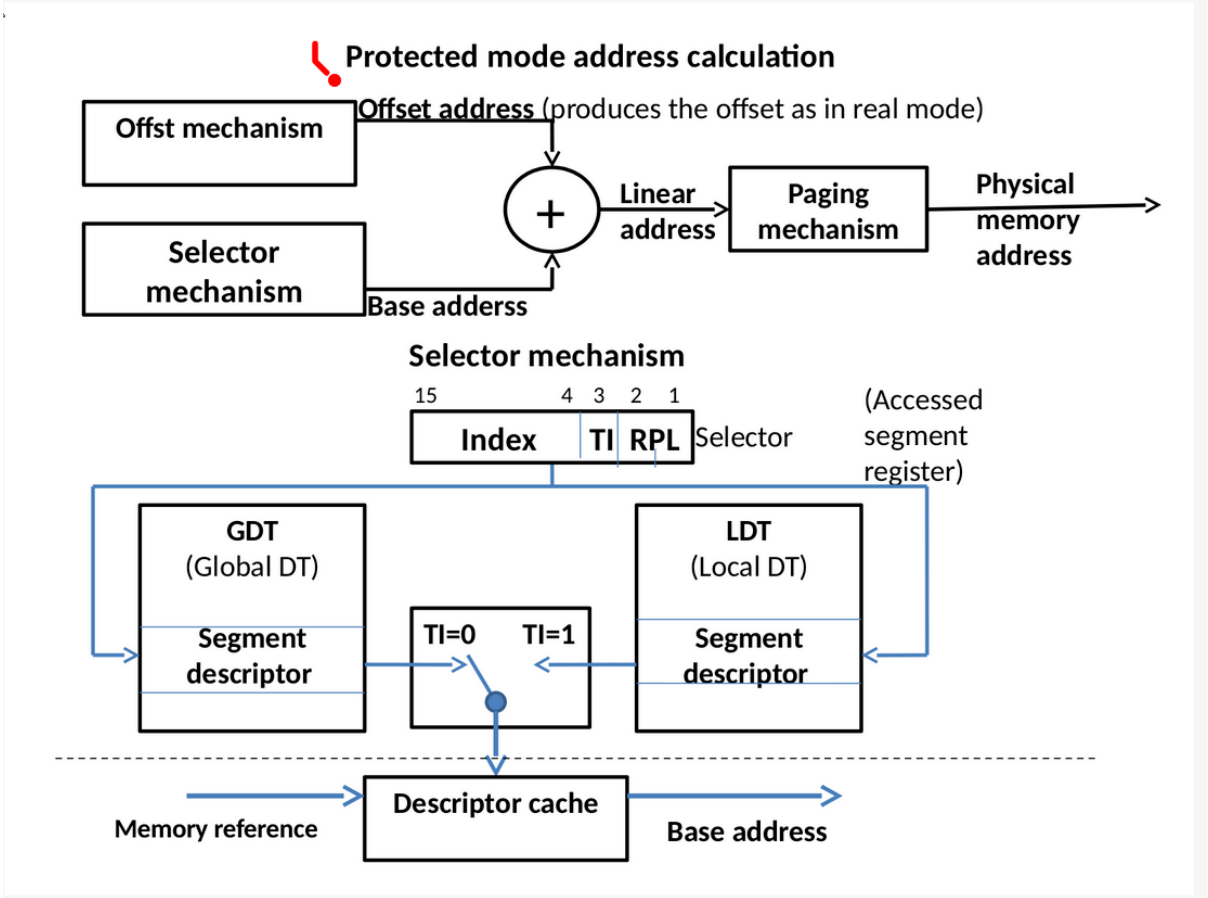


a. Real mode address calculation - jest to tryb który określa jak jest wybierany adres komórki do której chcemy się odwołać w procesorach intel 8086.
b. Zawiera on rejestr segmentowy (4 rejestry segmentowe) który były mnożone przez 16. Do tego był dodawany offset(przesunięcie) w ramach segmentu i tak powstawał adres fizyczny
c. Offset address calculation - schemat pokazujący w jaki sposób generowany jest offset. Generowanie offsetu ma wiele kombinacji może być to suma wszystkich 3 składników (displacement, index, base), mogą być to też pojedyncze składniki np. samo index itd.
d. Displacement - liczba w rozkazie (przesunięcie)
e. Base - zawartość jakiegoś rejestru bazowego
f. Index - zawartość rejestru indeksowanego przemnożona przez 1, 2, 4, 8

ADDRESSING MODE	BASE	INDEX	SCALE FACTOR (SF)	DISPLA-CEMENT (DP)	OFFSET ADDRESS (OA)
Direct				X	OA = DP
Register indirect	X				OA = [BASE]
Based	X			X	OA = [BASE] + DP
Index		X		X	OA = [INDEX] +DP
Scaled index		X	X	X	OA = ([INDEX]*SF) + DP
Based index	X	X			OA = [BASE] = [INDEX]
Based scaled index	X	X	X		OA=[BASE] + ([INDEX]*SF)
Based index with displacement	X	X		X	OA= [BASE] + [INDEX]+DP
Based scaled index with displacement	X	X	X	X	OA= [BASE] + ([INDEX]*SF + DP

a. Tabelka pokazująca w jaki sposób można wygenerować offset.

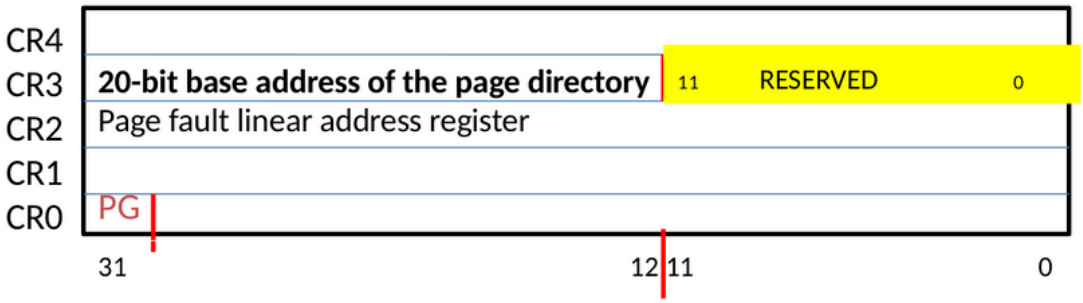
2. Protected mode address calculation



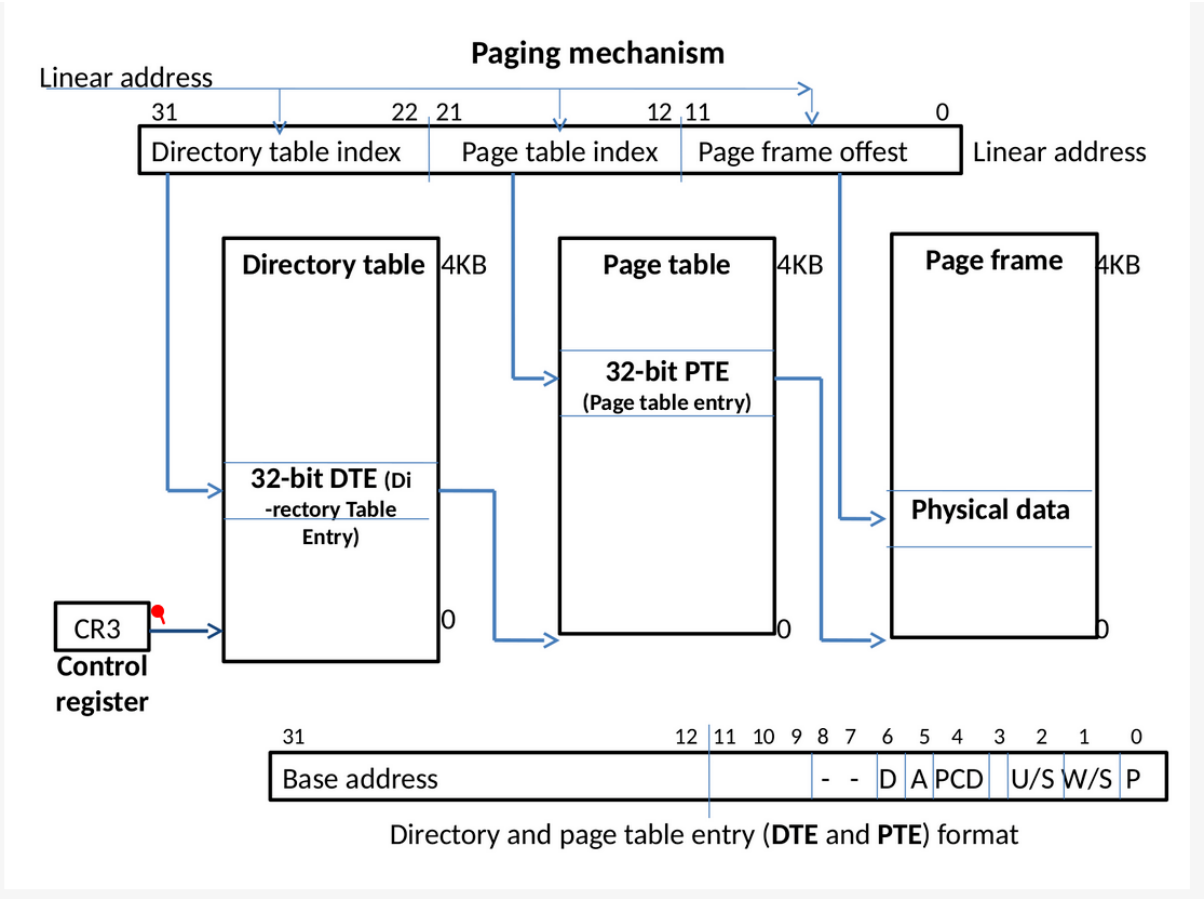
a. Tryb protected - stosowany we wszystkich procesorach 32 bitowych i końcowych 16 bitowych. Też jest to sposób generowania adresu lecz został nieco zmieniony sposób.
b. Rejestr segmentowy został zamieniony na selector mechanism
c. Bity od 4 do 15 wskazywały na położenie deskryptora segmentu w tabeli GDT (Global DT) lub w tablicy LDT (Local DT)
d. TI - bit decydujący którą tablicę deskryptorów wybrać
e. RPL - bity które pozwalają na rozdzielenie informacji np. od wielu użytkowników

3. Memory paging - stronicowanie pamięci

a. Adres liniowy jest to adres generowany przez program
b. Adres fizyczny jest to adres do ramu która jest dołączona do procesora
c. Celem tej metody jest przypisanie adresu liniowemu adresu fizycznego
d. Pamięć fizyczna łącznie z dyskiem i system operacyjny który umożliwia przepisywanie stron pomiędzy pamięcią fizyczną która jest podzielona na ramki. W ramce można zapisać jedną stronę. Przesyłanie stron pomiędzy pamięcią fizyczną (adresy fizyczne), a twardym dyskiem pozwala na zwiększenie pamięci. Wydaje się że pamięć podłączona do mikroprocesora jest duża nawet gdy pamięć fizyczna nie jest duża.
e. Jak to funkcjonuje. Mamy rejestry CR0, CR1 itd. W rejestrze CR0 mamy bit PG - paging który sygnalizuje czy stronicowanie jest włączone czy nie. W rejestrze CR3 jest 20 bitowy adres do tak zwanej page directory jest to strona w której znajduje się 1000 elementów które stanowią adresy. CR3 jest to adres początku tej strony.

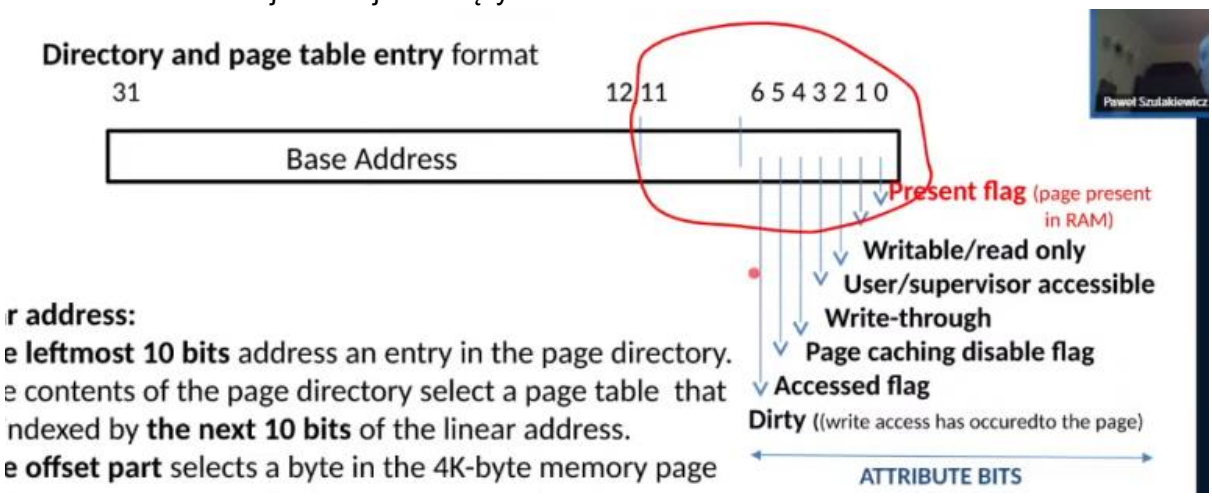


f. Mechanizm stronicowania zamienia adres liniowy na odpowiadający mu adres fizyczny lub generuje przerwanie jeśli wymagana strona nie znajduje się w ramie. Wówczas system operacyjny przesyła żądaną stronę z dysku do pamięci. W procesorach 80486 te strony są 4 Kbytes. W przypadku stronicowania nie ma fragmentacji gdyż ramki w pamięci ram mają po 4 Kbytes i strony zajmują tyle samo co pozwala na pełne zapełnienie.
g. System używa dwie tabele directory table oraz page tables.
h. Directory table - jest jedna taka tablica. Zawiera ona 1000 elementów (32 bitowych adresów) każdy wskazuje na jedną page tables. Każda z page tables jest stroną. Liczba page tables zależy od wielkości pamięci ram. Page tables definiuję ramkę w pamięci fizycznej w której znajduje się strona też o pojemności 4 Kbytes.



i. W CR3 mamy adres wskazujący na Directory table. 10 bitów z adresu liniowego wskazuje na 1 element w Directory table. Następnie ten adres wskazuje na początek jednej z Page table. Następne 10 bitów z adresu liniowego wskazuje na element w Page table. On wskazuje na Page frame. 12 bitów z adresu liniowego wskazuje na element w page frame.

j. Z uwagi na to że każda strona ma 4Kbytes. To aby znaleźć odpowiedni element w directory table lub page table nie potrzebujemy 32 bitów wystarczy 10 najbardziej znaczących bitów.



k. 12 najmniej znaczących bitów directory table oraz page table oznaczają atrybuty danej ramki. Atrybuty takie jak czy ramka jest w ramie, czy jest tylko do odczytu czy też zapisu, czy jest dostępna dla użytkownika czy tylko dla

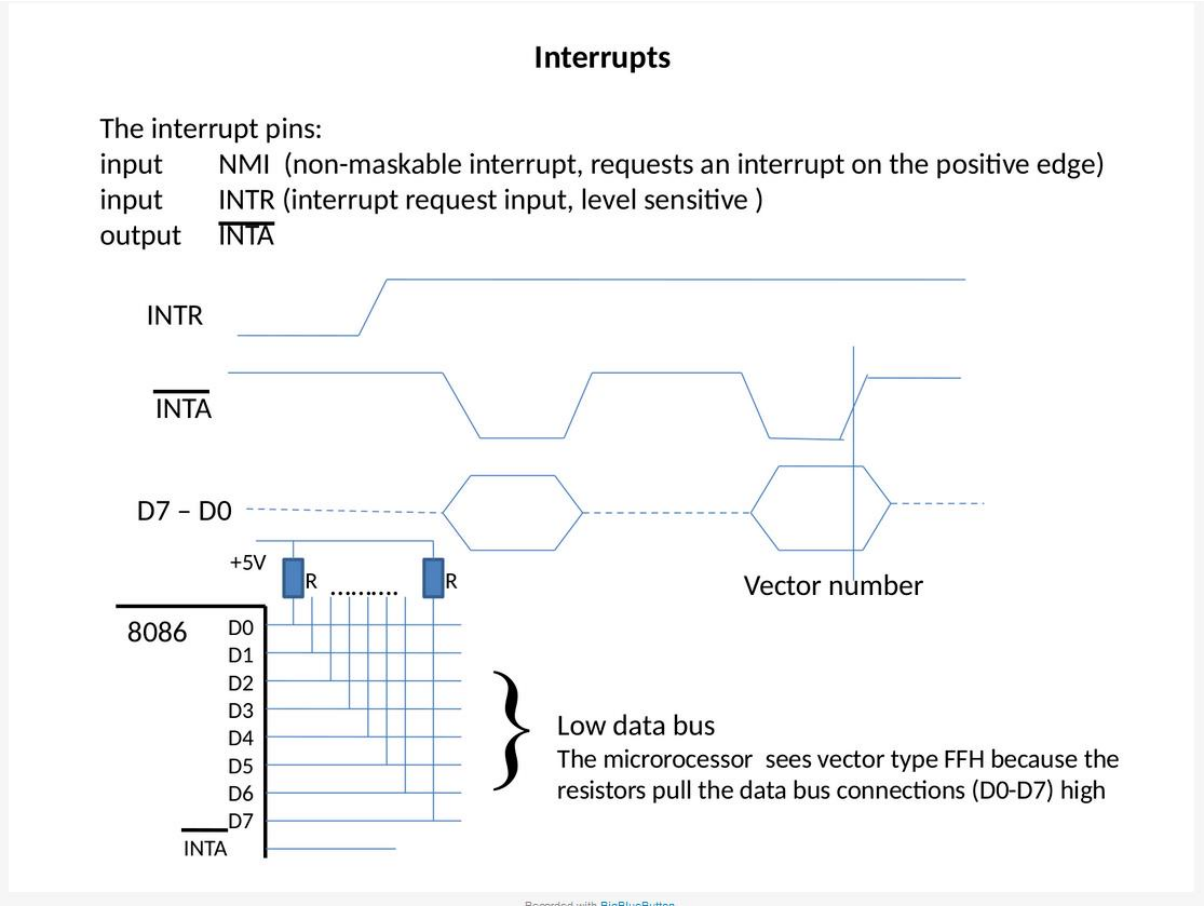
systemu, czy zostało w niej w ogóle coś zapisane, czy ktoś coś z niej odczytywał.

l. W nowszych procesorach Pentium 4 czy core 2 nie ma już segmentacji, może zostać użyte stronicowanie.

4. Przerwania

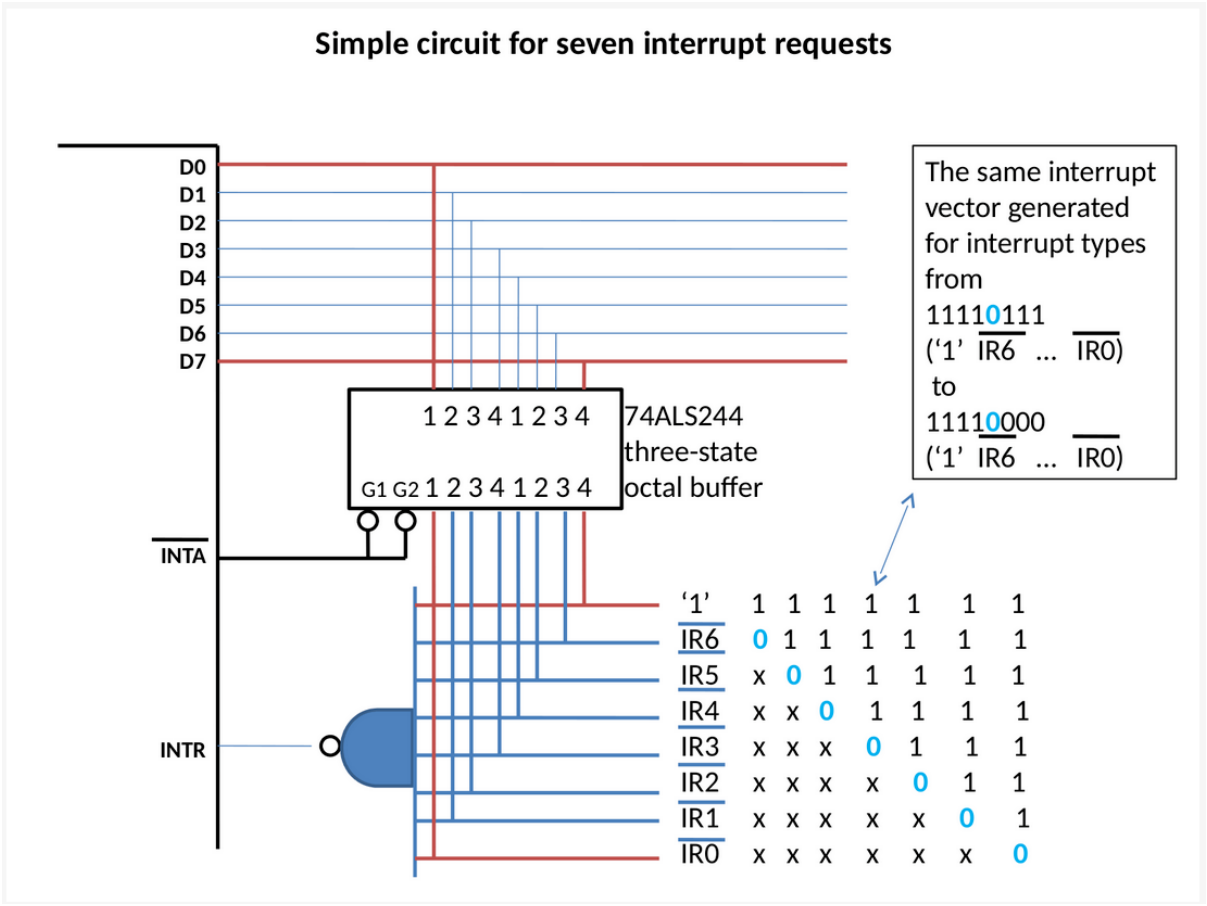
a. W trybie Real (pierwsze procesory intela)
   i. Pojawia się żądanie przerwania
   ii. Mikroprocesor otrzymuje typ przerwania
   iii. Jeśli przerwanie jest zaakceptowane to flaga przerwania zostaje wyzerowana. Następnie zostaje przesłany na stos rejestr statusowy oraz 2 innych rejestrów (CS i IP), które mówią gdzie w programie jest przerwanie
   iv. Procesor otrzymał typ przerwania co pozwala na wskazanie nowego wektora przerwania. Wektórów przerwania jest 256 i one znajduja się na początku pamięci.

b. W trybie Protected
   i. Różnica jest tak że typ przerwania wskazuje na miejsce w tabeli deskryptorów przerwań



c. Żądanie przerwania oznacza 1 na INTR. Gdy pojawi się tak 1 to procesor odpowiada dwukrotnym sygnałem INTA i podczas drugiego sygnału INTA musi się pojawić typ przerwania (8 bitowa liczba która pozwala trafić do miejsca gdzie znajduje się wektor przerwania)

d. Jednym ze sposobów wysłania do mikroprocesora informacji o typie przerwania jest wysłanie na szynę danych d0-d7 wartości 255 czyli samych 1.



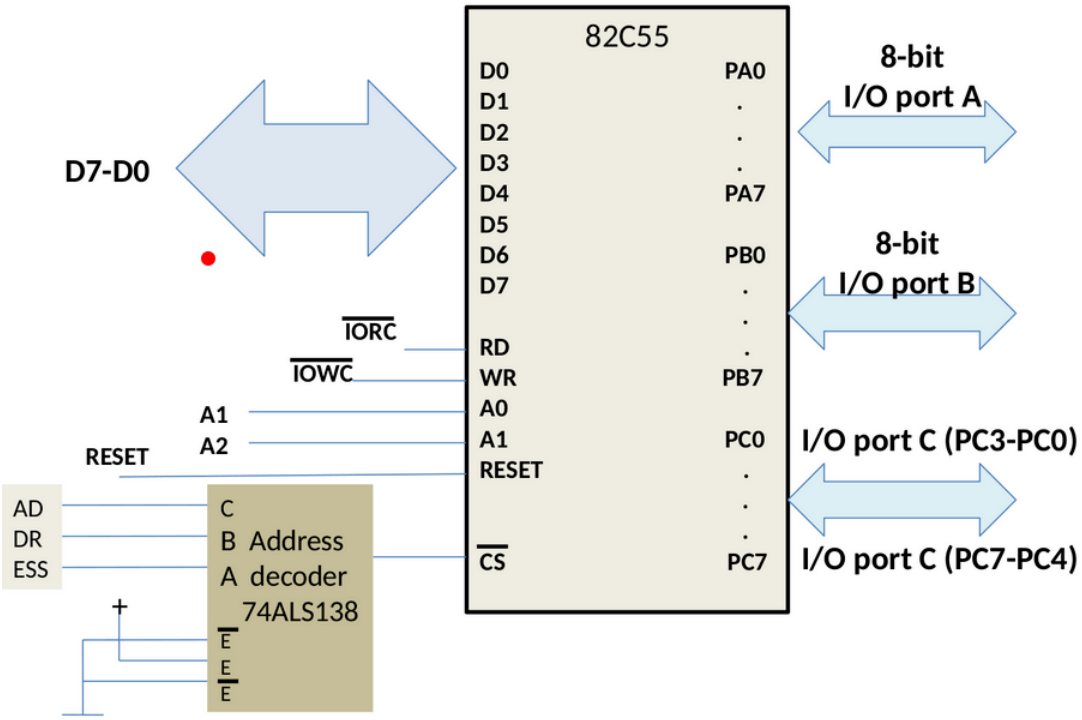
e. Gdy mamy więcej wejść przerwań to możemy zastosować bufor trójstanowy. Łączymy żądania przerwań z szyną danych.
f. Mamy 7 możliwych źródeł przerwań od IR0-IR7. Pierwsza jest '1' dlatego aby używać przerwań od 127 gdyz poniżej tej liczby mogą być przerwania zarezerwowane.
g. '0' na IR6-ir0 oznacza przerwanie IR6 ma najwyższy priorytet
h. Jeśli chcemy mieć więcej niż 6 przerwań to trzeba zastosować specjalne układy zwane sterownikami przerwań.
i. Krok po kroku jak działa przerwanie:
    i. Interfejs wysyła żądanie przerwania do mikroprocesora
    ii. Pojawia się INTA
    iii. Typ przerwania zostaje wysłany do procesora
    iv. Bieżąca wartość PSW, CS, IP zostaje wysłana na stos w celu późniejszego powrotu do programu.
    v. Flaga zezwalająca na przerwanie IF zostaje wyzerowana.
    vi. Obliczanie miejsca w pamięci gdzie znajdują się nowe wartości IP oraz CS
    vii. Następnie jest skok do obsługi programu przerwania
    viii. Wykonanie przerwania
    ix. Zakończenie przerwania rozkazem IRET
    x. Powrót do programu głównego.

5. I/O interface

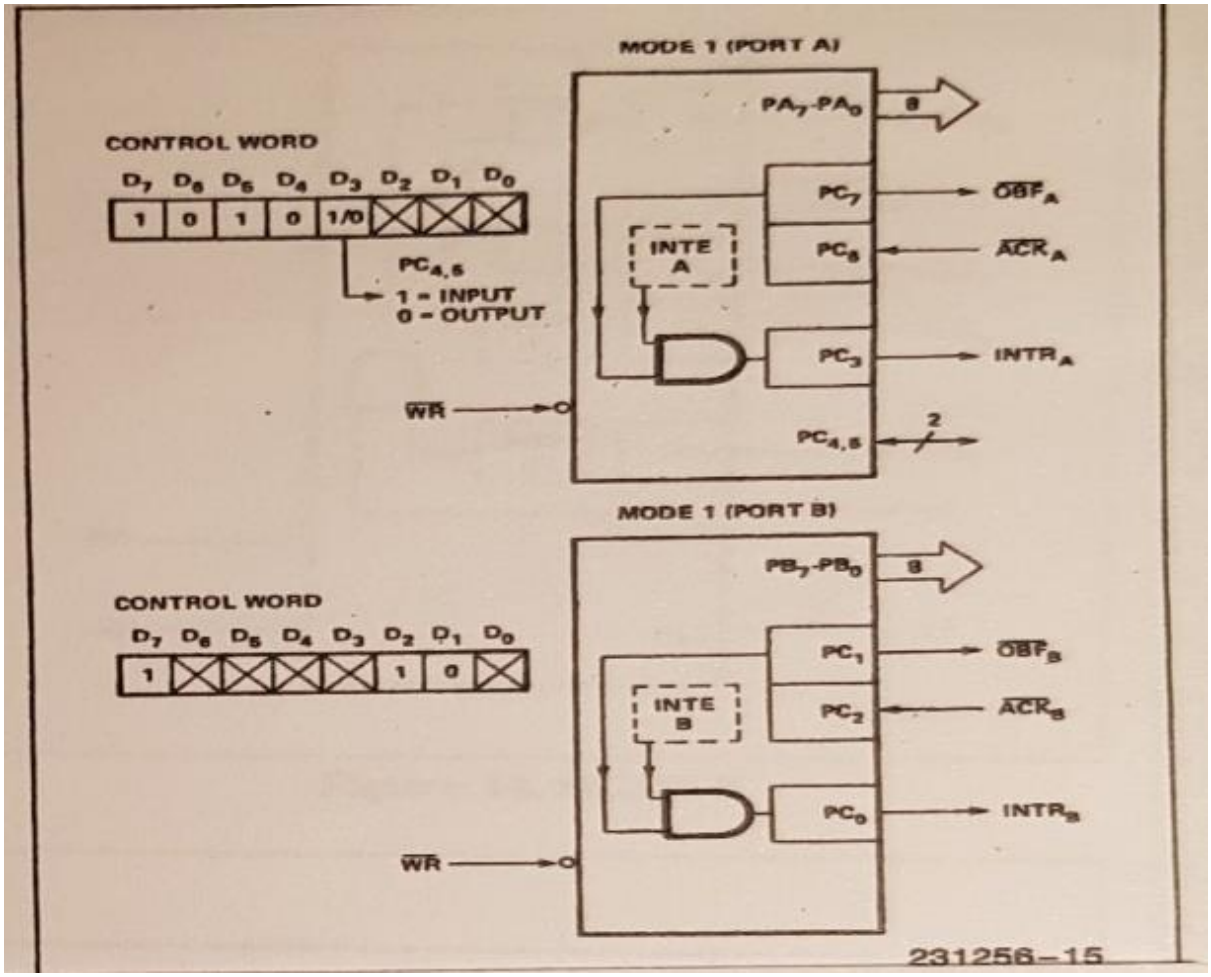
a. W procesorze mamy przestrzeń adresową pamięci oraz przestrzeń adresową urządzeń wejścia wyjścia. Adresy urządzeń wejścia wyjścia to adresy 16 bitowe. Mamy 2 rozkazy które to obsługują IN oraz OUT.
b. Np. IN AX, DX - oznacza to że z jakiegoś portu który jest w rejestrze DX (mamy jakiś zewnętrzny rejestr) i wartość zostaje przepisana do rejestru AX.
c. OUT DX, EAX - oznacza że EAX zostaje zapisane do portu którego adres znajduje się w DX

I/O interface

The **82C55** programmable peripheral interface – low-cost , very popular interfacing device

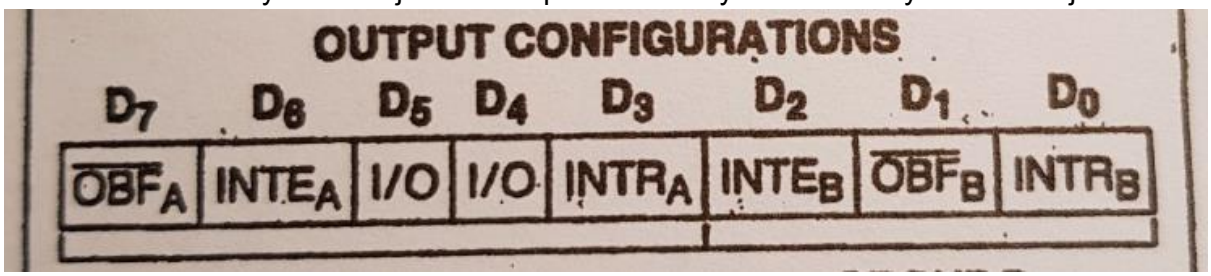


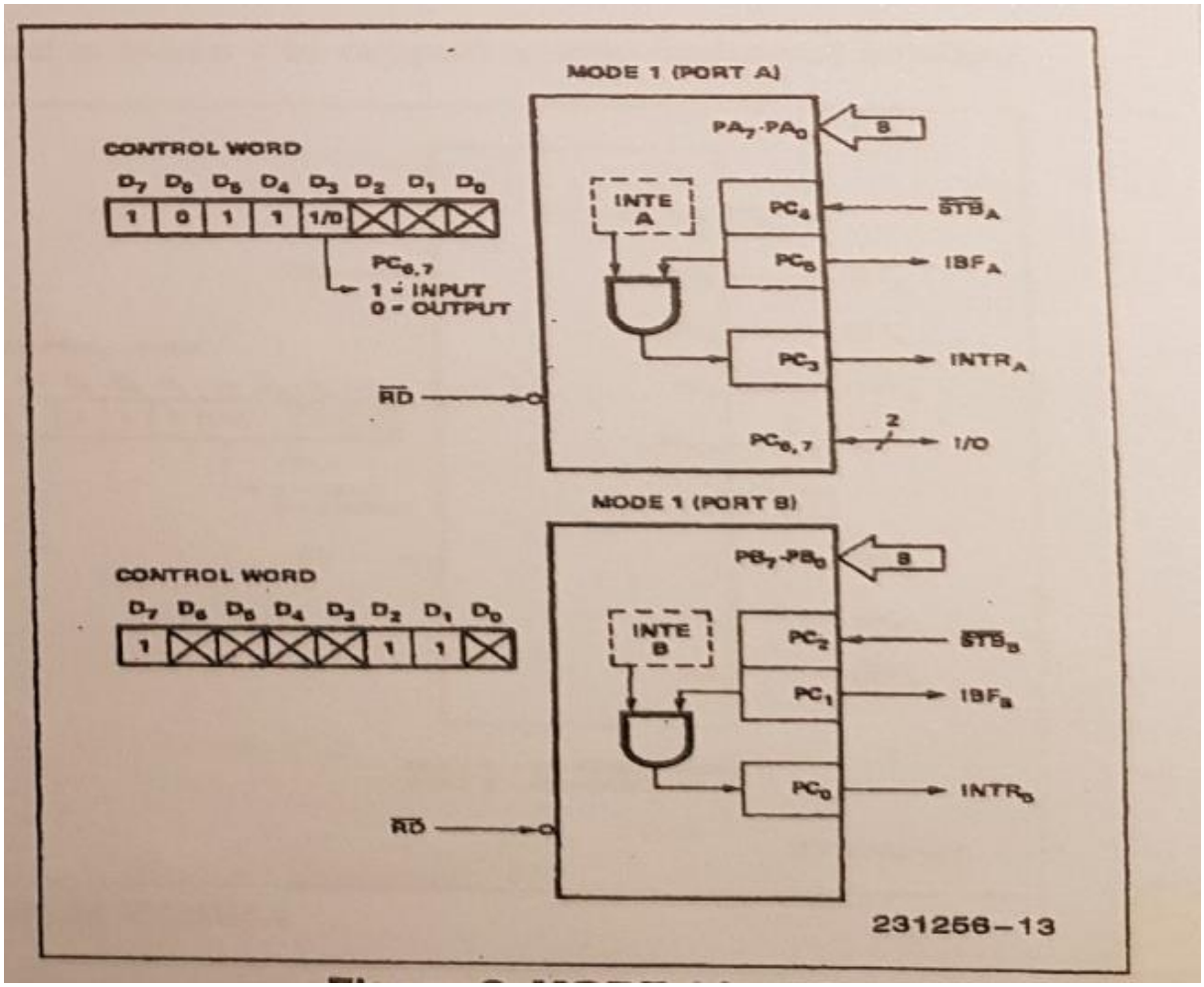
d. 82C55 - jedno z wielu programowalnych interfejsów peryferyjnych. Mamy szynę danych D0-D7 która idzie do procesora oraz mamy 3 porty A, B, C. Wewnątrz są 4 rejestry: rejestr portu A, rejestr portu B, rejestr portu C oraz rejestr sterujący (Który pozwala zaprogramować to urządzenie np. który port pracuje jako wejście a który jako wyjście).
e. CS (Chip select)- jeśli pojawia się na nim 0 oznacza że procesor wybrał to urządzenie.
f. RD, WR - oznacza odczyt lub zapis
g. A0, A1 - oznacza wybór odpowiedniego rejestru



h. 8255 tryb 1 praca jako output - Na przykładzie drukarki. Procesor wysyła do układu przez szynę danych bity(informacja). Przy pomocy sygnału WR zostaje to zapisane np do rejestru A. Następnie mamy sygnał OBF(output buffer full). Jeśli zostało coś zapisane do tego rejestru to ten sygnał OBF zostanie zmieniony na 0 wtedy drukarka wie że ma wydrukować to co jest w rejestrze A. Jak drukarka wykorzysta to co było w buforze wysyła sygnał na ACK i wtedy OBF = 1 oraz może wysłać sygnał INTR do procesora że wykorzystała to co było w buforze i że następny bajt może zostać wpisany do rejestru.

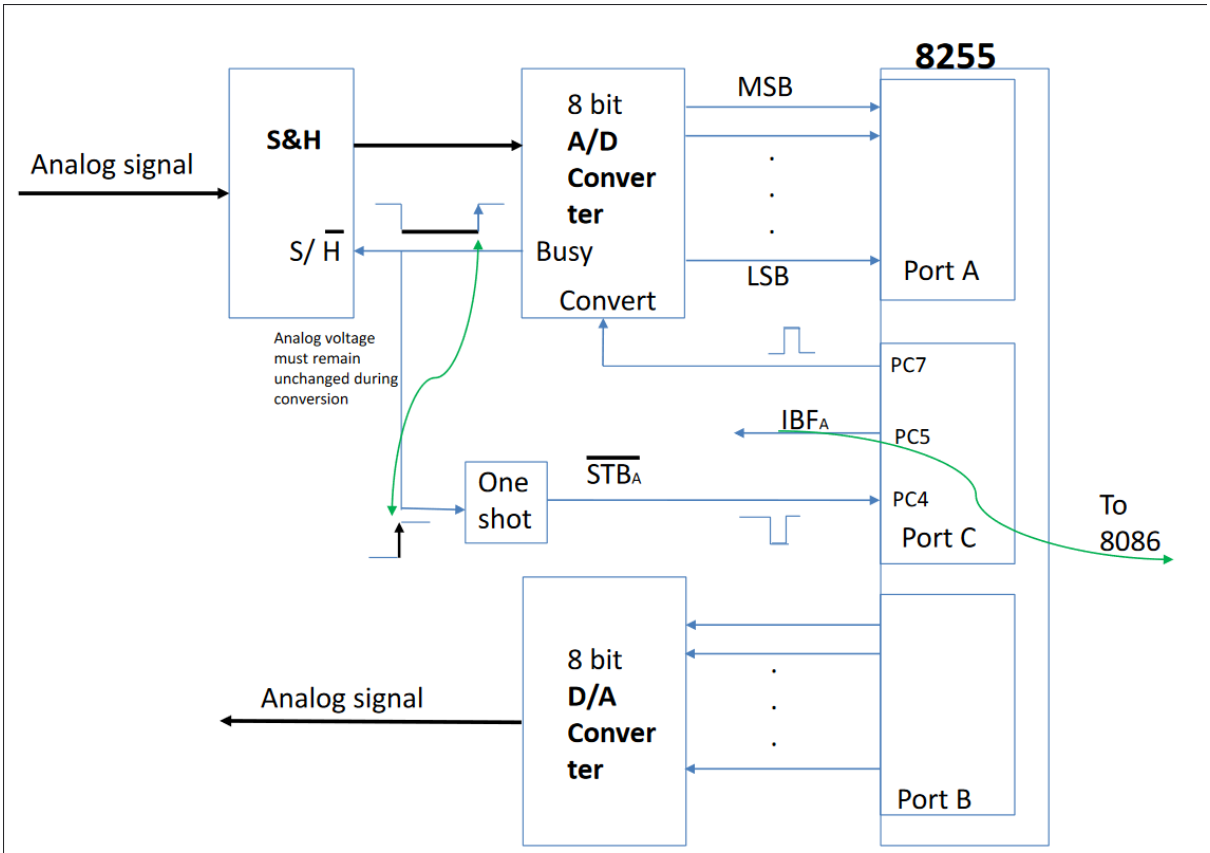
i. Istnieje jeszcze jeden sposób aby procesor mógł wiedzieć że drukarka skończyła pracę. Może on odczytać to co jest w rejestrze portu C i tam na D7 mamy informacje o OBF z portu A. Mamy też wiele innych informacji





j. W trybie input mamy np klawiaturę i idzie sygnał do mikroprocesora. Aby zapisać bity do rejestru portu A wysyła sygnał STB. Gdy już zapisze do rejestru pojawia się sygnał IBF (Input buffer full). Następnie procesor może wykorzystać INTR lub odczytać rejestr portu C i przy pomocy sygnału RD odczytuje zawartość bufora rejestru A. Następnie gdy już odczyta to IBF zmienia stan idzie do klawiatury i można wpisywać kolejne bity.

6. Przykład który wykorzystuje procesor 8086, układ 8255 oraz dwa przetworniki A/D , D/A



a. Schemat połączenia
b. Port B działa jako wyjście w trybie 0
c. Port A działa jako wejście w trybie 1
d. Wyjścia portu C wspomagają pracę portu A w trybie 1
e. Układ S&H(Sample and hold) - układ który próbkuje sygnał wchodzący. Sygnał narasta gdy s/H jest równe 1, gdy jest równe 0 to napięcie jest stałe i przetwornik A/D analizuje sygnał
f. Procesor wysyła do A/D sygnał i wchodzi on na wejście Convert wtedy przetwornik rozpoczyna pracę i na wyjściu Busy jest 0 czyli S&H jest wstrzymane. Jak przetwornik skończy Busy = 1 i S&H pracuje oraz na przerzutniku monostabilnym OneShot jest wygenerowane zero które idzie na PC4 i powoduje to zapisanie liczby z przetwornika A/D do rejestru A. Kolejno na PC5 pojawił się sygnał IBF który informuje procesor o zakończeniu przetwarzania i że może odczytać rejestr A.
g. Następnie wyniki tych przetwarzań są wysyłana na port B i zamieniane na sygnał analogowy poprzez przetwornik D/A

```
Lets assume the following addresses:
FFF8        Port A
FFF9        Port B
FFFA        Port C
FFFB        Control register

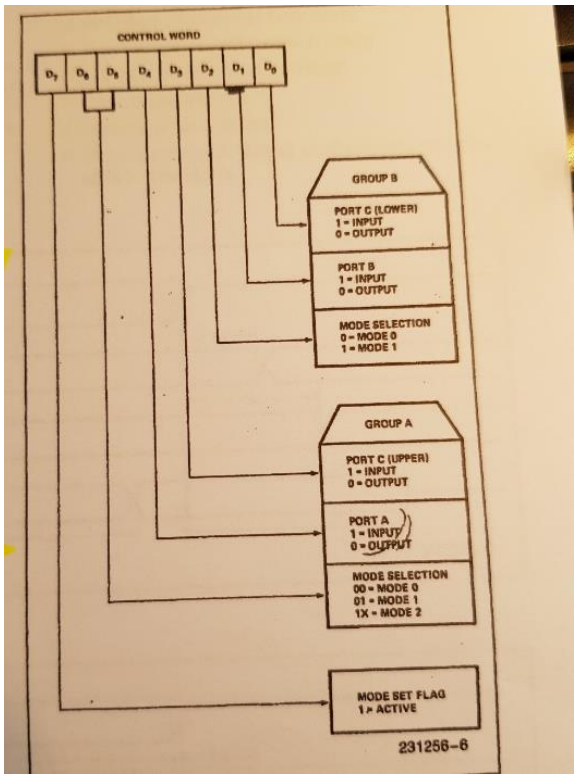
;  Port A in mode 1 - input.  Port B in mode 0 – output.  PC 7 - output
MOV         DX, 0FFFBH
MOV         AL, 10110000B; command byte =   1, 2-bit (PA and PC7-4 ) mode, 1-bit(PA) in/out, 1-bit (PC4-7) in/out,
                           ;2-bit (PB and PC3-0) mode, 1-bit (PB) in/out, 1-bit (PC3-0)in(1)/out(0)
OUT         DX, AL

; Output a pulse to the convert pin of the A/D converter
MOV         DX, 0FFFBH
AOV         AL, 00001111B;Command byte, bit 7th of port C set
OUT         DX, AL        ; PC7 is set
MOV         AL, 00001110B; bit 7th of port C cleared
OUT         DX, AL        ; PC7 is cleared

;|Wait until conversion is finished and write the result to AL
            MOV         DX, 0FFFAH ;
AGAIN:      IN          AL, DX
            TEST        AL, 00100000B  ; test PC5 ,  OPRD1^OPRD2, only flags are affected
            JZ          AGAIN
            MOV         DX, 0FFF8H
            IN          AL, DX

;Output a byte from AL to D/A converter
MOV         DX, 0FFF9H
OUT         DX, AL
```

h. Kod realizujący powyższy schemat
i. FFF8, FFF9 - adresy rejestrów portów A, b itd
j. MOV DX, 0FFFBH - wpisanie do dx adresu rejestru sterującego
k. MOV AL, 10110000B - wpisanie do dolnej części rejestru A słowo 10110000B bierze się ono z tego schematu. Najbardziej znaczący bit musi być 1. Mówi że trzeba mu wierzyć że to ustawia port A w trybie 1 i jako wejście, będzie wspomagany przez bity portu C, a port B będzie działał jako wyjście w trybie 0.



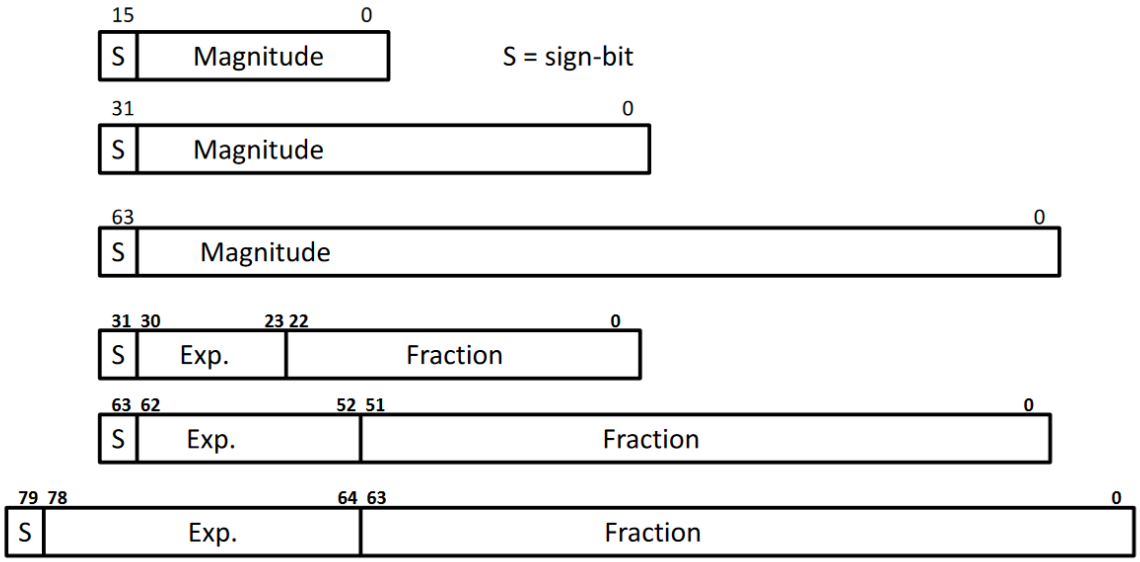
l. OUT DX, AL - oznacza wysłanie tego co jest w AL pod adres znajdujący się w DX.
m. Następny krok to generowanie sygnału który uruchamia przetwornik A/D
n. Kolejno sprawdzamy czy przetwornik zakończył pracę i na PC5 pojawił się sygnał
o. Następnie odczytujemy co znajduje sie na porcie A
p. I ostatni krok to wysłanie tego co było na porcie A na port B

7. Arithmetic coprocessor

THE ARITHMETIC COPROCESSOR

The family of coprocessors (80X87) is able to multiply, divide, add, subtract, find the square root, and calculate the partial tangent, partial arctangent, and logarithms.

Data types include: 16-, 32-, and 64-bit signed integers; 18-digit BCD data (80-bit number, sign-byte + 18 4-bit digits); 32-, 64-, and 80-bit floating-point numbers.



a. Jest to układ przetwarzający liczby
b. Mogą to być 16, 32 i 64 bitowe liczby całkowite lub 32, 64 oraz wewnątrz coprocesora 80 bitowa zmiennoprzecinkowa.
c. Trochę o tych liczbach:

Word integer, 2 bytes, 16 bits, range: -32768 to 32767
Short integer, 4 bytes, 32 bits, range: $-2x10^9$ to $2x\ 10^9$
Long integer, 8 bytes, 64 bits, range: $-9x10^{18}$ to $9x\ 10^{18}$

d. 17 liczb w kodzie bcd. Jest błąd nie 64 bits tylko 80

Packed BCD ,10 bytes, 64 bits, 18 digits D17...D0
[Sign bit,0...0, D17(4 bits),D16(4bits), D15,D14, ... ,D1(4 bits),D0(4 bits)]

e. Liczby zmiennoprzecinkowe

Short **real**, 4 bytes, 32 bits, $\pm 10^{-38}$ to $\pm 10^{38}$
[Sign bit, E7 … E0, F1 … F23], where E7 … E0 = bits of exponent, F1…F23 =bits of fraction or mantissa, bit F0=1 is not present.
Long **real**, 8 bytes, 64 bits, range $\pm 10^{-308}$ to $\pm 10^{308}$
[S, E10 … E0, F1 … F52] , bit F0=1 is not present
Temporary **real**, 10 bytes, 80 bits, range $\pm 10^{-4932}$ to $\pm 10^{4932}$
[S, E14 … E0, F0 … F63], where bit F0=1 is present.

f. W tym coprosesorze znajdują się tylko liczby 80bitowe
g. F0 - występuje tylko w liczbie 80 bitowej
h. Sposób wyliczania liczb bardzo podobny jak na AISD różnica jest taka że od wykładnika odejmujemy BIAS który zależy od typu liczby. To BIAS jest po to aby wykładnik był zawsze dodatni

Real: $(-1)^S (2^{E-BIAS}) (F0\ F1 \ldots) = (-1)^S \times (2^{E-BIAS}) \times 1.F$, where 1=F0 is implied and is not physically stored, mantissa is in the form 1.F

where BIAS = 127 = 7FH for Short real
BIAS = 1023 = 3FFH for Long real
BIAS = 16383= 3FFFH for Temp. real

i. Przykłady zamiany

Example 1. Converting to floating point form

1. Convert the decimal number into binary
   100.25 D = 1100100.01 B
2. Normalize the binary number. **(** Normalized binary number = (**1.** …1) x $\mathbf{2^{exponent}}$ **)**
   1100100.01 B = **1**.10010001 x 2^6
3. E-127=6 → biased exponent E=127+6
   E = 6+127 D = 110 + 0111 1111 B = 10000101
4. 32-bit floating point number
   01000010 11001000 10000000 00000000

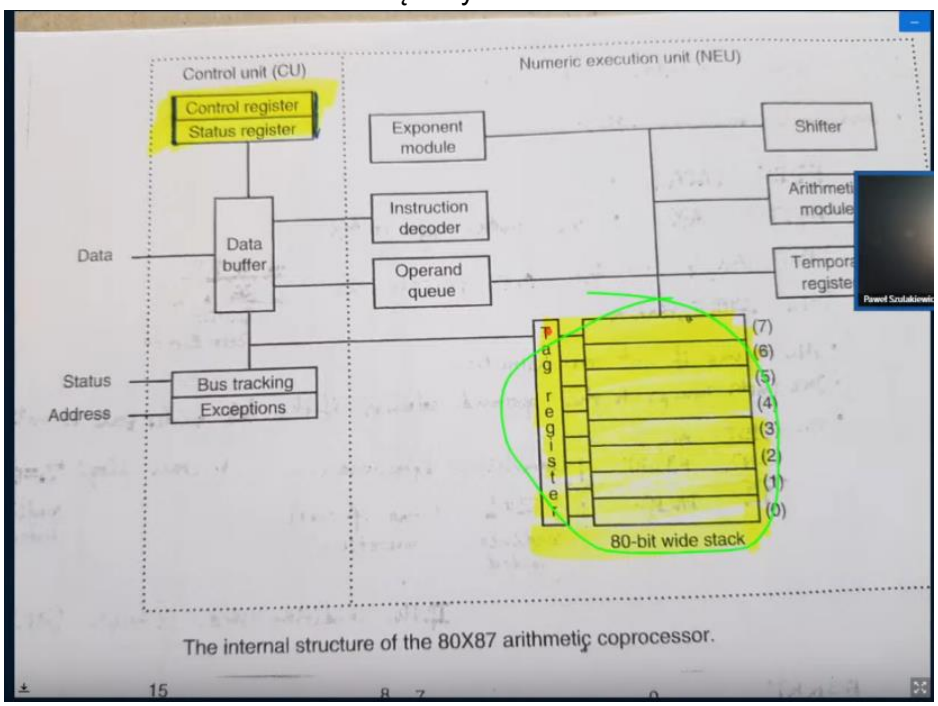
Eample 2. Converting from floating point form

1.Separate the sign, biased exponent and fraction
11000001 11001001 00000000 00000000
2 Convert the biased exponent into a true one
10000011 – 01111111 = 100B = 4D
3. Write the number as a normalized binary number
1.1001001 x 2^4 where **1=F0**
4. Convert into unnormalized binary number
11001.001
5. Convert into decimal
- 25.125

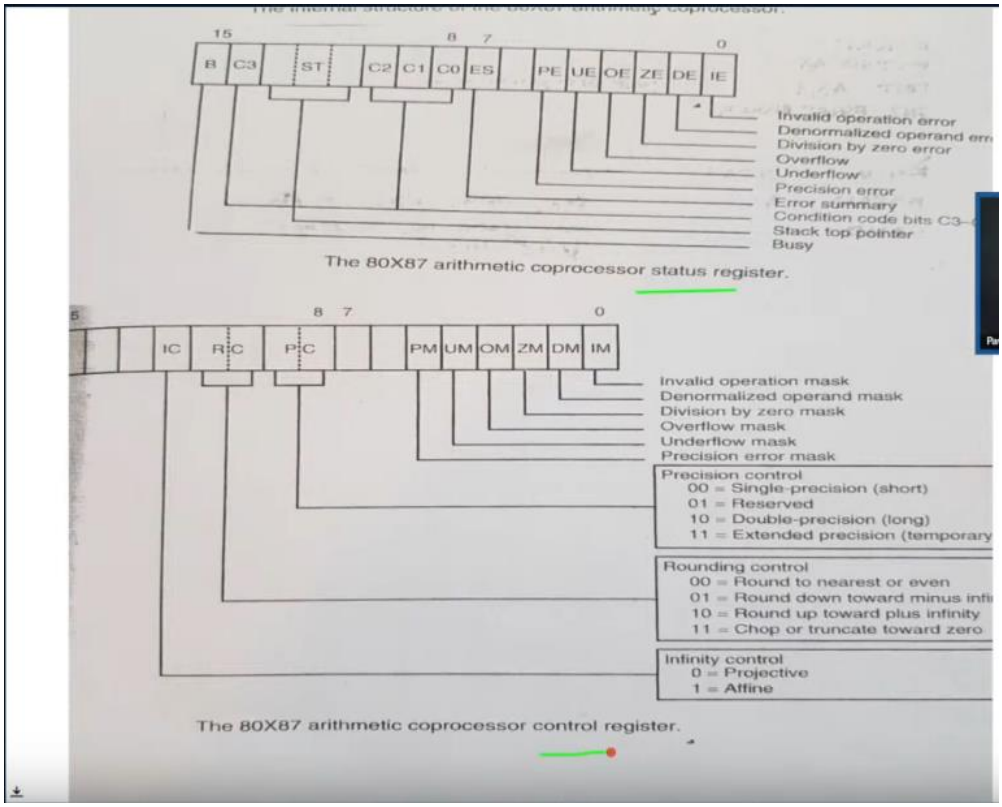
j. Normalizacja liczby binarnej oznacza przesunięcie przecinka tak aby otrzymać 1.coś * 2^x gdzie x to liczba o ile przesunęliśmy przecinek
k. mantysę do wyniku przepisujemy bez najbardziej znaczącej 1
l. Czerwony to bit znaku, niebieski to wykładnik, czarny mantysa.

m. Odwrotna konwersja to od wykładnika odejmujemy BIAS. Do mantysy dopisujemy najbardziej znaczącą 1. Przesuwamy przecinek o tyle pozycji ile nam wyszło z wykładnika. Zamieniamy na dziesiętny i mamy liczbę.

8. Arithmetic coprocessor - struktura wewnętrzna
   a. Coprocesor składa się z: rejestr statusowy, rejestr sterujący, oraz 8 rejestrów bufora każdy po 80 bitów, rozkazów.
   b. Schemat wewnętrzny



   c. Opis rejestrów



d. Rejestr statusowy informuje nas o tym czy nie wykonano błędnej operacji np. dzielenie przez zero, ma on też bity mówice gdzie znajduje się stos.
e. Rejestr sterujący ustawia parametry takie jak: precyzja, zaokrąglenia
f. Przykład wyliczania pole koła

```
; Memory location RADIUS contains radius
; Memory location AREA contains the result
; Both memory locations contain short floating point numbers
AREA_CIRCLE PROC   NEAR          ; Beginning of a procedure. NEAR proc. must
                                 ; appear in the current code segment.
     FLD          RADIUS         ; PUSH radius. (Decrement ST, convert the contents of RADIUS to
                                 ;temporary real and put the result in ST )
                                          RADIUS    (top of stack ST(0) )
                                                    (ST(1))
     FMUL         ST, ST(0)      ; Square radius
                                                    RADIUS²      ST(0)
                                                                 ST(1)
     FLDPI                       ; Push π.  (Decrement ST, π→(ST)  )
                                                    π
                                                    RADIUS²
     FMUL         ST, ST(1)      ; Multiply π x R²
                                                    π x RADIUS²
                                                    RADIUS²
     FST          AREA           ;Store AREA. (Convert (ST) into the destination's real format and store
                                 ; the result in memory location AREA
                                                    π x RADIUS²
                                                    RADIUS²
     FCOMP                       ; Clear stack. (The instruction pops the stack twice, which removes
                                 ; both numbers from the stack to clear it for use by another procedure.)
                                                                 ST(0)
                                                                 ST(1)
                                                                 ST(2)
     RET                         ; Return from procedure
AREA_CIRCLE     ENDP             ; ENDP indicates  the end of  a procedure
```

g. Po lewej są rozkazy po prawej jest stos
h. FLD RADIUS - wprowadza nam na stos RADIUS
i. FMUL ST, ST(0) - mnoży szczyt stosu z komórką 0 stosu i wynik umieszczamy na szczycie stosu
j. FLDPI - wprowadzenie na stos PI
k. FMUL ST, ST(1) - mnożenie szczytu stosu oraz 1 komórki stosu
l. FST AREA - ładujemy to co na stosie do AREA
m. FCOMP - porównanie nas nie interesuje głównym zadaniem tego rozkazu jest wyczyszczenie stosu

Address data bus (linie adresowe A0-A16) – typ wejścia wyjścia – Adres pojawia się w pierwszym okresie zegara T_1 (Cykl zegara dzielimy na T_1 T_2 T_3 T_w [wait] T_4)

BUS HIGH ENABLE/STATUS: During T_1 the bus high enable signal ($\overline{BHE}$) should be used to enable data onto the most significant half of the data bus, pins D_{15}–D_8. Eight-bit oriented devices tied to the upper half of the bus would normally use $\overline{BHE}$ to condition chip select functions. $\overline{BHE}$ is LOW during T_1 for read, write, and interrupt acknowledge cycles when a byte is to be transferred on the high portion of the bus. The S_7 status information is available during T_2, T_3, and T_4. The signal is active LOW, and floats to 3-state OFF in "hold". It is LOW during T_1 for the first interrupt acknowledge cycle.

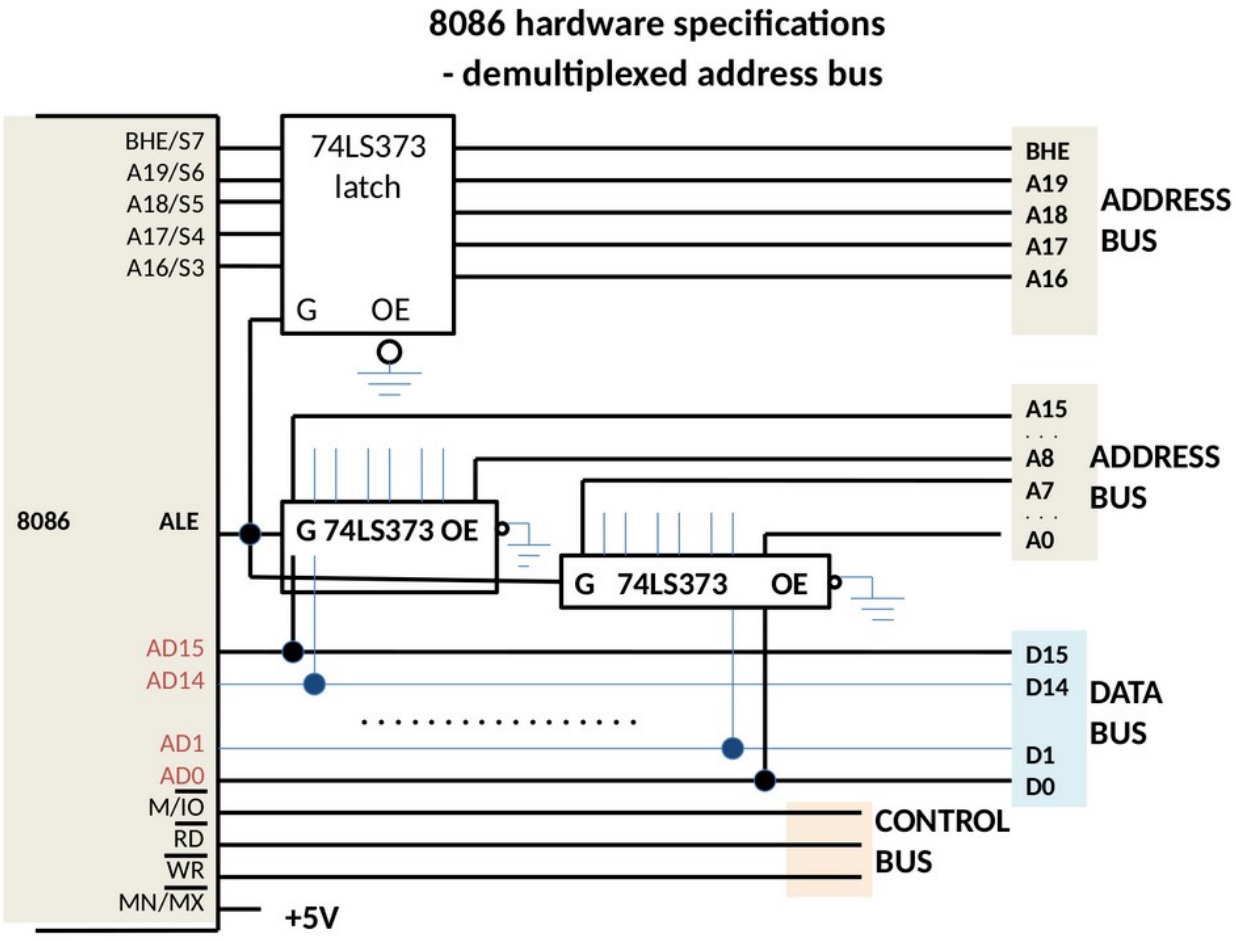
$\overline{BHE}$	A_0	Characteristics
0	0	Whole word
0	1	Upper byte from/to odd address
1	0	Lower byte from/to even address
1	1	None

T_{wait} – jest to dodatkowy takt zegara dzięki któremu jeżeli pamięć działa z inną prędkością niż procek, to możemy wprowadzić sygnał czekania w którym procesor nic nie będzie robił.

BHE i A0– BUS HIGH ENABLE – jeżeli mikroprocesor zapisuje coś do pamięci, pojawia się adres do pamięci. W rozkazie jest informacja czy do pamięci ma być zapisane pod tym adresem słowo 16 bitowe (całe, tylko górne bity, tylko dolne i td) Identyfikacja odbywa się w połączeniu z bitem A_0

ADDRESS/STATUS (linie adresowe A16-A19

Połączenie między mikroproceosorem a szyną danych i adresową – rysunek:



Tylko pokazał, nie omawiał

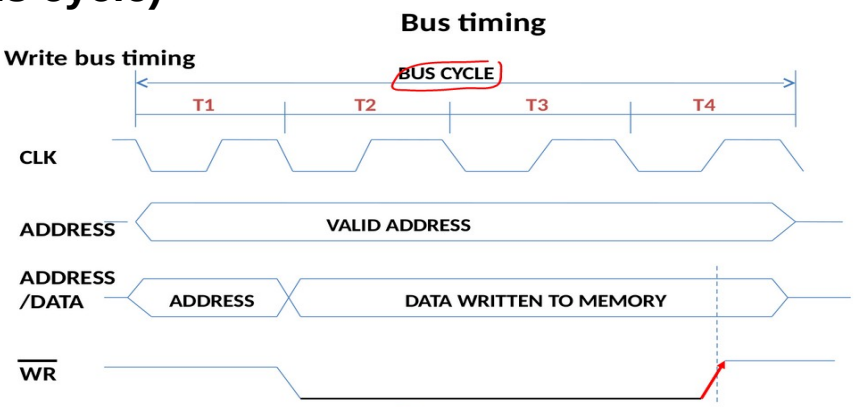
Cykl maszynowy (Bus cycle)

Jak odbywa się zapis i odczyt w mikroprocesorze (4 takty zegara). Okres nazywamy Stanem

W czasie T1 pojawia się adres na szynie adresowej, ten adres musi być zapisany do rejestru przy pomocy sygnału ALE.

W czasie T2 dane są zapisywane do bufora

W czasie T3 (Wait), mamy stan czekania



T1,T2,T3,T4 - four system-clocking periods (T states)
During T1 the memory or I/O location address is sent out. The control signals ALE, DT/ R, M/IO are also output.
During T2 the WR and DEN (turns on the data bus buffers) signals are sent out, and the data to be written appear on the data bus. (READY is sampled at the end of T2. If READY=0 T3 becomes a wait state (Tw).
In T4 the trailing edge of the WR signal transfers data to the memory or I/O.

Następnie na narastającym zboczu (przerywana linia na rysunku) następuje zapis do komórki pamięci

Maksymalny i minimalny tryb pracy

W trybie maksymalnym mamy dodatkowy sterownik szyny generujący tryby pracy (enable i td), tym się różnią

Minimum mode operation

Minimum mode operation is the simplest way to operate the 8086/8088 microprocessors. All the control signals for the memory and I/O are generated by the microprocessor.

Maximum mode operation

Maximum mode operation requires the additional external bus controller – the 8288 bus controller. Maximum mode is used when the system contains the external arithmetic coprocessor (8087).

8086 Zarządzanie pamięcią

Pamięć w tym mikroprocesorze dzielimy na 4 segmenty:

- **Segment kodu (CODE [CS]) –** zawiera program (Jeżeli wykonujemy jakąś instrukcje predefiniowaną
- **Segment stosu (STACK [SS]) -** zawiera stos (jeżeli robimy push,pop i td)
- **Segment Danych (DATA [DS]) –** zawiera dane (jeżeli robimy np.: MOV)
- **Segment dodatkowy (EXTRA [ES]) –** też na dane

Rejestry mają 16 bitów jednak są uzupełnione o 4 zera na końcu (razem 20)

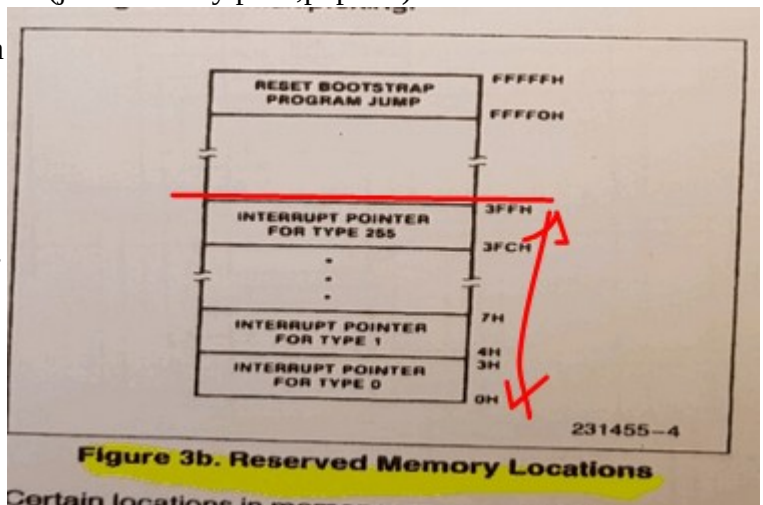


Figure 3b. Reserved Memory Locations

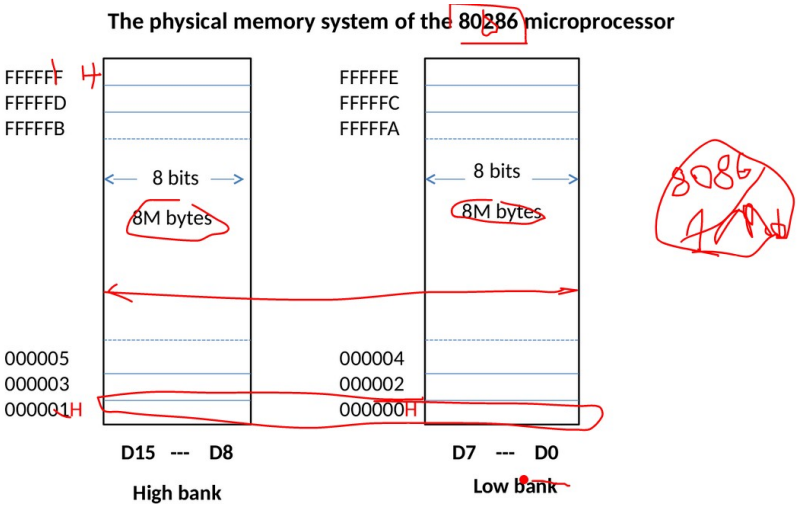
8086 ma jeden megabajt pamięci 2^{20} (od A0 do A19 – czyli 20 rejestrów)

W celu zarezerwowania pamięci, możemy skorzystać z 256 adresów przerwań (intel mówi na to b **TYPE)**

Mikroprocesor 80286 (2 banki pamięci, 16 mega, słowa 16 bitowe)

Różnice

Od 8086 różnił się wielkością pamięci - 16 mega [1 mega w 8086]

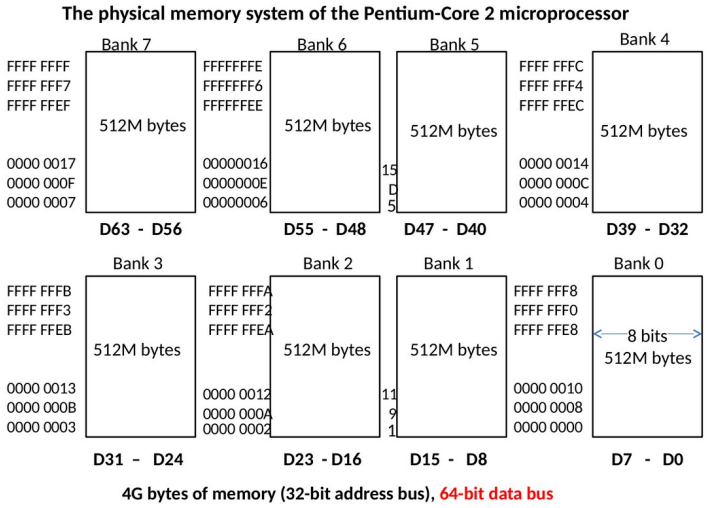


oraz wprowadzono memory management unit (układ zarządzania pamięcią z segmentacją). Umożliwiało to realizacje wielozadaniowość

Nowy tryb protected

Pentium-Core 2

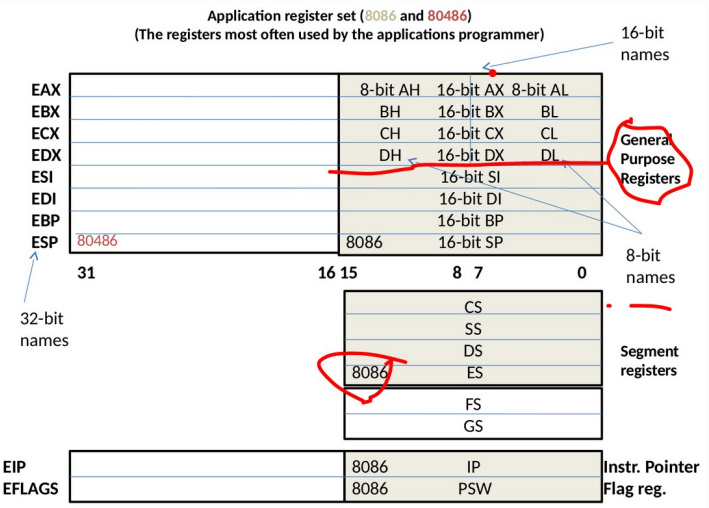
- Miał 8 banków pamięci adresowej
- Słowa 64 bitowe
- Szyna adresowea 32 bitowa
- 4 GB pamięci



Najczęściej używane rejestry 8086 i 80486

General Purpose Register – rejestry ogólnego przeznaczenia – 4 pierwsze z nich (AX, BX, CX, DH) mają pewne specyficzne właściwości:

- można je podzielić na rejestr banku dolnego (low) i górnego (High) np. BX → {BL i BH} patrz rysunek obok
- np. CX – to licznik counter



kolejne rejestry:

- BP – BASE POINTER

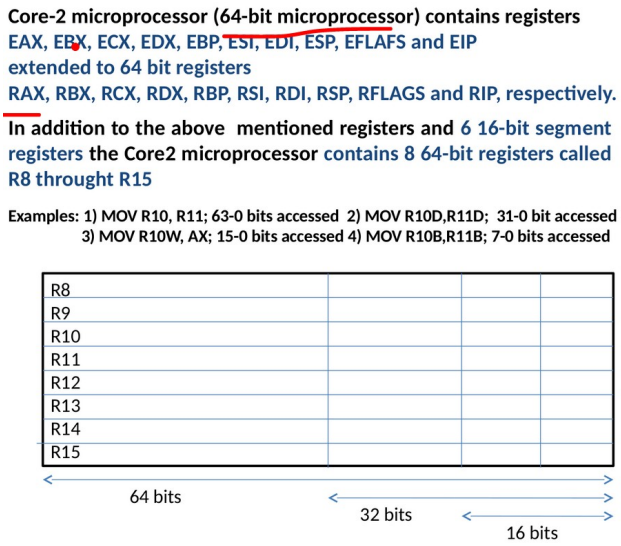
- SP – STOCK POINTER
- SI/DI – Source/Destination index

To są takie typowe zastosowania tych rejestrów ale w dalszym ciągu są one ogólnego przeznaczenia czyli nie muszą pełnić tej roli którą mają przypisaną a dowolną wedle uznania

Należy zauważyć że w procesorze 8086 nie ma rejestru PC a zamiast tego jest **Instruction Pointer** i **PSW (rejestr flag)**

Na rysunku powyżej:

- Niebieskie rejestry dotyczą procesora 8086 (16 bitowego)
- Procesor 80486 (32 bitowy), tworzą rejestry niebieskie oraz rozszerzone o kolejne 16 bitów (rejestry białe na rysunku)
  - Czyli np. rejestr EAX to rozszerzony o dodatkowe 16bitów rejestr AX
- Tak samo sprawa ma się z prockami 64 bitowymi, tam np. do EAX dodajemy 32 bity i nazywamy go wtedy RAX i td
  - dodatkowo w 64 bit proc mamy jeszcze rejestry od R8 do R15



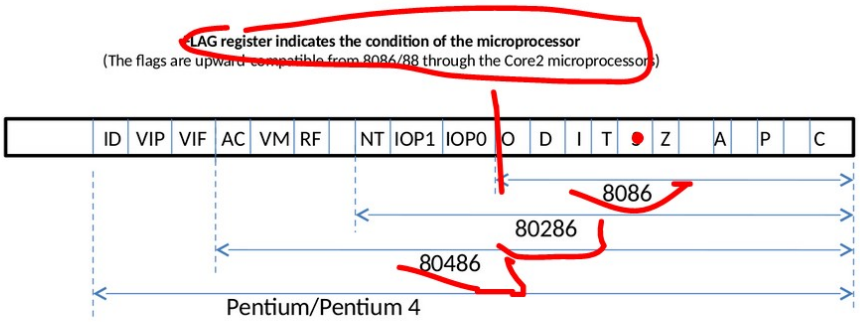
- Tam gdzie jest napisane 8086 – dotyczy tylko 8086

Różnica 80486 (32 bit)

Wprowadzono mechanizm stronnicowania (pagination)

Rejestr wskaźników – flagi

Obok pokazane są flagi w poszczególnych procesorach



IOP – I/O privilege level (used in protected mode)
NT – nested task ((the current task is nested within another task)
RF – the resume flag
VM – virtual mode operation (allows multiple DOS memory partitions that are 1M byte in length to coexist in memory
AC – the alignment check bit
VIF – virtual interrupt
VIP – virtual interrupt pending
ID – the Pentium-Pentium 4 microprocessors support the CPUID instruction
(TI486 Microprocessor Reference Guide , pages 2-15, 2-16)

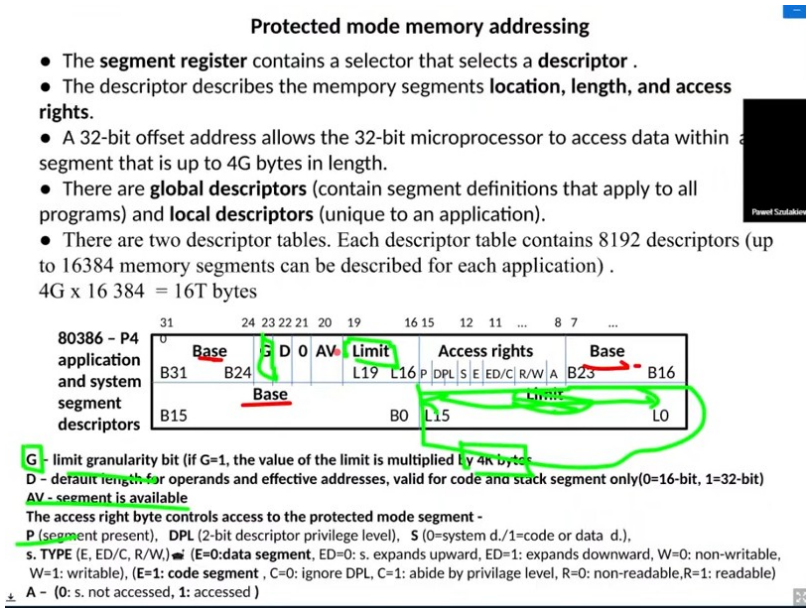
Tryb Real

Rejestr segmentowy uzupełniony o 4zera wskazuje na początek segmentu

Tryb protected

Rejestr segmentowy nie wskazuje na początek segmentu a wskazuje na deskryptor. Deskryptor składa się z 8 bajtów. Mamy pewną tablicę w pamięci i tam znajdują się deskryptory. Dzielimy je na:

- **Globalne deskryptory GDTR (globalna tablica deskryptorów)**
- **Lokalne deskryptory LDTR (Lokalna tablica deskryptorów**



Przykład deskryptora masz na obrazku, deskryptor generalnie służy do tego żeby opisywać segment (Zawiera adres początku segmentu, wielkość segmentu, granulacje [w czym liczony jest wielkość segmentu bajtach, 4K bajtach i td], prawa dostępu do segmentu

Skonczyłem na 65:00 Memory Addressing modes

Tryby adresowania pamięci - Memory Addressing modes

Offset – przesunięcie od początku segmentu do miejsca którego sięgamy.

Czyli idziemy pod dany bazowy adres a potem przesuwamy się o dany offset i mamy punkt docelowy

Na przeźroczu masz wzór na wyliczanie offsetu przy różnych trybach adresowania

Memory addressing modes

ADDRESSING MODE	BASE	INDEX	SCALE FACTOR (SF)	DISPLA-CEMENT (DP)	OFFSET ADDRESS (OA)
Direct				X	OA = DP
Register indirect	X				OA = [BASE]
Based	X			X	OA = [BASE] + DP
Index		X		X	OA = [INDEX] +DP
Scaled index		X	X	X	OA = ([INDEX]*SF) + DP
Based index	X	X			OA = [BASE] = [INDEX]
Based scaled index	X	X	X		OA=[BASE] + ([INDEX]*SF)
Based index with displacement	X	X		X	OA= [BASE] + [INDEX]+DP
Based scaled index with displacement	X	X	X	X	OA= [BASE] + ([INDEX]*SF + DP

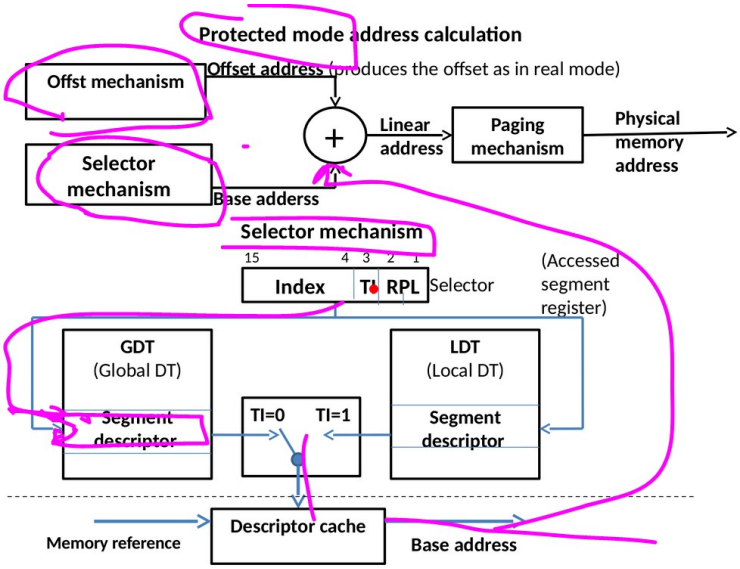
Co do rysunku obok:

bit **TI** oznacza którą tablicę deskryptorów wybieramy,

RPL – requested privileg level – kontrola dostępu

offset adress połączony z base adress nazywamy **Linear address**

Jeżeli nie ma stronnicowania (o tym potem) to stanowi on **adres fizyczny**



Instrukcje

powiedział że nie będziemy tego używać i wyjebka na to więc nie opisuje.

Przerwania 8086 – INTERRUPTS

Wektory przerwań są na początku pamięci, mamy różne typy przerwań (0-255), każdy wektor przerwań to 2 słowa 14 bitowe

Co się dzieje gdy zaczyna się przerwanie?

REAL MODE

- Bieżąca instrukcja musi się wykonać do końca,
- flagi przerwań lecą na stos
- flaga IF (interupt flag – zezwolenie na przerwania) zostaje ustawiona na 0, blokuje tym samym inne przerwania
- Zawartość 2 rejestrów CS (wsk segmentu) i IP wlatuje na stos
- odpowiedni wektor przerwań trafia do procesora
- rozpoczyna się program obsługi przerwania

Interrupts

Real mode interrupt

When the microprocessor determines an active interrupt the following sequence of events occurs:

- The contents of the flag reg. are pushed onto the stack.
- The interrupt (IF) and trap (TF) flags are cleared.
- The stack ← the contents of CS
- The stack ← the contents of IP
- The interrupt vector contents fetched and placed into IP and CS (the interrupt service procedure addressed by the interrupt vector)

Protected mode interrupt

The interrupt vector numer is used to locate an entry in the interrupt descriptor table (IDT)

In place of interrupt vectors, protected mode uses a set of **256 8-byte interrupt descriptors** that are stored in **an interrupt descriptor table (IDT).**

The protected Mode Interrupt Descriptor

31 … 16	15 14 13 12 11 … 8	7 … 0	
OFFSET 31-16	P DPL 0 1110	0 0 0 Parameters	4
SELECTOR 15-0	OFFSET 15-0		0

P – Segment present, DPL – Descriptor privilege level,

The address of the interrupt service procedure is in the form of a segment selector and a 32-bit offset address.

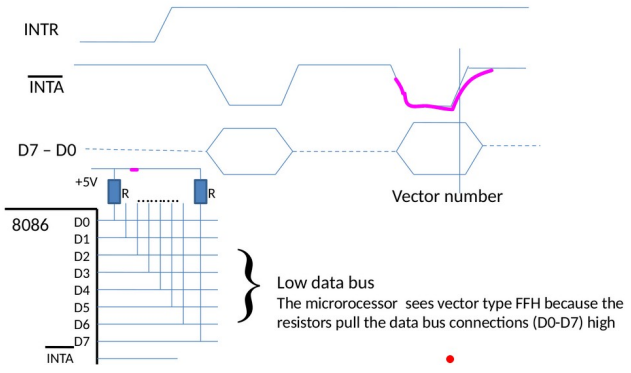
Interrupts

The interrupt pins:

input NMI (non-maskable interrupt, requests an interrupt on the positive edge)

input INTR (interrupt request input, level sensitive)

output $\overline{\text{INTA}}$



- IRET – po zakonczeniu, powrót z przerwania do programu głównego
- zawartość rejestrów która była wrzucona na stos, wraca do nich

W kontekscie rysunku po prawo:

pierwsze 0 potwierdza że przerwanie zostało zaakceptowane

W czasie drugiego zera musi zostać określony typ przerwania (0-255) dzięki czemu wiemy jak obsłużyć przerwanie

Układ 8255 – programmable peripheral interface

Zalety: niski koszt, popularność układu

Zawiera 3 porty wejścia wyjścia i 8 bitową szynę danych

znajdują się w nim 4 rejestry (po 1 dla 3 portów oraz rejestr kontrolny)

rejestr kontrolny – zawiera informacje na temat tego czy port to wyjście, wejście, w jakim trybie pracuje

3 tryby pracy: mod 0, mod 1, mod 2 (mod 2 się nie zajmujemy)

tryb 0 – najprostrzy, dane pojawiają się na wejściach D0-D7 i trafią na wyjście lub na odwrót w zależności od tego czy porty pracują jako wejścia czy wyjścia

- jeżeli port pracuje jako wyjście to z mikroprocesora dane trafiają do rejestru danego portu
- jeżeli jak wyjście to leci od razu do procka i pojawia się jako słowo 8 bit

tryb 1 – strobbed input/output – w tym trybie mamy specjalny sygnał który WR który zezwala na zapis do rejestru

Jako Output

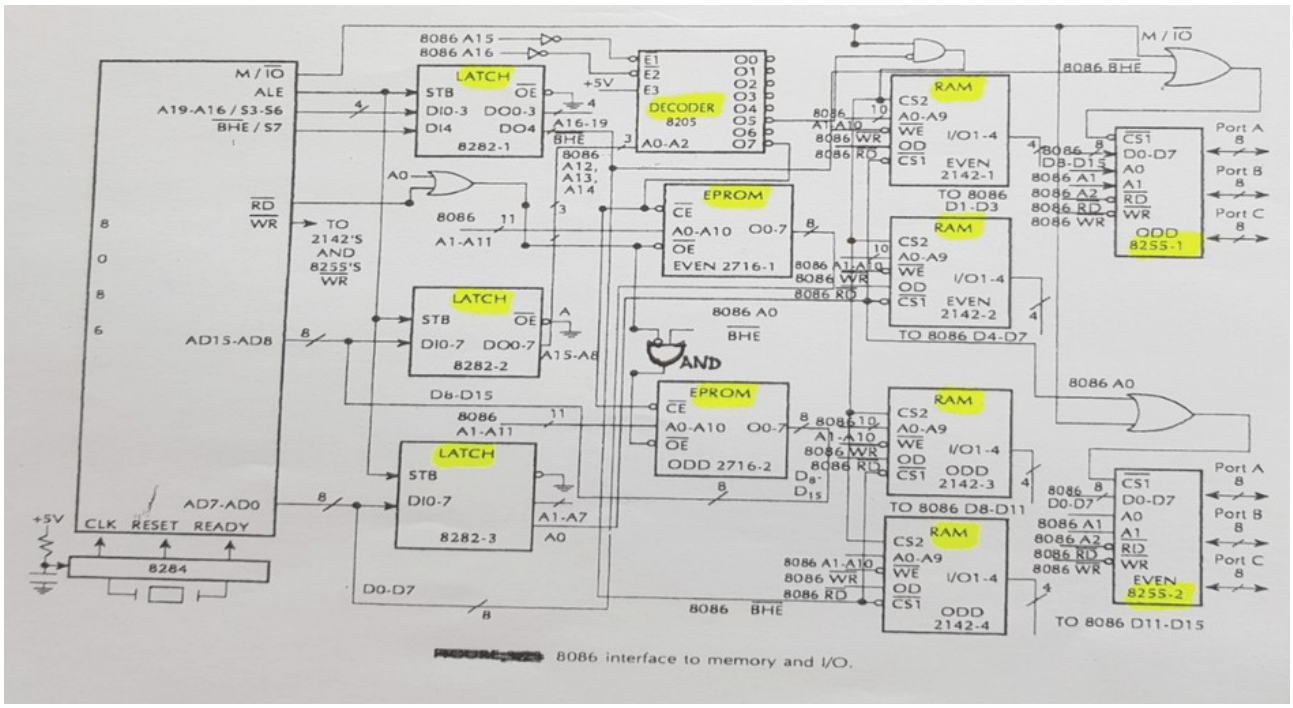
- w tym trybie mamy specjalny sygnał który WR który zezwala na zapis do rejestru
- Po zapisie, na wyjściu pojawia się sygnał OBF (**Output buffer full**) który informuje o tym że rejestr jest zapełniony danymi, jest to informacja dla urządzenia wyjściowego
- urządzenie wyjściowe odpowiada sygnałem **ACK** że przyjeło tą informacje
- przyjecie sygnału ACK spowoduje że OBF zaneguje się, bo już wiadomo że urządzenie wyj wie o tym że bufor jest pełny i odczytuje dane
- następuje poinformowanie procesora o tym że może już nadać następne dane. Można to zrobić na 2 sposoby:
    - poprzez sygnał INTERUPT przerwania który idzie do procka
    - procesor może przeczytać zawartość słowo w porcie

Jako Input:

- sygnał strobujący wchodzi do bufora

- pojawia się sygnał (**input buffer full)**
- wywoływane jest przerwanie do procesora
- procesor odczytuje dane z bufora za pomocą sygnału read

Mikrokomputer 8086 – Schemat



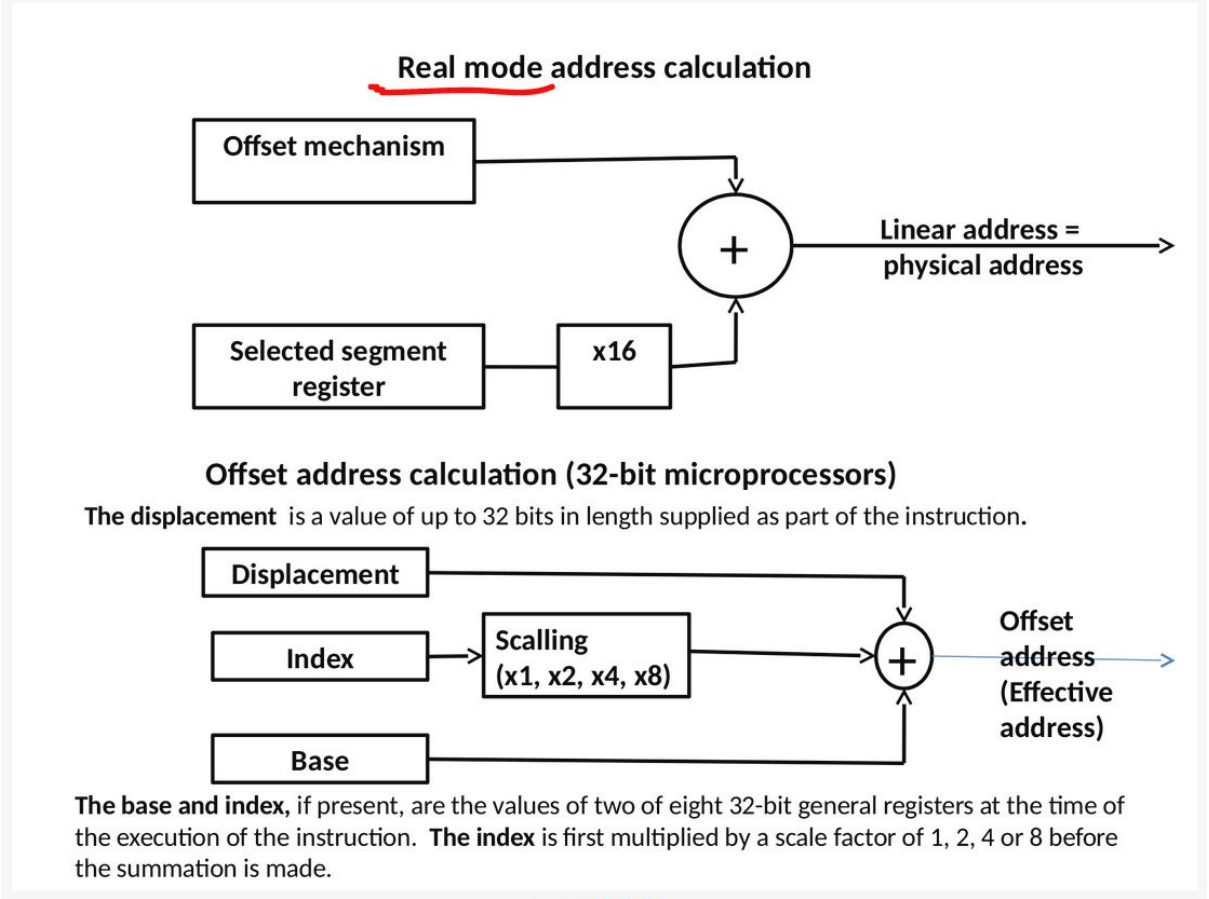
8086 interface to memory and I/O.

Na schemacie mamy :

- procesor 8086
- 2 x pamięć eprom
- 2 układy 8255
- dekoder – służy do wyboru pomiędzy dwoma epromami albo RAM
- 4 x ram

Początek wykładu 6

1. **Real mode address calculation**

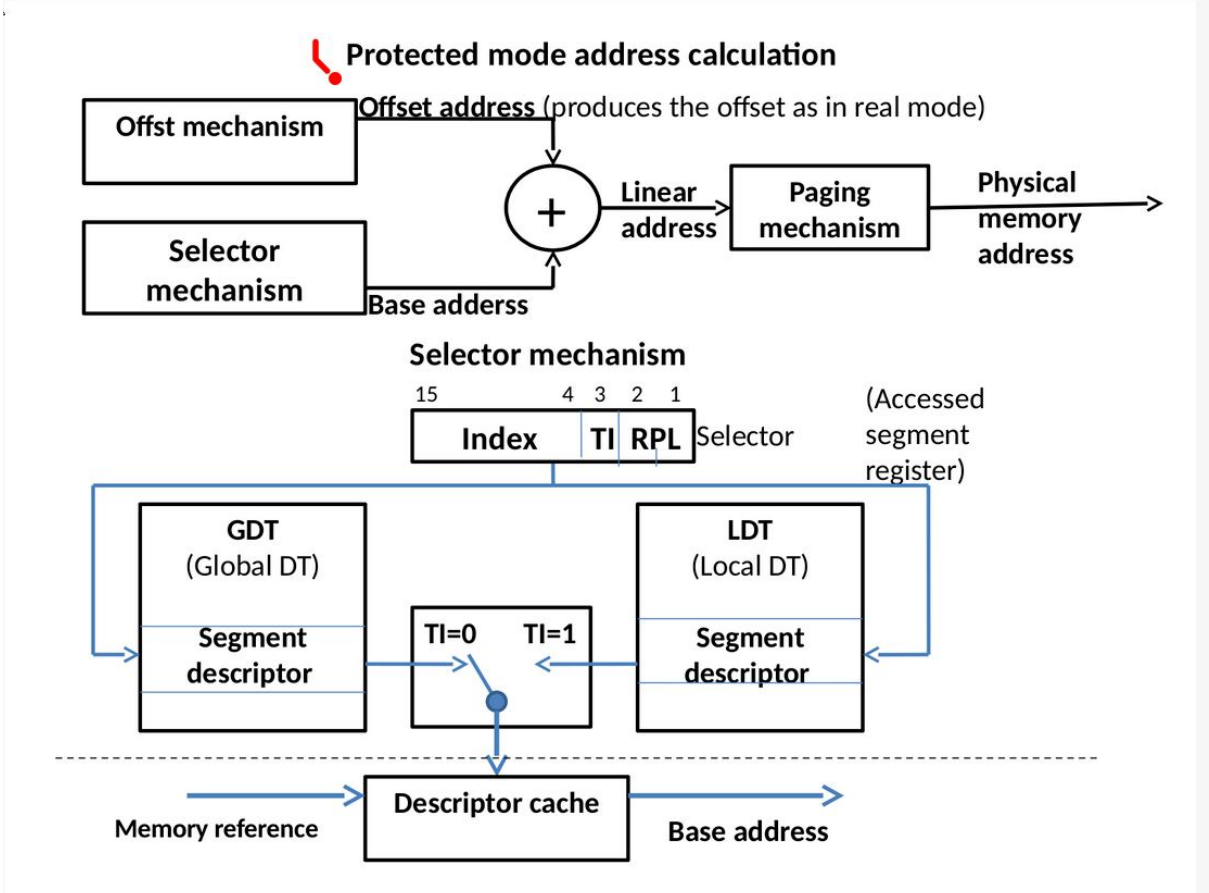


a. Real mode address calculation - jest to tryb który określa jak jest wybierany adres komórki do której chcemy się odwołać w procesorach intel 8086.
b. Zawiera on rejestr segmentowy (4 rejestry segmentowe) który były mnożone przez 16. Do tego był dodawany offset(przesunięcie) w ramach segmentu i tak powstawał adres fizyczny
c. Offset address calculation - schemat pokazujący w jaki sposób generowany jest offset. Generowanie offsetu ma wiele kombinacji może być to suma wszystkich 3 składników (displacement, index, base), mogą być to też pojedyncze składniki np. samo index itd.
d. Displacement - liczba w rozkazie (przesunięcie)
e. Base - zawartość jakiegoś rejestru bazowego
f. Index - zawartość rejestru indeksowanego przemnożona przez 1, 2, 4, 8

ADDRESSING MODE	BASE	INDEX	SCALE FACTOR (SF)	DISPLA-CEMENT (DP)	OFFSET ADDRESS (OA)
Direct				X	OA = DP
Register indirect	X				OA = [BASE]
Based	X			X	OA = [BASE] + DP
Index		X		X	OA = [INDEX] +DP
Scaled index		X	X	X	OA = ([INDEX]*SF) + DP
Based index	X	X			OA = [BASE] = [INDEX]
Based scaled index	X	X	X		OA=[BASE] + ([INDEX]*SF)
Based index with displacement	X	X		X	OA= [BASE] + [INDEX]+DP
Based scaled index with displacement	X	X	X	X	OA= [BASE] + ([INDEX]*SF + DP

a. Tabelka pokazująca w jaki sposób można wygenerować offset.

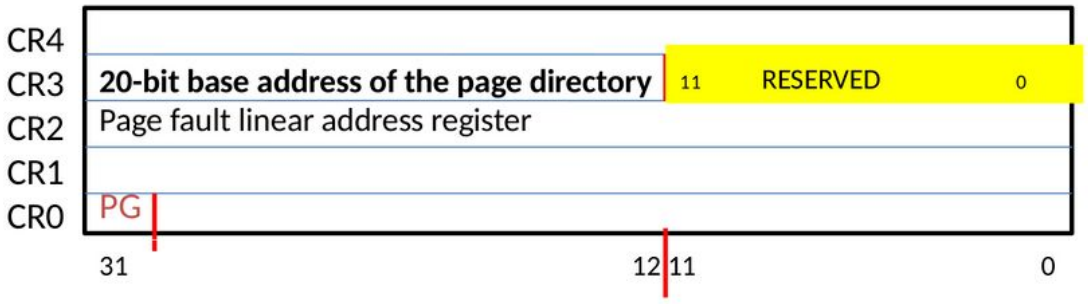
2. Protected mode address calculation



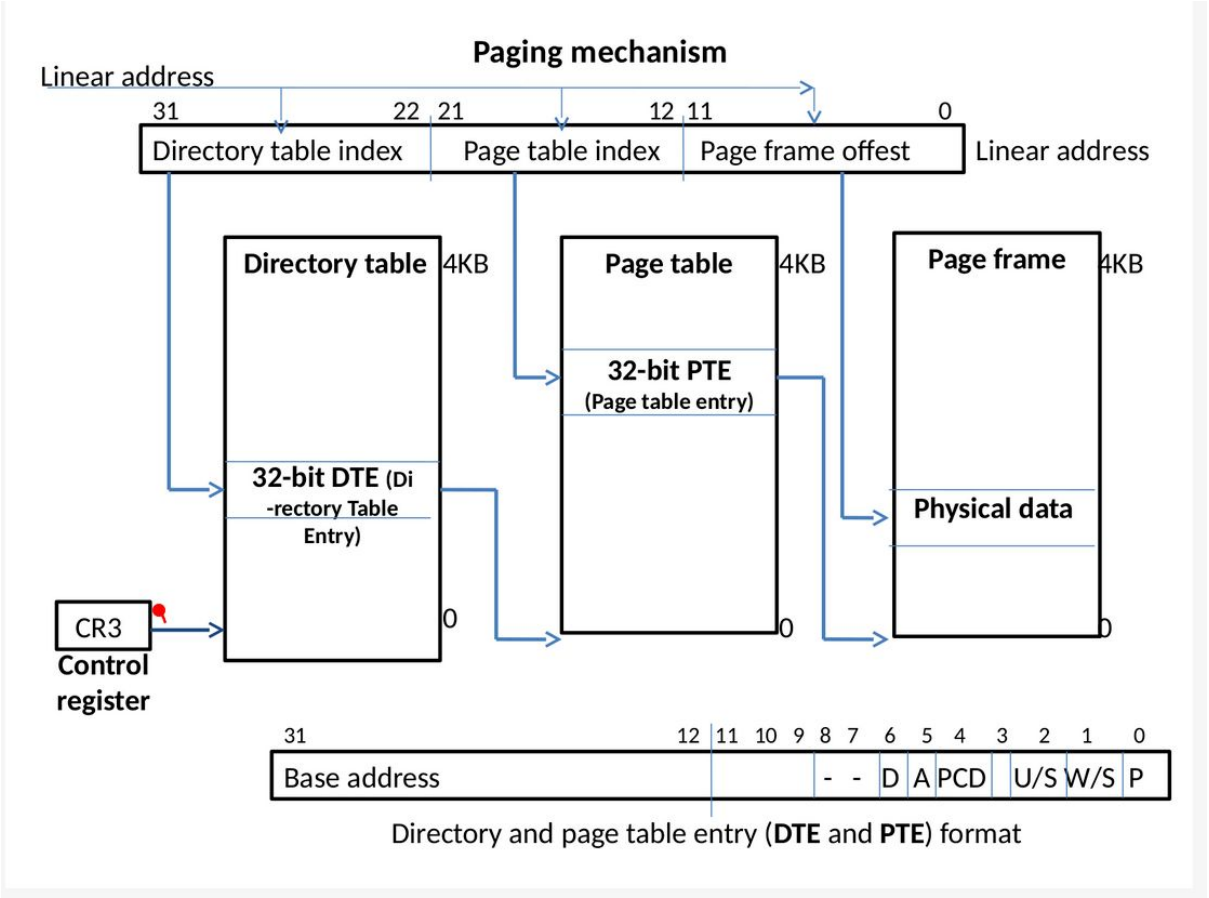
a. Tryb protected - stosowany we wszystkich procesorach 32 bitowych i końcowych 16 bitowych. Też jest to sposób generowania adresu lecz został nieco zmieniony sposób.
b. Rejestr segmentowy został zamieniony na selector mechanism
c. Bity od 4 do 15 wskazywały na położenie deskryptora segmentu w tabeli GDT (Global DT) lub w tablicy LDT (Local DT)
d. TI - bit decydujący którą tablicę deskryptorów wybrać
e. RPL - bity które pozwalają na rozdzielenie informacji np. od wielu użytkowników

3. Memory paging - stronicowanie pamięci

a. Adres liniowy jest to adres generowany przez program
b. Adres fizyczny jest to adres do ramu która jest dołączona do procesora
c. Celem tej metody jest przypisanie adresu liniowemu adresu fizycznego
d. Pamięć fizyczna łącznie z dyskiem i system operacyjny który umożliwia przepisywanie stron pomiędzy pamięcią fizyczną która jest podzielona na ramki. W ramce można zapisać jedną stronę. Przesyłanie stron pomiędzy pamięcią fizyczną (adresy fizyczne), a twardym dyskiem pozwala na zwiększenie pamięci. Wydaje się że pamięć podłączona do mikroprocesora jest duża nawet gdy pamięć fizyczna nie jest duża.
e. Jak to funkcjonuje. Mamy rejestry CR0, CR1 itd. W rejestrze CR0 mamy bit PG - paging który sygnalizuje czy stronicowanie jest włączone czy nie. W rejestrze CR3 jest 20 bitowy adres do tak zwanej page directory jest to strona w której znajduje się 1000 elementów które stanowią adresy. CR3 jest to adres początku tej strony.

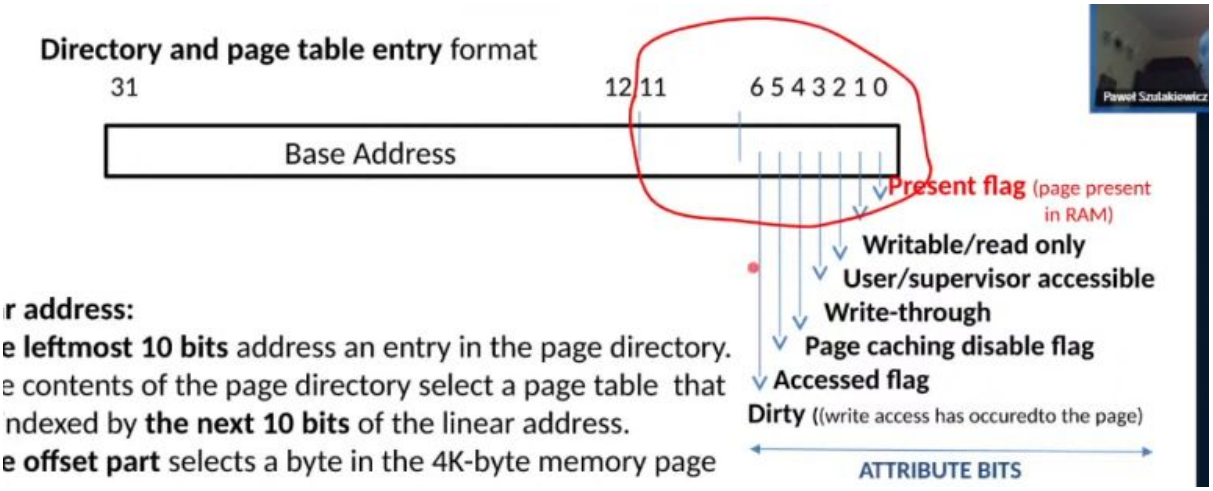


f. Mechanizm stronicowania zamienia adres liniowy na odpowiadający mu adres fizyczny lub generuje przerwanie jeśli wymagana strona nie znajduje się w ramie. Wówczas system operacyjny przesyła żądaną stronę z dysku do pamięci. W procesorach 80486 te strony są 4 Kbytes. W przypadku stronicowania nie ma fragmentacji gdyż ramki w pamięci ram mają po 4 Kbytes i strony zajmują tyle samo co pozwala na pełne zapełnienie.
g. System używa dwie tabele directory table oraz page tables.
h. Directory table - jest jedna taka tablica. Zawiera ona 1000 elementów (32 bitowych adresów) każdy wskazuje na jedną page tables. Każda z page tables jest stroną. Liczba page tables zależy od wielkości pamięci ram. Page tables definiuję ramkę w pamięci fizycznej w której znajduje się strona też o pojemności 4 Kbytes.



i. W CR3 mamy adres wskazujący na Directory table. 10 bitów z adresu liniowego wskazuje na 1 element w Directory table. Następnie ten adres wskazuje na początek jednej z Page table. Następne 10 bitów z adresu liniowego wskazuje na element w Page table. On wskazuje na Page frame. 12 bitów z adresu liniowego wskazuje na element w page frame.

j. Z uwagi na to że każda strona ma 4Kbytes. To aby znaleźć odpowiedni element w directory table lub page table nie potrzebujemy 32 bitów wystarczy 10 najbardziej znaczących bitów.



k. 12 najmniej znaczących bitów directory table oraz page table oznaczają atrybuty danej ramki. Atrybuty takie jak czy ramka jest w ramie, czy jest tylko do odczytu czy też zapisu, czy jest dostępna dla użytkownika czy tylko dla

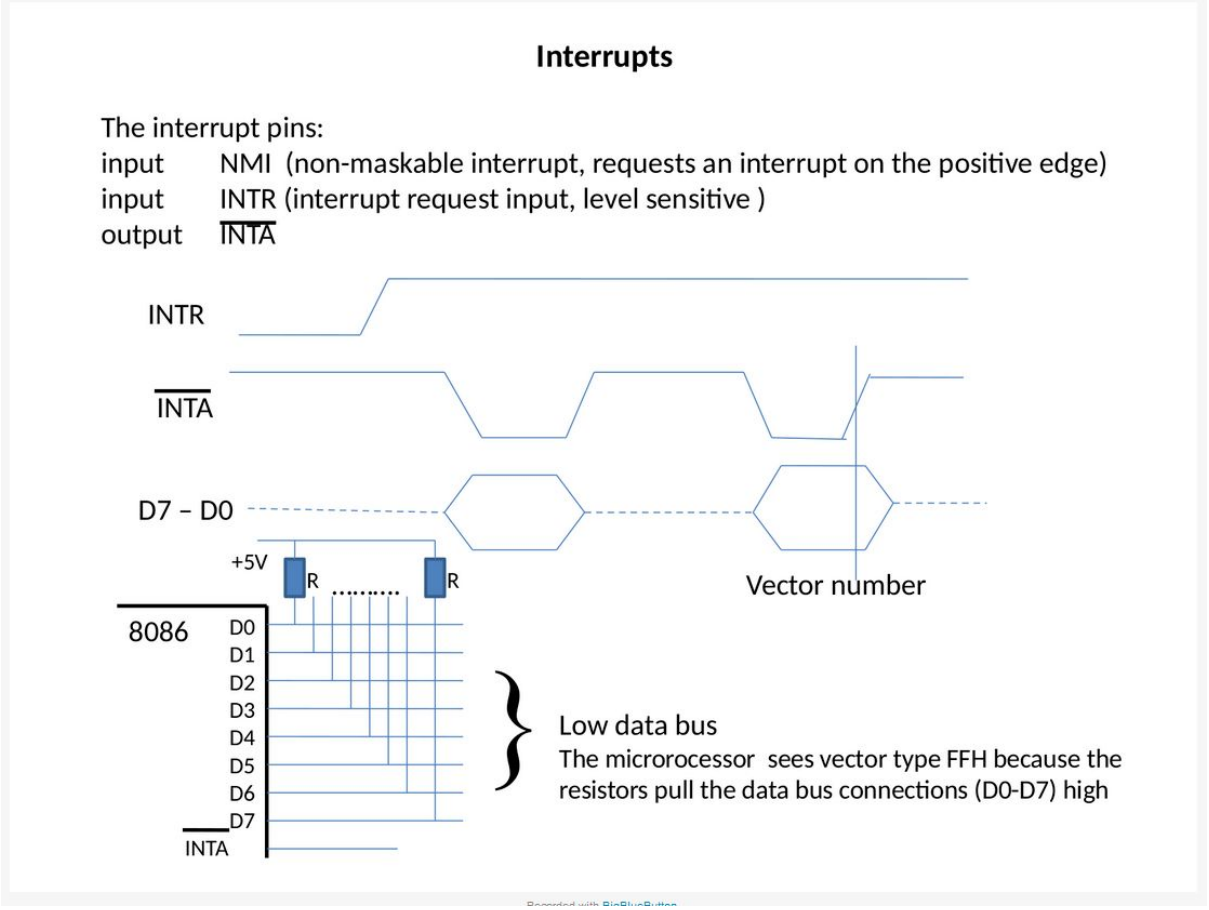
systemu, czy zostało w niej w ogóle coś zapisane, czy ktoś coś z niej odczytywał.

l. W nowszych procesorach Pentium 4 czy core 2 nie ma już segmentacji, może zostać użyte stronicowanie.

4. Przerwania

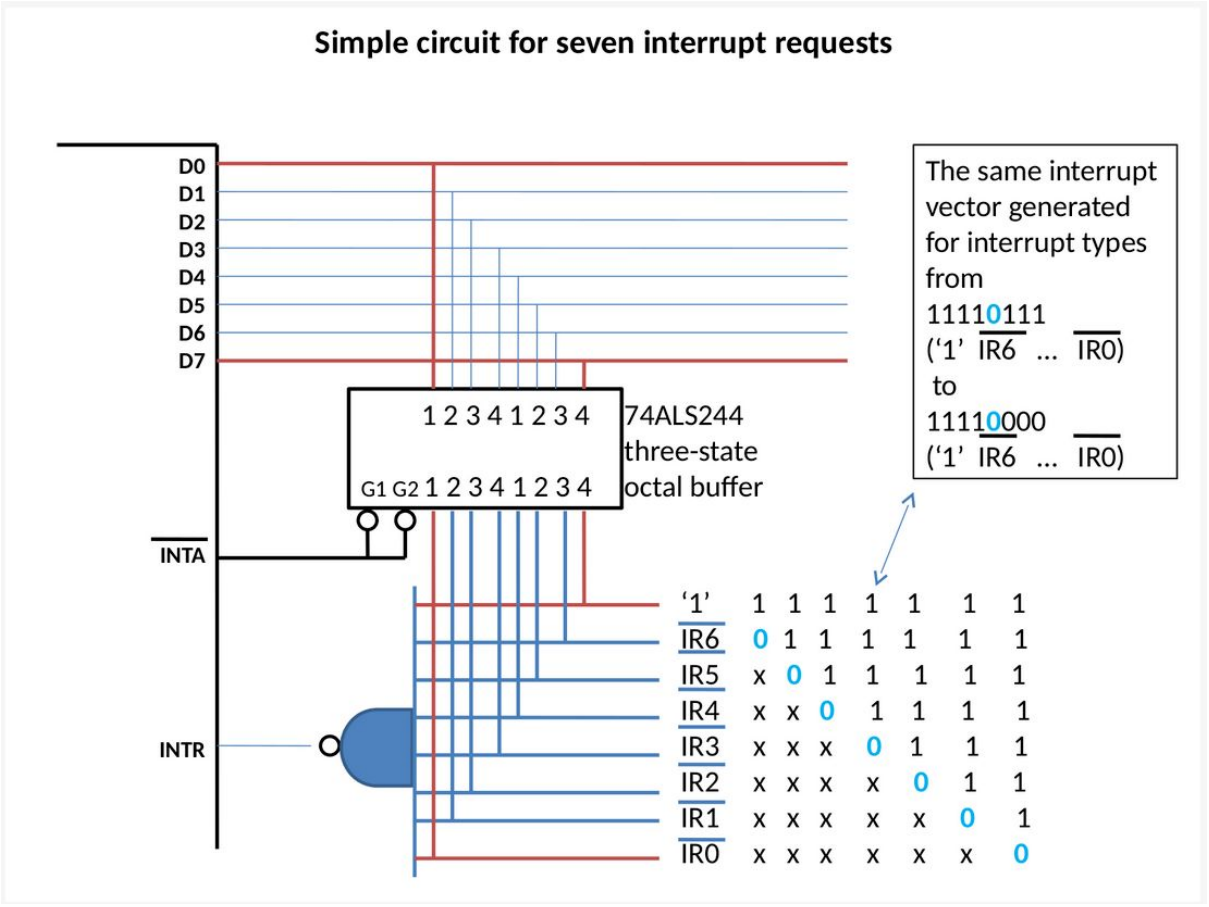
a. W trybie Real (pierwsze procesory intela)
   i. Pojawia się żądanie przerwania
   ii. Mikroprocesor otrzymuje typ przerwania
   iii. Jeśli przerwanie jest zaakceptowane to flaga przerwania zostaje wyzerowana. Następnie zostaje przesłany na stos rejestr statusowy oraz 2 innych rejestrów (CS i IP), które mówią gdzie w programie jest przerwanie
   iv. Procesor otrzymał typ przerwania co pozwala na wskazanie nowego wektora przerwania. Wektórów przerwania jest 256 i one znajduja się na początku pamięci.

b. W trybie Protected
   i. Różnica jest tak że typ przerwania wskazuje na miejsce w tabeli deskryptorów przerwań



c. Żądanie przerwania oznacza 1 na INTR. Gdy pojawi się tak 1 to procesor odpowiada dwukrotnym sygnałem INTA i podczas drugiego sygnału INTA musi się pojawić typ przerwania (8 bitowa liczba która pozwala trafić do miejsca gdzie znajduje się wektor przerwania)

d. Jednym ze sposobów wysłania do mikroprocesora informacji o typie przerwania jest wysłanie na szynę danych d0-d7 wartości 255 czyli samych 1.



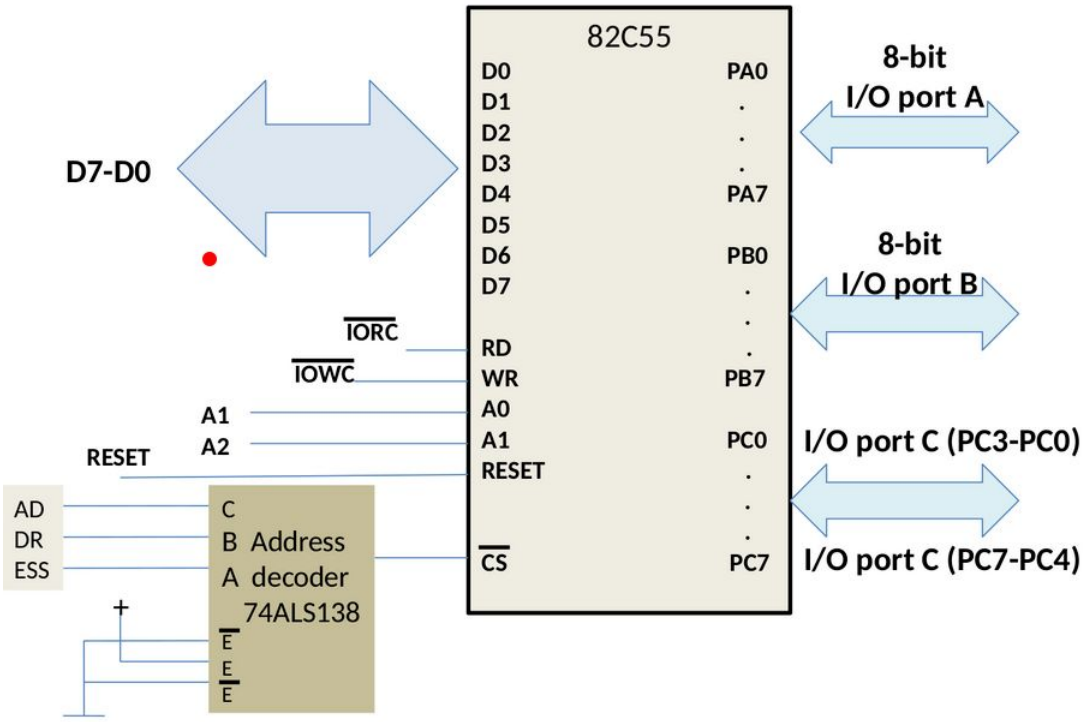
e. Gdy mamy więcej wejść przerwań to możemy zastosować bufor trójstanowy. Łączymy żądania przerwań z szyną danych.
f. Mamy 7 możliwych źródeł przerwań od IR0-IR7. Pierwsza jest '1' dlatego aby używać przerwań od 127 gdyz poniżej tej liczby mogą być przerwania zarezerwowane.
g. '0' na IR6-ir0 oznacza przerwanie IR6 ma najwyższy priorytet
h. Jeśli chcemy mieć więcej niż 6 przerwań to trzeba zastosować specjalne układy zwane sterownikami przerwań.
i. Krok po kroku jak działa przerwanie:
    i. Interfejs wysyła żądanie przerwania do mikroprocesora
    ii. Pojawia się INTA
    iii. Typ przerwania zostaje wysłany do procesora
    iv. Bieżąca wartość PSW, CS, IP zostaje wysłana na stos w celu późniejszego powrotu do programu.
    v. Flaga zezwalająca na przerwanie IF zostaje wyzerowana.
    vi. Obliczanie miejsca w pamięci gdzie znajdują się nowe wartości IP oraz CS
    vii. Następnie jest skok do obsługi programu przerwania
    viii. Wykonanie przerwania
    ix. Zakończenie przerwania rozkazem IRET
    x. Powrót do programu głównego.

5. I/O interface

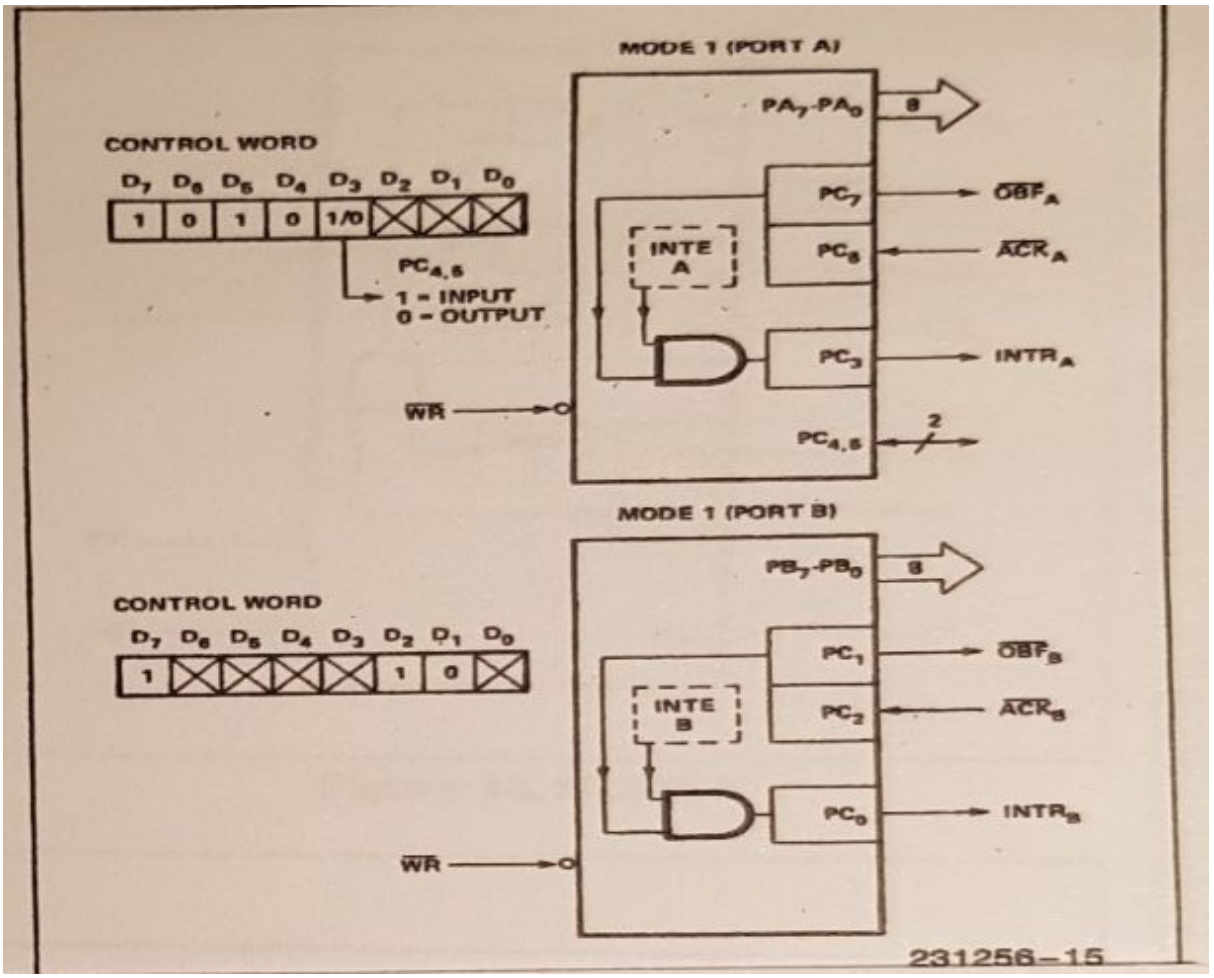
a. W procesorze mamy przestrzeń adresową pamięci oraz przestrzeń adresową urządzeń wejścia wyjścia. Adresy urządzeń wejścia wyjścia to adresy 16 bitowe. Mamy 2 rozkazy które to obsługują IN oraz OUT.
b. Np. IN AX, DX - oznacza to że z jakiegoś portu który jest w rejestrze DX (mamy jakiś zewnętrzny rejestr) i wartość zostaje przepisana do rejestru AX.
c. OUT DX, EAX - oznacza że EAX zostaje zapisane do portu którego adres znajduje się w DX

I/O interface

The **82C55** programmable peripheral interface – low-cost , very popular interfacing device

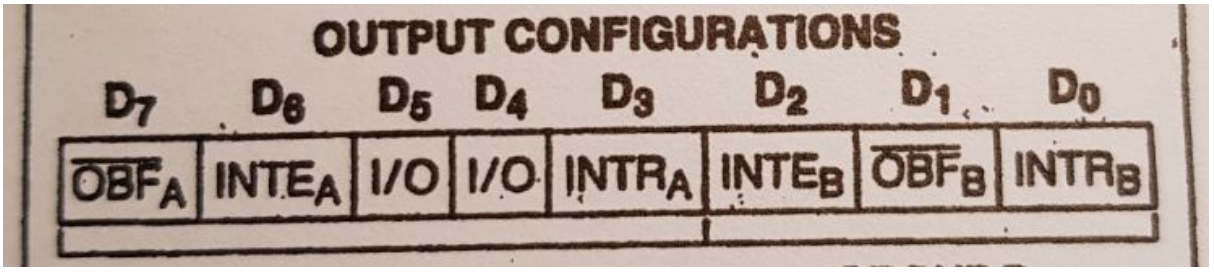


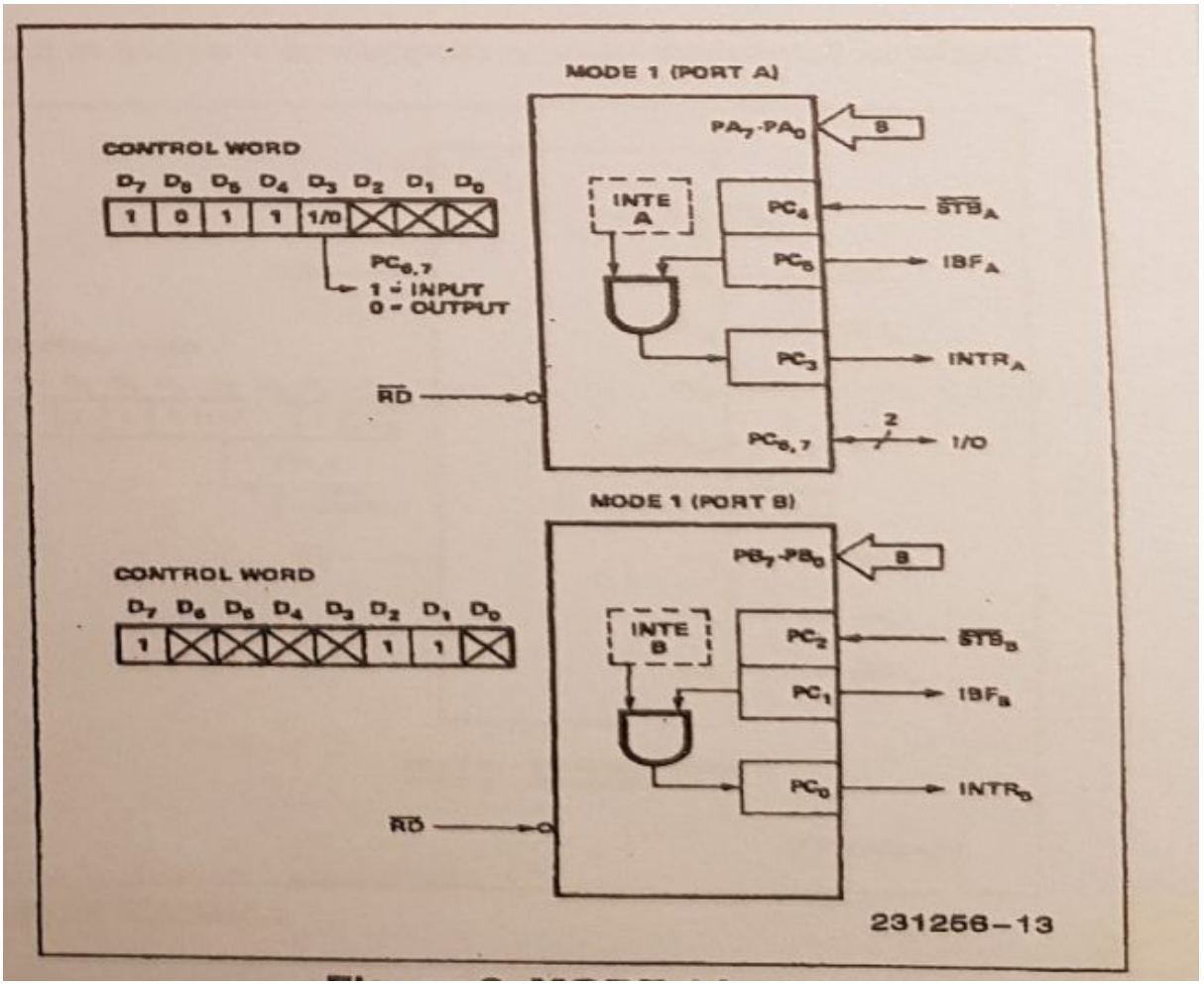
d. 82C55 - jedno z wielu programowalnych interfejsów peryferyjnych. Mamy szynę danych D0-D7 która idzie do procesora oraz mamy 3 porty A, B, C. Wewnątrz są 4 rejestry: rejestr portu A, rejestr portu B, rejestr portu C oraz rejestr sterujący (Który pozwala zaprogramować to urządzenie np. który port pracuje jako wejście a który jako wyjście).
e. CS (Chip select)- jeśli pojawia się na nim 0 oznacza że procesor wybrał to urządzenie.
f. RD, WR - oznacza odczyt lub zapis
g. A0, A1 - oznacza wybór odpowiedniego rejestru



h. 8255 tryb 1 praca jako output - Na przykładzie drukarki. Procesor wysyła do układu przez szynę danych bity(informacja). Przy pomocy sygnału WR zostaje to zapisane np do rejestru A. Następnie mamy sygnał OBF(output buffer full). Jeśli zostało coś zapisane do tego rejestru to ten sygnał OBF zostanie zmieniony na 0 wtedy drukarka wie że ma wydrukować to co jest w rejestrze A. Jak drukarka wykorzysta to co było w buforze wysyła sygnał na ACK i wtedy OBF = 1 oraz może wysłać sygnał INTR do procesora że wykorzystała to co było w buforze i że następny bajt może zostać wpisany do rejestru.

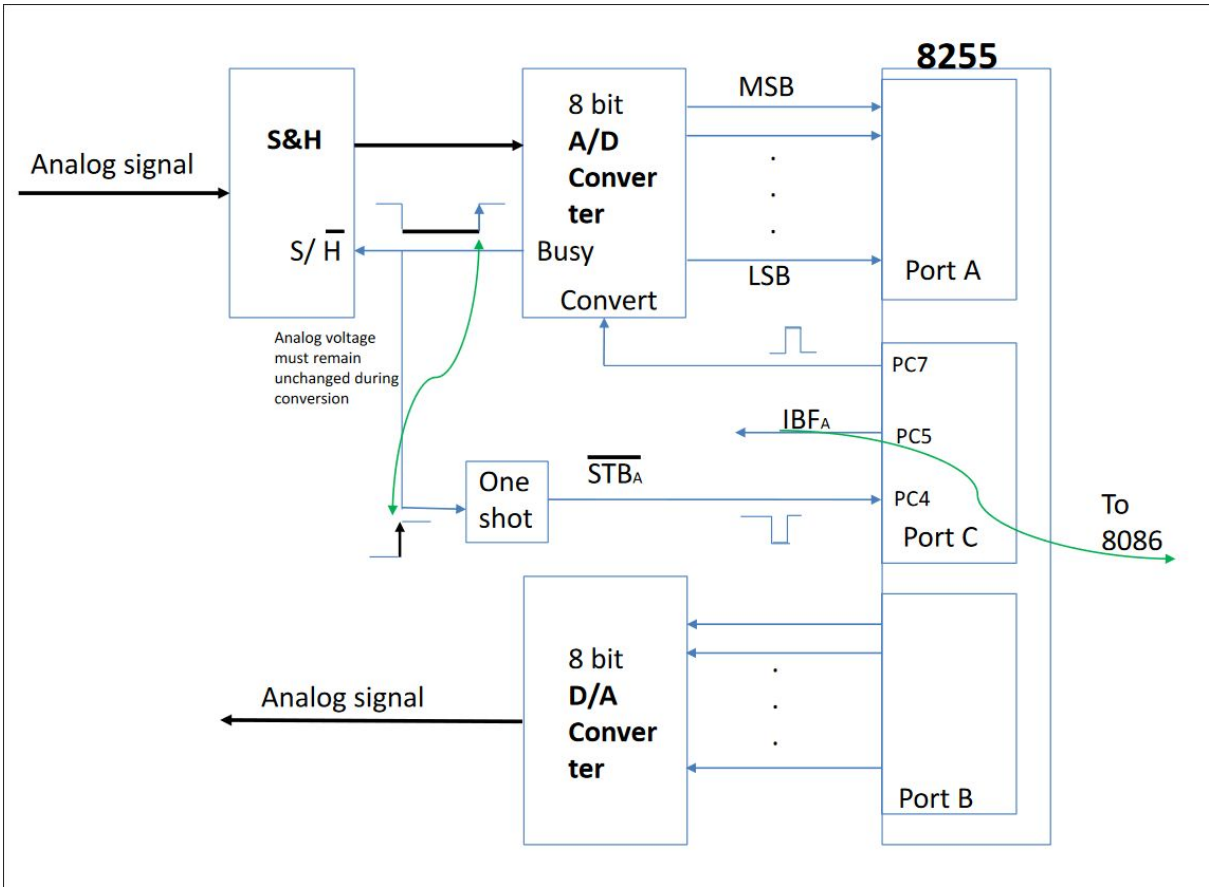
i. Istnieje jeszcze jeden sposób aby procesor mógł wiedzieć że drukarka skończyła pracę. Może on odczytać to co jest w rejestrze portu C i tam na D7 mamy informacje o OBF z portu A. Mamy też wiele innych informacji





j. W trybie input mamy np klawiaturę i idzie sygnał do mikroprocesora. Aby zapisać bity do rejestru portu A wysyła sygnał STB. Gdy już zapisze do rejestru pojawia się sygnał IBF (Input buffer full). Następnie procesor może wykorzystać INTR lub odczytać rejestr portu C i przy pomocy sygnału RD odczytuje zawartość bufora rejestru A. Następnie gdy już odczyta to IBF zmienia stan idzie do klawiatury i można wpisywać kolejne bity.

6. Przykład który wykorzystuje procesor 8086, układ 8255 oraz dwa przetworniki A/D , D/A



a. Schemat połączenia
b. Port B działa jako wyjście w trybie 0
c. Port A działa jako wejście w trybie 1
d. Wyjścia portu C wspomagają pracę portu A w trybie 1
e. Układ S&H(Sample and hold) - układ który próbkuje sygnał wchodzący. Sygnał narasta gdy s/H jest równe 1, gdy jest równe 0 to napięcie jest stałe i przetwornik A/D analizuje sygnał
f. Procesor wysyła do A/D sygnał i wchodzi on na wejście Convert wtedy przetwornik rozpoczyna pracę i na wyjściu Busy jest 0 czyli S&H jest wstrzymane. Jak przetwornik skończy Busy = 1 i S&H pracuje oraz na przerzutniku monostabilnym OneShot jest wygenerowane zero które idzie na PC4 i powoduje to zapisanie liczby z przetwornika A/D do rejestru A. Kolejno na PC5 pojawił się sygnał IBF który informuje procesor o zakończeniu przetwarzania i że może odczytać rejestr A.
g. Następnie wyniki tych przetwarzań są wysyłana na port B i zamieniane na sygnał analogowy poprzez przetwornik D/A

```
Lets assume the following addresses:
FFF8        Port A
FFF9        Port B
FFFA        Port C
FFFB        Control register

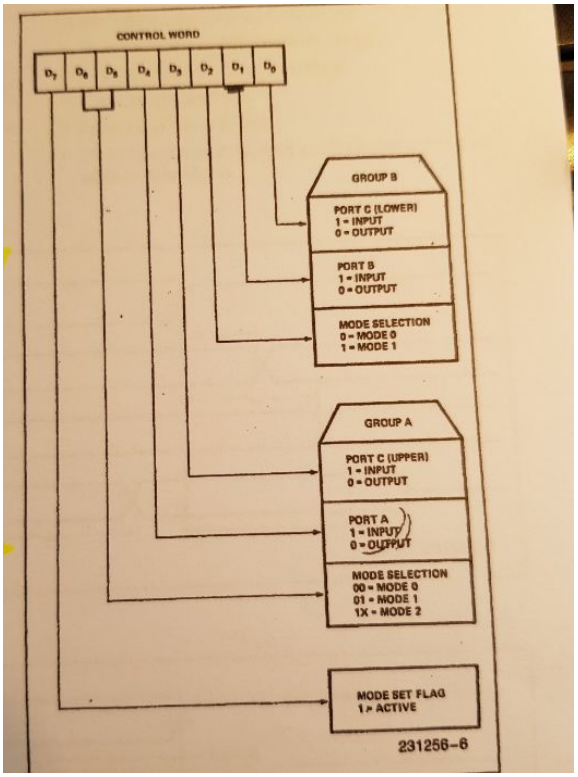
;  Port A in mode 1 - input.  Port B in mode 0 – output.  PC 7 - output
MOV         DX, 0FFFBH
MOV         AL, 10110000B; command byte =   1, 2-bit (PA and PC7-4 ) mode, 1-bit(PA) in/out, 1-bit (PC4-7) in/out,
                          ;2-bit (PB and PC3-0) mode, 1-bit (PB) in/out, 1-bit (PC3-0)in(1)/out(0)
OUT         DX, AL

; Output a pulse to the convert pin of the A/D converter
MOV         DX, 0FFFBH
AOV         AL, 00001111B;Command byte, bit 7th of port C set
OUT         DX, AL        ; PC7 is set
MOV         AL, 00001110B; bit 7th of port C cleared
OUT         DX, AL        ; PC7 is cleared

;lWait until conversion is finished and write the result to AL
            MOV         DX, 0FFFAH ;
AGAIN:      IN          AL, DX
            TEST        AL, 00100000B  ; test PC5 ,  OPRD1^OPRD2, only flags are affected
            JZ          AGAIN
            MOV         DX, 0FFF8H
            IN          AL, DX

;Output a byte from AL to D/A converter
MOV         DX, 0FFF9H
OUT         DX, AL
```

h. Kod realizujący powyższy schemat
i. FFF8, FFF9 - adresy rejestrów portów A, b itd
j. MOV DX, 0FFFBH - wpisanie do dx adresu rejestru sterującego
k. MOV AL, 10110000B - wpisanie do dolnej części rejestru A słowo 10110000B bierze się ono z tego schematu. Najbardziej znaczący bit musi być 1. Mówi że trzeba mu wierzyć że to ustawia port A w trybie 1 i jako wejście, będzie wspomagany przez bity portu C, a port B będzie działał jako wyjście w trybie 0.



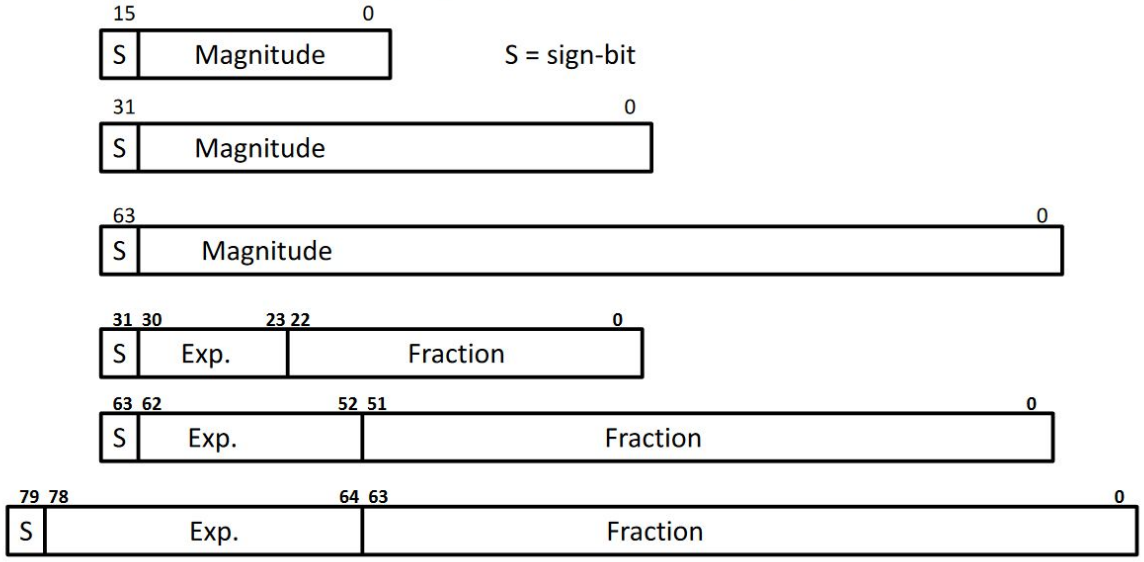
l. OUT DX, AL - oznacza wysłanie tego co jest w AL pod adres znajdujący się w DX.
m. Następny krok to generowanie sygnału który uruchamia przetwornik A/D
n. Kolejno sprawdzamy czy przetwornik zakończył pracę i na PC5 pojawił się sygnał
o. Następnie odczytujemy co znajduje sie na porcie A
p. I ostatni krok to wysłanie tego co było na porcie A na port B

7. Arithmetic coprocessor

THE ARITHMETIC COPROCESSOR

The family of coprocessors (80X87) is able to multiply, divide, add, subtract, find the square root, and calculate the partial tangent, partial arctangent, and logarithms.

Data types include: 16-, 32-, and 64-bit signed integers; 18-digit BCD data (80-bit number, sign-byte + 18 4-bit digits); 32-, 64-, and 80-bit floating-point numbers.



a. Jest to układ przetwarzający liczby
b. Mogą to być 16, 32 i 64 bitowe liczby całkowite lub 32, 64 oraz wewnątrz coprocesora 80 bitowa zmiennoprzecinkowa.
c. Trochę o tych liczbach:

Word integer, 2 bytes, 16 bits, range: -32768 to 32767
Short integer, 4 bytes, 32 bits, range: $-2x10^9$ to $2x\ 10^9$
Long integer, 8 bytes, 64 bits, range: $-9x10^{18}$ to $9x\ 10^{18}$

d. 17 liczb w kodzie bcd. Jest błąd nie 64 bits tylko 80

Packed BCD ,10 bytes, 64 bits, 18 digits D17…D0
[Sign bit,0…0, D17(4 bits),D16(4bits), D15,D14, … ,D1(4 bits),D0(4 bits)]

e. Liczby zmiennoprzecinkowe

Short **real**, 4 bytes, 32 bits, $\pm 10^{-38}$ to $\pm 10^{38}$
[Sign bit, E7 … E0, F1 … F23], where E7 … E0 = bits of exponent, F1…F23 =bits of fraction or mantissa, bit F0=1 is not present.
Long **real**, 8 bytes, 64 bits, range $\pm 10^{-308}$ to $\pm 10^{308}$
[S, E10 … E0, F1 … F52] , bit F0=1 is not present
Temporary **real**, 10 bytes, 80 bits, range $\pm 10^{-4932}$ to $\pm 10^{4932}$
[S, E14 … E0, F0 … F63], where bit F0=1 is present.

f. W tym coprosesorze znajdują się tylko liczby 80bitowe
g. F0 - występuje tylko w liczbie 80 bitowej
h. Sposób wyliczania liczb bardzo podobny jak na AISD różnica jest taka że od wykładnika odejmujemy BIAS który zależy od typu liczby. To BIAS jest po to aby wykładnik był zawsze dodatni

Real: $(-1)^S (2^{E-BIAS}) (F0\ F1\ \ldots) = (-1)^S \times (2^{E-BIAS}) \times 1.F$, where 1=F0 is implied and is not physically stored, mantissa is in the form 1.F

where BIAS = 127 = 7FH for Short real
BIAS = 1023 = 3FFH for Long real
BIAS = 16383= 3FFFH for Temp. real

i. Przykłady zamiany

Example 1. Converting to floating point form

1. Convert the decimal number into binary
   100.25 D = 1100100.01 B
2. Normalize the binary number. **(** Normalized binary number = (**1.** …1) x $\mathbf{2^{exponent}}$ **)**
   1100100.01 B = **1**.10010001 x 2^6
3. E-127=6 → biased exponent E=127+6
   E = 6+127 D = 110 + 0111 1111 B = 10000101
4. 32-bit floating point number
   01000010 11001000 10000000 00000000

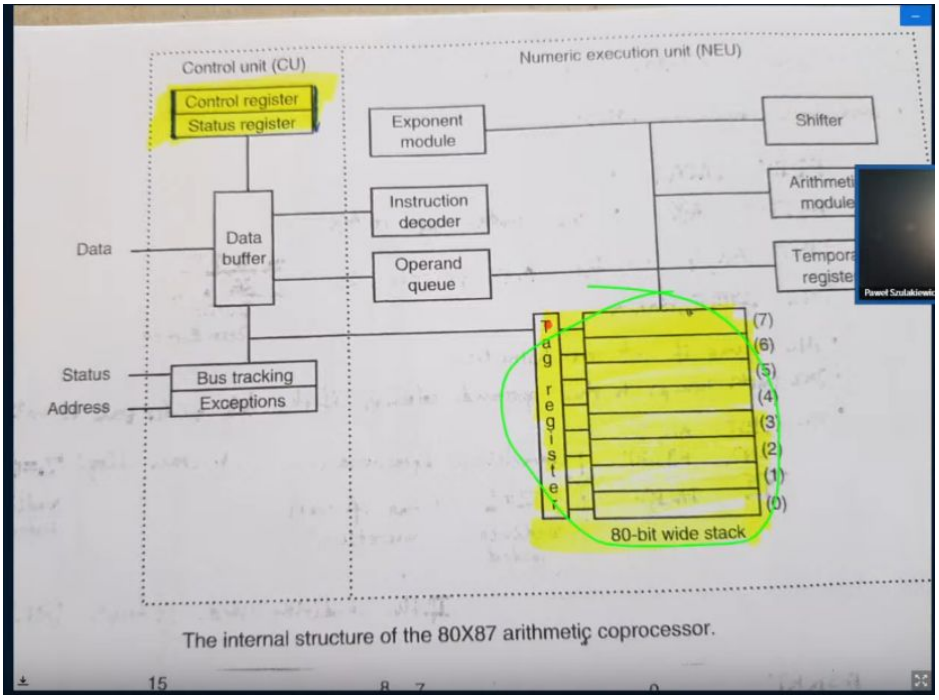
Eample 2. Converting from floating point form

1.Separate the sign, biased exponent and fraction
11000001 11001001 00000000 00000000
2 Convert the biased exponent into a true one
10000011 – 01111111 = 100B = 4D
3. Write the number as a normalized binary number
1.1001001 x 2^4 where **1=F0**
4. Convert into unnormalized binary number
11001.001
5. Convert into decimal
- 25.125

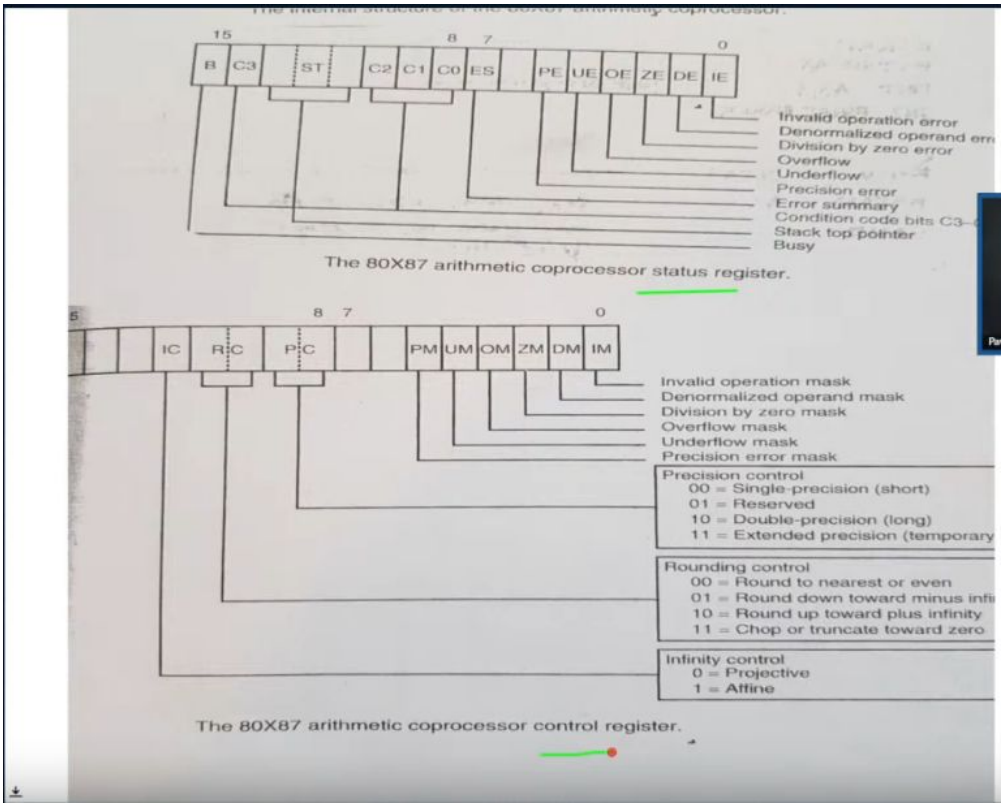
j. Normalizacja liczby binarnej oznacza przesunięcie przecinka tak aby otrzymać 1.coś * 2^x gdzie x to liczba o ile przesunęliśmy przecinek
k. mantysę do wyniku przepisujemy bez najbardziej znaczącej 1
l. Czerwony to bit znaku, niebieski to wykładnik, czarny mantysa.

m. Odwrotna konwersja to od wykładnika odejmujemy BIAS. Do mantysy dopisujemy najbardziej znaczącą 1. Przesuwamy przecinek o tyle pozycji ile nam wyszło z wykładnika. Zamieniamy na dziesiętny i mamy liczbę.

8. Arithmetic coprocessor - struktura wewnętrzna
   a. Coprocesor składa się z: rejestr statusowy, rejestr sterujący, oraz 8 rejestrów bufora każdy po 80 bitów, rozkazów.
   b. Schemat wewnętrzny



   c. Opis rejestrów



d. Rejestr statusowy informuje nas o tym czy nie wykonano błędnej operacji np. dzielenie przez zero, ma on też bity mówice gdzie znajduje się stos.
e. Rejestr sterujący ustawia parametry takie jak: precyzja, zaokrąglenia
f. Przykład wyliczania pole koła

```
; Memory location RADIUS contains radius
; Memory location AREA contains the result
; Both memory locations contain short floating point numbers
AREA_CIRCLE PROC  NEAR          ; Beginning of a procedure. NEAR proc. must
                                ; appear in the current code segment.
    FLD         RADIUS          ; PUSH radius. (Decrement ST, convert the contents of RADIUS to
                                ;temporary real and put the result in ST )
                                    RADIUS   (top of stack ST(0) )
                                    (ST(1))
    FMUL        ST, ST(0)       ; Square radius
                                    RADIUS²   ST(0)
                                              ST(1)
    FLDPI                       ; Push π.  (Decrement ST, π→(ST)  )
                                    π
                                    RADIUS²
                                    
    FMUL        ST, ST(1)       ; Multiply π x R²
                                    π x RADIUS²
                                    RADIUS²
                                    
    FST         AREA            ;Store AREA. (Convert (ST) into the destination's real format and store
                                ; the result in memory location AREA
                                    π x RADIUS²
                                    RADIUS²
                                    
    FCOMP                       ; Clear stack. (The instruction pops the stack twice, which removes
                                ; both numbers from the stack to clear it for use by another procedure.)
                                              ST(0)
                                              ST(1)
                                              ST(2)
    RET                         ; Return from procedure
AREA_CIRCLE     ENDP            ; ENDP indicates  the end of  a procedure
```

g. Po lewej są rozkazy po prawej jest stos
h. FLD RADIUS - wprowadza nam na stos RADIUS
i. FMUL ST, ST(0) - mnoży szczyt stosu z komórką 0 stosu i wynik umieszczamy na szczycie stosu
j. FLDPI - wprowadzenie na stos PI
k. FMUL ST, ST(1) - mnożenie szczytu stosu oraz 1 komórki stosu
l. FST AREA - ładujemy to co na stosie do AREA
m. FCOMP - porównanie nas nie interesuje głównym zadaniem tego rozkazu jest wyczyszczenie stosu

Wykład 7

Procesory ARM Cortex M3 i M4

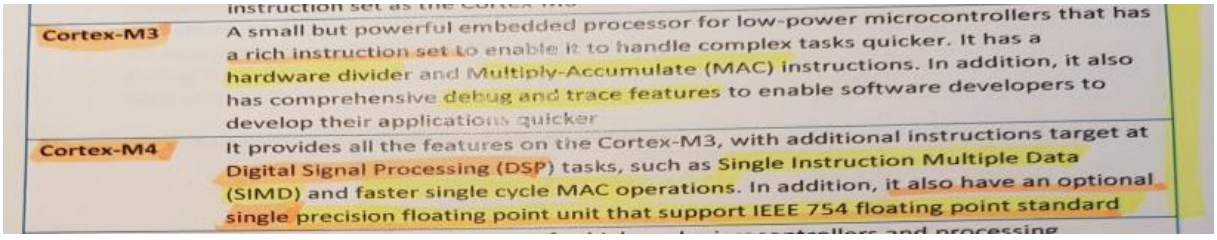
ARM nie tworzy mikrokontrolerów, oni sprzedają licencje na procesor. Odbiorcy sami tworzą procesory i dodają co jest potrzebne do mikrokontrolera który jest potrzebny rozwiązania nazywa się to IP (intelectual property) licensing.

ARM dzielą się na 3 profile:

- A
    - przeznaczony do wysokowydajnych platform otwartych aplikacji (najwyższa moc obliczeniowa, pamięć wirtualna, MMU). Przykładowe produkty: smartfony, tablety, telewizory).
    - Taktowanie na poziomie >1GHz
    - Obsługują MMU (Memory Management Unit) wymagany przez OS (linux, android, MS Windows)
- R
    - jest przeznaczony do zaawansowanych systemów wbudowanych, w których wymagana jest wydajność w czasie rzeczywistym (kontrolery dysków twardych, kontrolery pasma podstawowego dla komunikacji mobilnej, systemy samochodowe).
    - Taktowanie na poziomie 200 MHz - >1GHz
    - Nie ma MMU
    - Mają zazwyczaj MPU (Memory Protection Unit)
    - Pełny OS nie będzie działał, ale można korzystać na nich z RTOS (Real Time Operating System)
- M
    - przeznaczony do głęboko osadzonych systemów z mikrokontrolerem. (Procesor jest ukierunkowany na aplikacje na mniejszą skalę, takie jak mikrokontrolery, w których ważne są takie kryteria, jak niski koszt, niski pobór mocy, energooszczędność i małe opóźnienia przerwań. Konstrukcja procesora jest łatwa w użyciu i może zapewnić deterministyczne zachowanie wymagane w wielu systemach sterowania w czasie rzeczywistym .)
    - Niższe taktowanie chodź są takie które mają ponad 200 MHz
    - Łatwe w użyciu
    - Popularne wśród mikrokontrolerów

The ARM Cortex-M4 processor posiada jako opcje system do wykonywania operacji na liczbach zmiennoprzecinkowych, wykorzystywany w przetwarzaniu sygnału

Różnice między M3 i M4



Cortex-M3	A small but powerful embedded processor for low-power microcontrollers that has a rich instruction set to enable it to handle complex tasks quicker. It has a hardware divider and Multiply-Accumulate (MAC) instructions. In addition, it also has comprehensive debug and trace features to enable software developers to develop their applications quicker
Cortex-M4	It provides all the features on the Cortex-M3, with additional instructions target at Digital Signal Processing (DSP) tasks, such as Single Instruction Multiple Data (SIMD) and faster single cycle MAC operations. In addition, it also have an optional single precision floating point unit that support IEEE 754 floating point standard

SIMD polega na tym że rozkaz 32 bitowy może mieć 2 rozkazy na 16 bitach lub 4 każdy na 1 bajcie

Thumb-2 przyjmuje rozkazy 16 I 32 bitowe

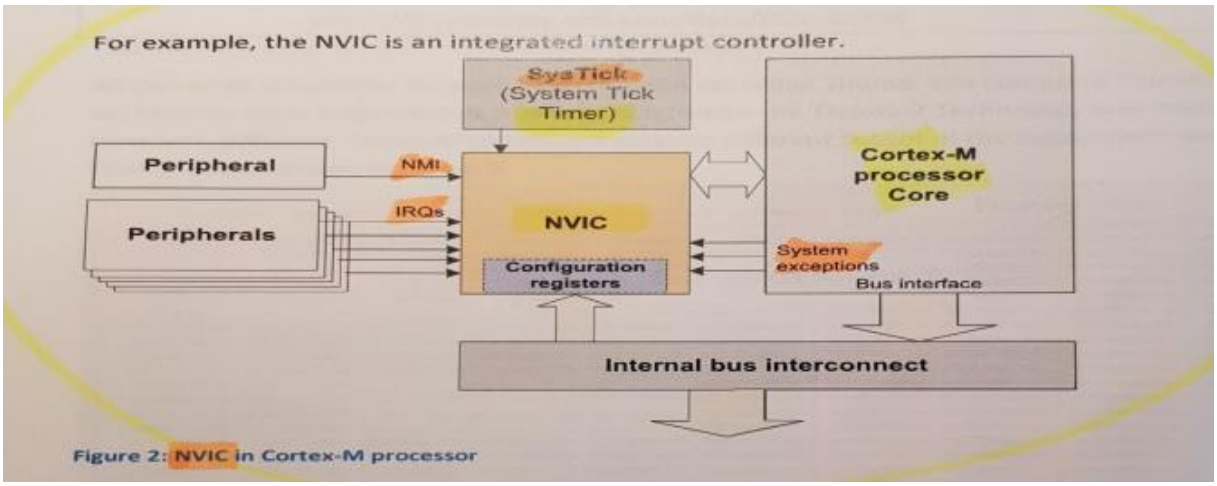
Quite different from legacy ARM processors (e.g. ARM7TDMI, ARM9), the Cortex-M processors have a very different architecture. For instance:

- Only support ARM Thumb® instructions, which have been extended to support both 16-bit and 32-bit instructions in Thumb-2.
- Interrupt handling is managed by a built-in interrupt controller called Nested Vector Interrupt Controller (NVIC), which provides automatic prioritization, masking and nesting of interrupts and system exceptions.
- Interrupt handlers can be written as normal C functions and the vectored interrupt handling mechanism avoided the need to use software to determine which interrupt to service. At the same time, interrupt responses are deterministic and have low latency.
- Vector table changed from branch instructions to starting addresses of interrupts and system exception handlers.
- The register bank and some details of the programmer's model have also been changed.

The changes mean many old assembly code written for classic ARM processors would need modifications, and old projects need to be modified and recompiled when migrated to the Cortex-M world. Detailed information on software porting is documented in an ARM document:

Jest tablica wektorów przerwań.

Mają sterownik przerwań NVIC Nested Vector Interupt Controller



Można sterować i dawać priorytety przerwaniom oraz je zmieniać jak na przykład jakieś przerwanie się zrealizuje.

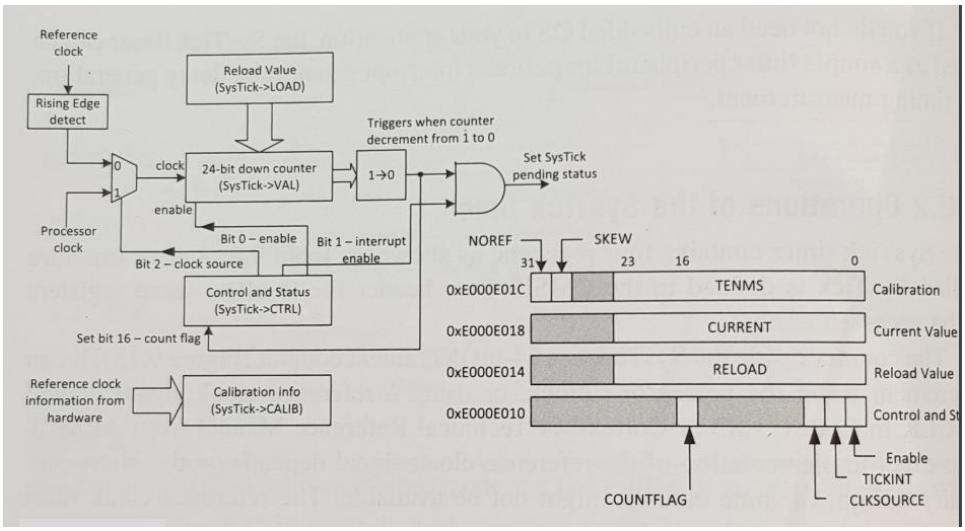
Przerwania z rdzenia procesora są nazywane Exceptions

Natomiast z peryferyjnych to są Interacts

Timer generuje przerwania

Urządzenia peryferyjne mogą przychodzić przerwania NMI (nie maskowalne) i IRQs (maskowalne)

Systick Timer



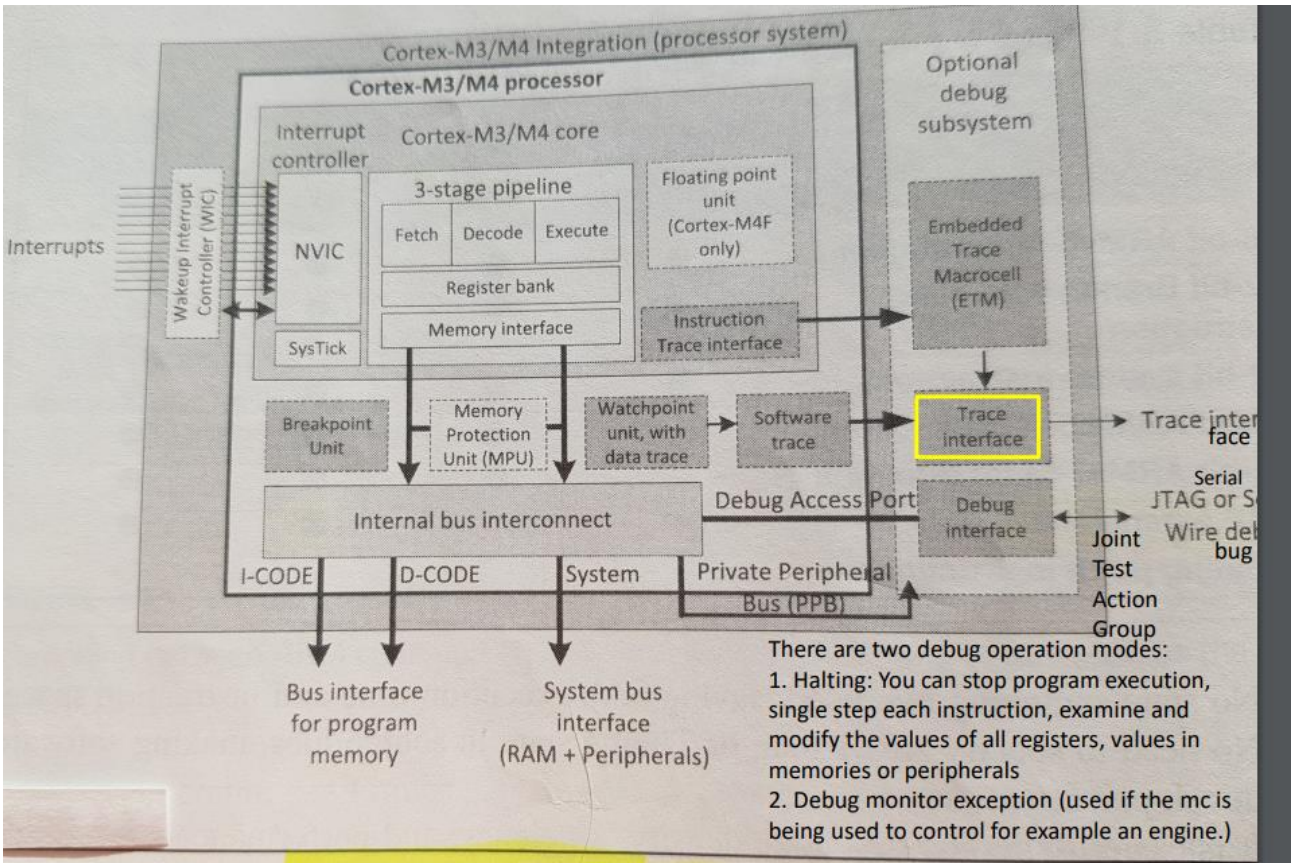
24 bitowy licznik

Bit 0 włącza licznik

Bit 1 włącza przerwanie licznika

Bit 2 decyduje o źródle zegara

Cortex M3/M4 processor



I-code - Kod operacji

D-Code - dane

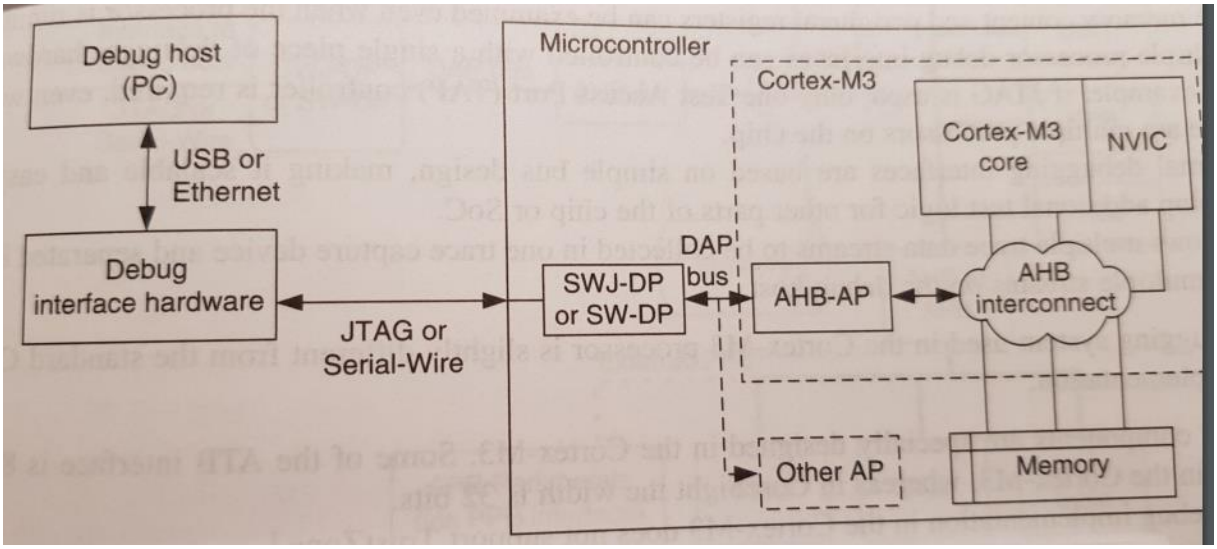
JTAG – Joint Test Action Group

Mamy twa tryby debugowania:

Halting: robimy po jednej instrukcji, możemy przeglądać wszystkie rejestry, wartość w pamięci i urządzenia peryferyjne

Debug monitor exception: używany gdy mikrokontoler jest używany do kontrolowania na przykład silnika.

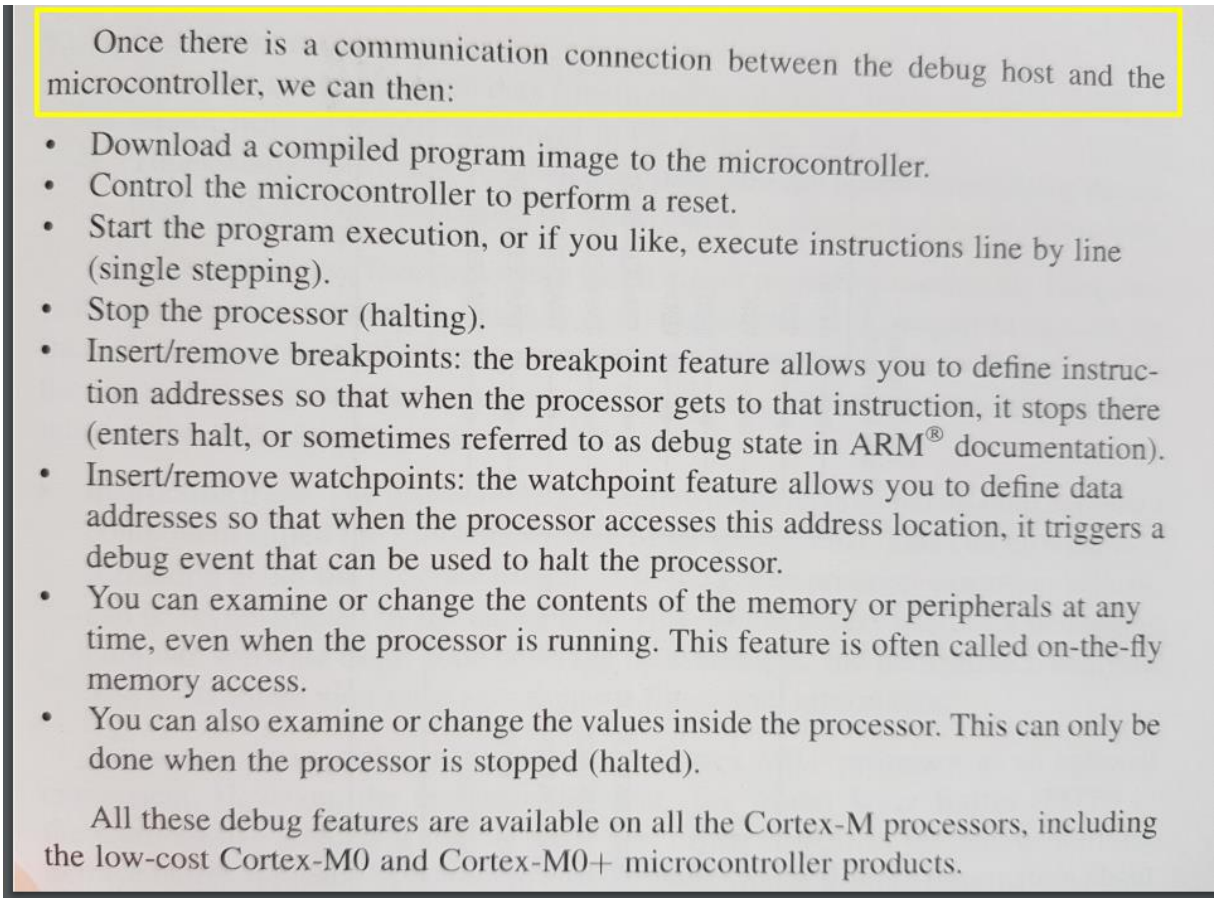
Debug Host połączony z mikrokontolerem



Cortex M4 processor features:

Low-cost debug solution that features:

- Debug access to all memory and registers in the system, including access to memory mapped devices, access to internal core registers when the core is halted, and access to debug control registers even while SYSRESETn is asserted.
- Serial Wire Debug Port (SW-DP) or Serial Wire JTAG Debug Port (SWJ-DP) debug access. (Both JTAG and SWD protocols are widely supported by many development tool vendors.)
- Optional Flash Patch and Breakpoint (FPB) unit for implementing breakpoints and code patches. (It allows to patch the program in a masked ROM with a small programmable memory. This enables enhancements made even when the main code is in masked ROM.)
- Optional Data Watchpoint and Trace (DWT) unit for implementing watchpoints, data tracing, and system profiling. (Can be used to generate watchpoint events to halt the processor when selected data is accessed, or to generate trace information that can be collected by the trace interface without stopping the processor.)
- Optional Instrumentation Trace Macrocell (ITM) for support of printf() style debugging. (This allows full visibility of the program flow during execution.)
- Optional Trace Port Interface Unit (TPIU) for bridging to a Trace Port Analyzer (TPA), including Single Wire Output (SWO) mode.
- Optional Embedded Trace Macrocell (ETM) for instruction trace. (This allows full visibility of the program flow during execution, which is very usefull for debugging complex software problems and code analysis.)



Once there is a communication connection between the debug host and the microcontroller, we can then:

- Download a compiled program image to the microcontroller.
- Control the microcontroller to perform a reset.
- Start the program execution, or if you like, execute instructions line by line (single stepping).
- Stop the processor (halting).
- Insert/remove breakpoints: the breakpoint feature allows you to define instruction addresses so that when the processor gets to that instruction, it stops there (enters halt, or sometimes referred to as debug state in ARM® documentation).
- Insert/remove watchpoints: the watchpoint feature allows you to define data addresses so that when the processor accesses this address location, it triggers a debug event that can be used to halt the processor.
- You can examine or change the contents of the memory or peripherals at any time, even when the processor is running. This feature is often called on-the-fly memory access.
- You can also examine or change the values inside the processor. This can only be done when the processor is stopped (halted).

All these debug features are available on all the Cortex-M processors, including the low-cost Cortex-M0 and Cortex-M0+ microcontroller products.

Co można zrobić z podłączonym debugerem:

- Załadować skompilowany program do mikrokontrolera
- Zresetować mikrokontroler
- Rozpocząć program lub wykonywać program linia po linii
- Zatrzymać proces
- Wstawić/Usunąć breakpoints (gdy program dojdzie do rozkazu to się zatrzyma)
- Wstawić/Usunąć watchpoints (gdy program będzie chciał skorzystać z danego adresu to się zatrzyma)
- Zbadać wartość pamięci i urządzeń peryferyjnych (on-the-fly memory access)
- Zbadać lub zmienić wartość w procesorze, można to robić tylko gdy program jest zatrzymany (halted)

Trace features

Możemy uzyskać wartości PC, adresów, danych i różnych informacji w czasie wykonywania programu bez zatrzymywania. Działa w czasie rzeczywistym z drobnym opóźnieniem

Tryby I stany mikroprocesora

- A low gate count processor core, with low latency interrupt processing that has:
- A subset of the Thumb instruction set.
- Banked Stack Pointer (SP). (Main Stack Pointer – MSP, Process Stack Pointer – PSP)
    - Ma bank Stack Pointerów
    - Wskaźnik stosu MSP
    - Szczyt stosu PSP

- Hardware integer divide instructions, SDIV and UDIV.
- Tryb Handler-tryb wykonywania programu obsługi przerwania
- Tryb Thread- w trybie wątku
- Thumb State – rozkazy ze zbioru rozkazów thumb, program działa normalnie
- Debug states - debuguje
- Wykorzystuje komendy PUSH i POP do stosu
- Automatic processor state saving and restoration for low latency Interrupt Service Routine (ISR) entry and exit.
- Support big-endian byte-invariant or little-endian accesses.
    - Jak mamy liczbę 32 bitową to liczba jest zapisana jako bajt o największym znaczenie jest przy większym rejestrze czy mniejszym

Optional Floating Point Unit (FPU) providing:

- 32-bit liczby zmiennoprzecinkowe pojedynczej precyzji
- Combined Multiply and Accumulate
- Hardware support for conversion, addition, subtraction, multiplication with optional accumulate, division, and square-root.
- Hardware support for denormals and all IEEE rounding modes.
- 32 dedicated 32-bit single precision registers, also addressable as 16 double-word registers.
- Decoupled three stage pipeline.
- Dwa rejestry 32 bitowe można zmienić na jeden 64 bitowy

Nested Vectored Interrupt Controller (NVIC) closely integrated with the processor core to achieve low latency interrupt processing. Features include:

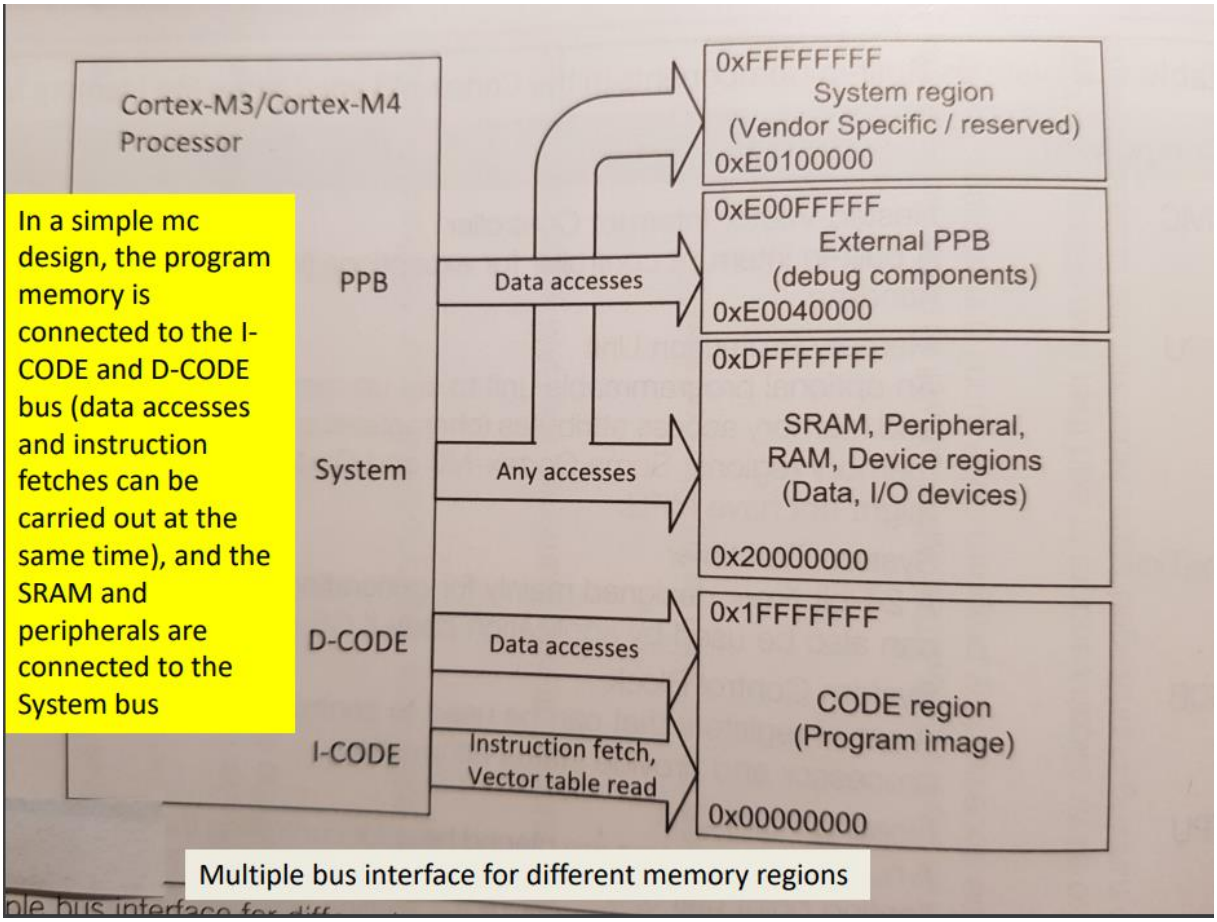
- External interrupts, configurable from 1 to 240.
- Bits of priority, configurable from 3 to 8.
- Dynamiczna zmiana priorytetów
- Priority grouping. This enables selection of preempting interrupt levels and non pre-empting interrupt levels.
- Support for tail-chaining and late arrival of interrupts. This enables back-to-back interrupt processing without the overhead of state saving and restoration between interrupts.
- Stan procesora jest zapamiętywany na początku przerwania i odzyskiwany na zakończeniu przerwania.
- Optional Wake-up Interrupt Controller (WIC), providing ultra-low power sleep mode support.

Bus interfaces:

Trzy systemy szyn: ICode, DCode, and System bus interfaces.

- I-CODE: Bus Interface primarily for program memory: Instruction fetch and vector fetch for address 0x0 to 0x1FFFFFFF. Based on Advanced Microcontroller Bus Architecture (AMBA) 3.0 Advanced High-Performance Bus (AHB) Lite bus protocol.
- D-CODE: Bus Interface primarily for program memory: Data and debugger accesses for address 0x0 to 0x1FFFFFFF. Based on AMBA 3.0 AHB Lite bus protocol. System: Bus Interface primarily for RAM and peripherals: Any accesses from address 0x20000000 to 0xFFFFFFFF (apart from PPB regions). Based on AMBA 3.0 AHB Lite bus protocol.

- Private Peripheral Bus (PPB) based on Advanced Peripheral Bus (APB) interface. External Private Peripheral Bus (PPB): For private debug componentson system level from address 0xE0040000 to 0xE00FFFFF. Based on AMBA 3.0 APB (Advanced Peripheral Bus) protocol.
- Bit-band support that includes atomic bit-band write and read operations.
- Memory access alignment.
- Write buffer for buffering of write data.
- Exclusive access transfers for multiprocessor systems.



Multiple bus interface for different memory regions

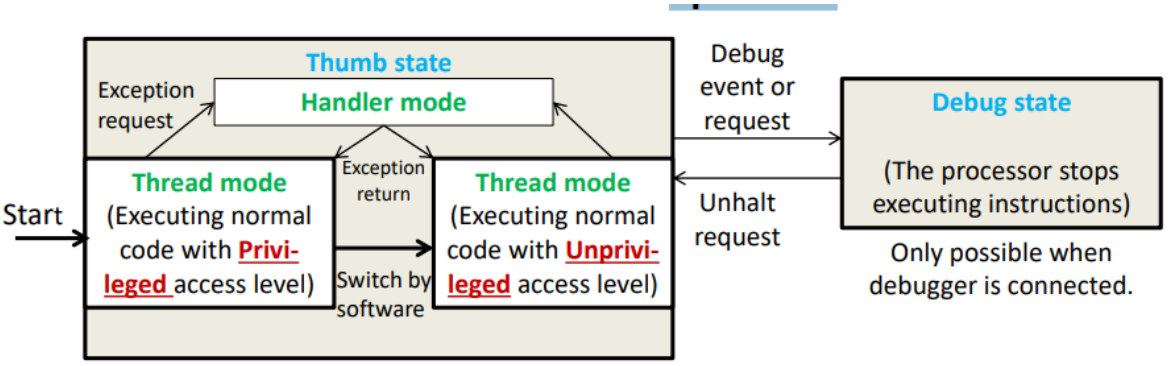
Program image- rejon gdzie mamy program Dane i rozkazy można jednocześnie pobierać

SRAM – statyczna pamięć ram

External PPB – urządzenia wyjściowe do debugowania

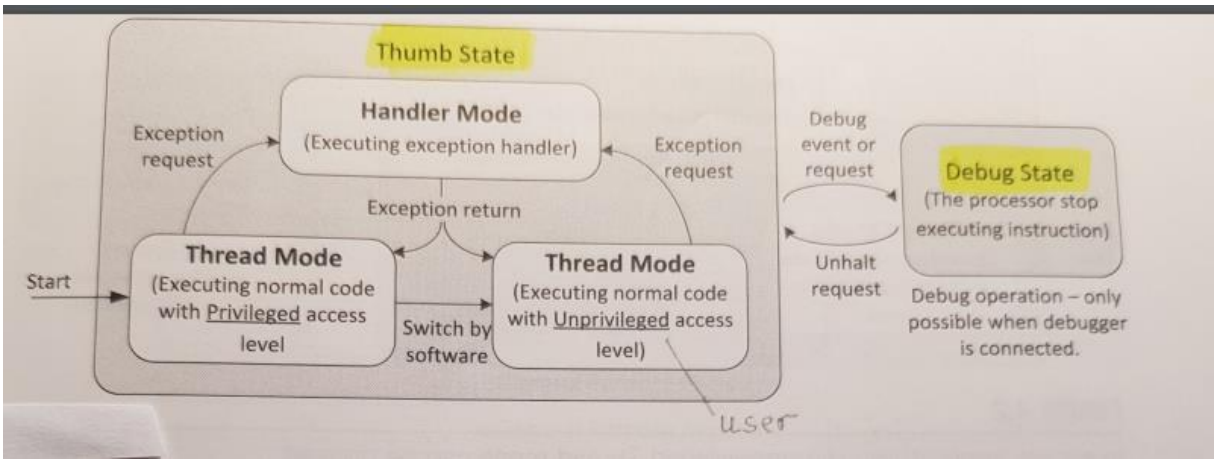
System region – pamięć dla kupujących procesory i tam mogą coś zapisywać

Introduction to the architecture – operation modes and states



The processors have two operation states, two modes and two access levels.

- Jeżeli procesor ma odpalony kod programu jest w stanie Thumb.
- Jeżeli procesor jest zatrzymany (halted) jest w stanie Debug.
- Tryb Handler – kiedy jest przerwanie takie jak Interrupt Service Routine (ISR). W tym trybie procesor ma najwyższy priorytet.
- Tryb Wątkowy (Thread) – kiedy aplikacja działa normalnie procesor może mieć przywilejowany lub nieprzywilejowany poziom dostępu. Jest to kontrolowane przez CONTROL register.
- Przywilejowany poziom dostępu (The privileged access level) ma dostęp do wszystkich zasobów w procesorze.
- Nieprzywilejowany (The unprivileged (User) access level) znaczy że niektóre rejony pamięci mogą być niedostępny I nie które operacje też mogą być niedostępne.
- To switch the processor back from unprivileged to privileged Thread mode the exception mechanism has to be used.

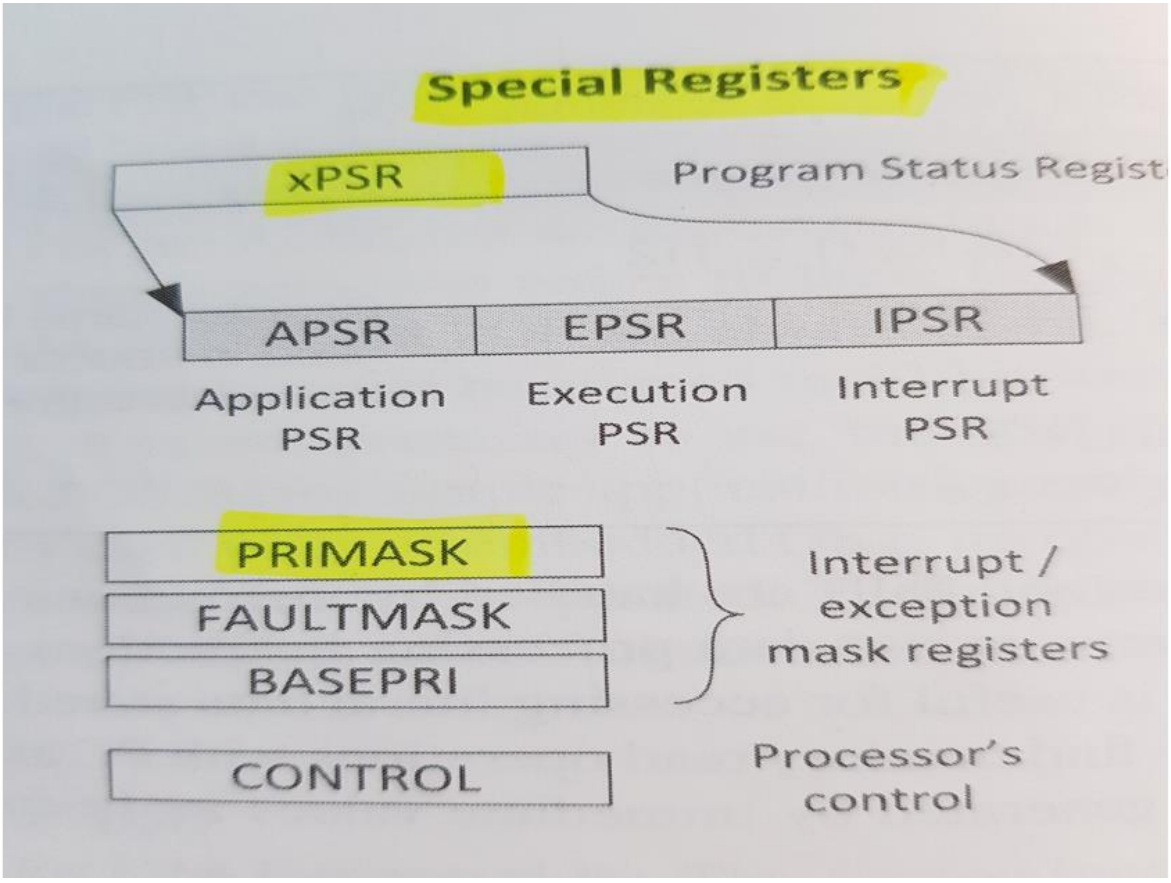


- Thumb state: If the processor is running program code (Thumb instructions), it is in the Thumb state. Unlike classic ARM® processors like ARM7TDMI, there is no ARM state because the Cortex-M processors do not support the ARM instruction set.

Operation modes

- Handler mode: When executing an exception handler such as an Interrupt Service Routine (ISR). When in handler mode, the processor always has privileged access level.
- Thread mode: When executing normal application code, the processor can be either in privileged access level or unprivileged access level. This is controlled by a special register called "CONTROL." We will cover this more in section 4.2.3.

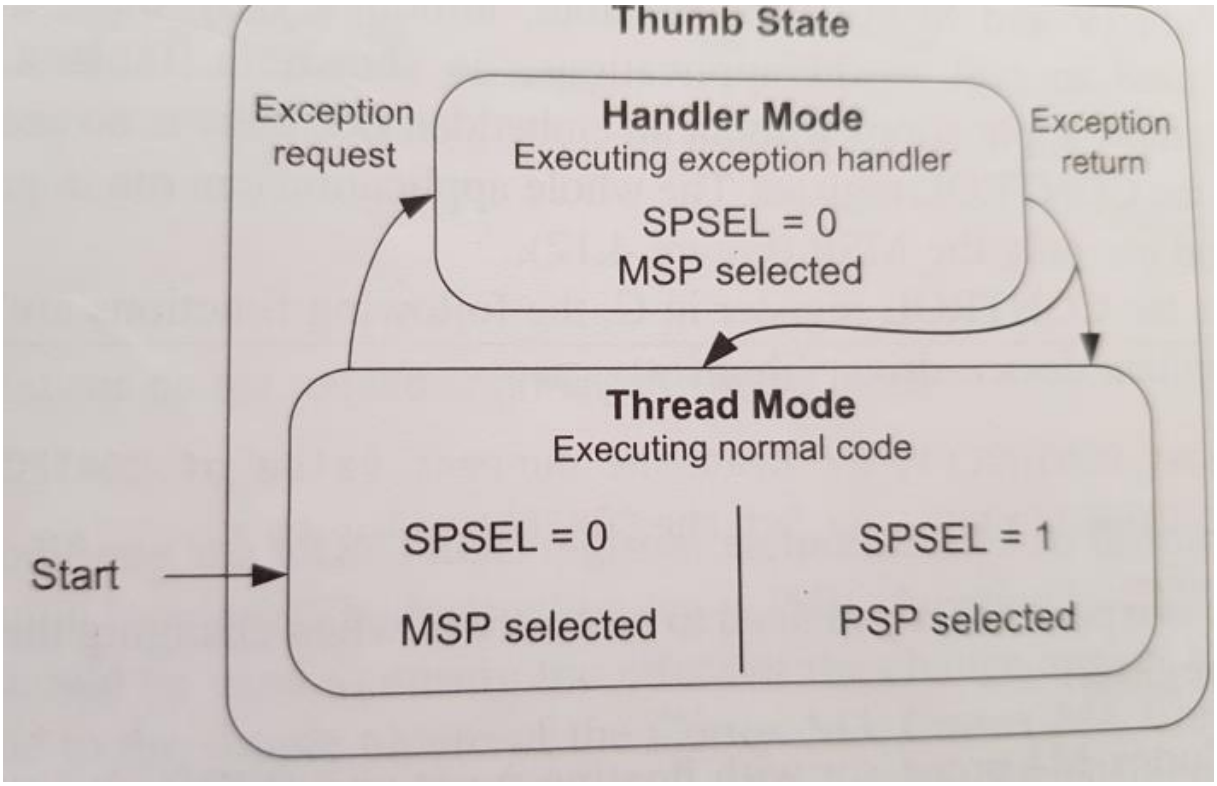
Rejestry specjalne



Rejestr Kontrolny

Table 4.3 Bit Fields in CONTROL Register

Bit	Function
nPRIV (bit 0)	Defines the privileged level in Thread mode: When this bit is 0 (default), it is privileged level when in Thread mode. When this bit is 1, it is unprivileged when in Thread mode. In Handler mode, the processor is always in privileged access level.
SPSEL (bit 1)	Defines the Stack Pointer selection: When this bit is 0 (default), Thread mode uses Main Stack Pointer (MSP). When this bit is 1, Thread mode uses Process Stack Pointer (PSP). In Handler mode, this bit is always 0 and write to this bit is ignored.
FPCA (bit 2)	Floating Point Context Active – This bit is only available in Cortex-M4 with floating point unit implemented. The exception handling mechanism uses this bit to determine if registers in the floating point unit need to be saved when an exception has occurred. When this bit is 0 (default), the floating point unit has not been used in the current context and therefore there is no need to save floating point registers. When this bit is 1, the current context has used floating point instructions and therefore need to save floating point registers. The FPCA bit is set automatically when a floating point instruction is executed. This bit is clear by hardware on exception entry. There are several options for handling saving of floating point registers. This will be covered in Chapter 13.



nPRIV i SPSEL w rejestrze kontrolnym

Different Combinations of nPRIV and SPSEL, (bit 0) and (bit 1) in CONTROL register respectively

nPRIV	SPSEL	Usage Scenario
0	0	Simple applications – the whole application is running in privileged access level. Only one stack is used by the main program and interrupt handlers. Only the Main Stack Pointer (MSP) is used.
0	1	Applications with an embedded OS, with current executing task running in privileged Thread mode. The Process Stack Pointer (PSP) is selected in current task, and the MSP is used by OS Kernel and exception handlers.
1	1	Applications with an embedded OS, with current executing task running in unprivileged Thread mode. The Process Stack Pointer (PSP) is selected in current task, and the MSP is used by OS Kernel and exception handlers.
1	0	Thread mode tasks running with unprivileged access level and use MSP. This can be observed in Handler mode but is less likely to be used for user tasks because in most embedded OS, the stack for application tasks is separated from the stack used by OS kernel and exception handlers.

If data is to be processed, it has to be loaded from the memory to registers in the register bank, processed inside the processor, and then written back to the memory („load-store architecture").

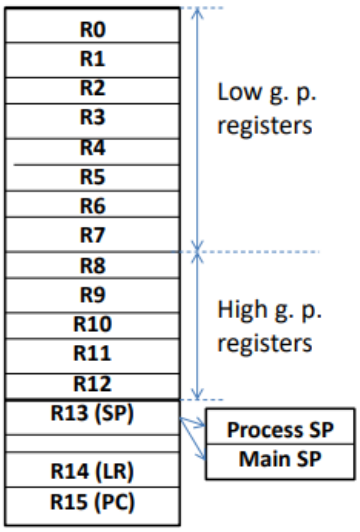
Registers R0 – R12 are general purpose registers. Many 16-bit instructions can only access the low registers. The high registers can be used with32-bit instructions, and a few with 16-bit instr…s.

R13 is the Stack Pointer. The stack memory is accessed via PUSH and POP operations. MSP is selected after reset (default SP) or when the processor is in Handler Mode. The PSP can only be used

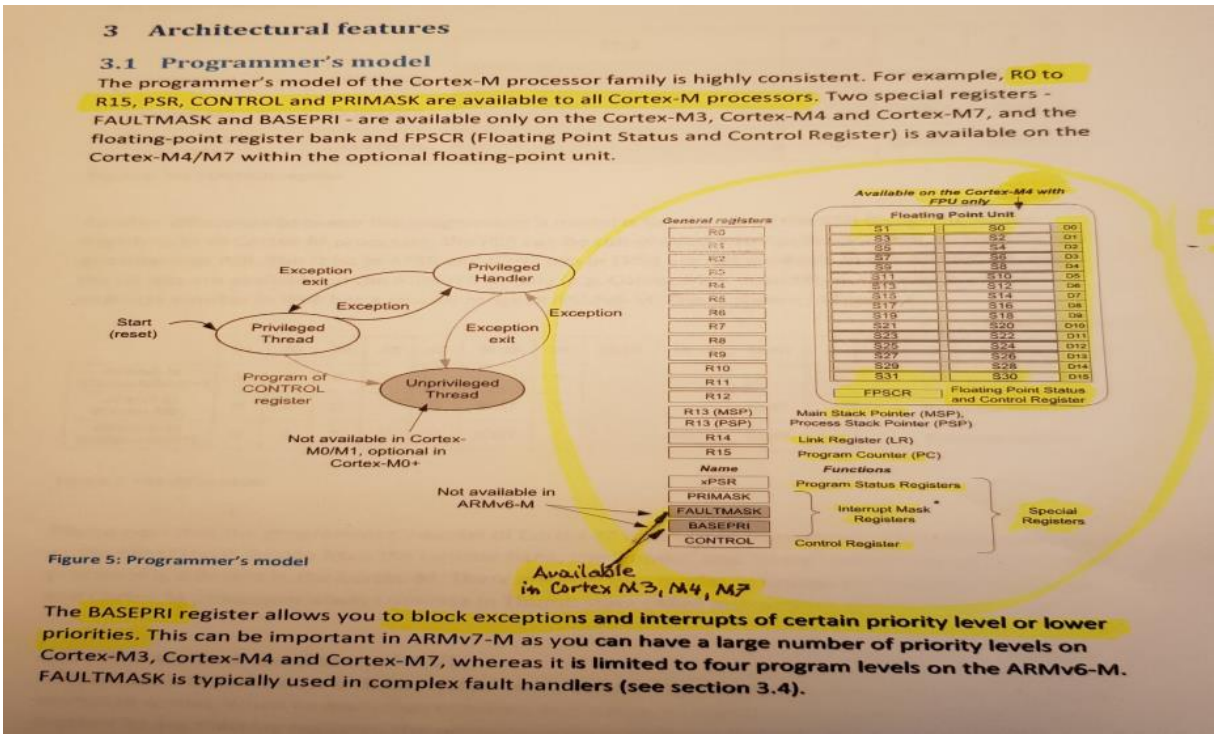
in Threat Mode. Many simple applications can rely on the MSP only. R14 is called the Link Register (LR). This is used for holding the return address when calling a function or subroutine. If a function needs to call another function, it needs to save the value of LR in the stack first.

R15 is the Program Counter. Writing to PC causes a branch operation.

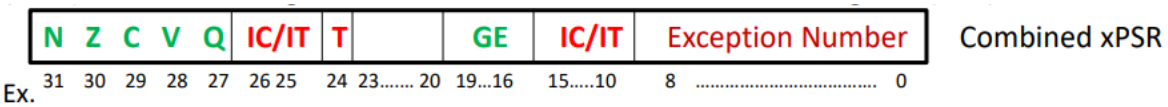
Allowed register names as assembly code: R0, R1 … R12, r0 … r12, R13, r13, SP, sp, R14, r14, LR, lr, R15, r15, PC, pc.



Register bank has **16 registers**. Therteen of them are **general purpose 32-bit registers**.



Program status registers: Application PSR (APSR), Execution PSR (EPSR), Interrupt PSR (IPSR). These three registers can be accessed as one combined register (PSR)



- MRS r0, PRS; read the combined program status register, move from PRS to r0
- MSR PSR, r0; write combined program state word, move from r0 to PSR
- MRS r0, APSR; read APSR (flag state) into R0

PRIMASK, FAULTMASK I BASEPRI to rejestry używane do exception lub maskowania przerwań.

The CONTROL register defines:

- Bit 1 mówi nam o wyborze stack pointera (Main SP lub Process SP)
- Bit 0 Mówi nam czy jesteśmy na poziomie uprzywilejowanym czy nie
- Dla Cortex-M4 z FPU (floating point unit) bit 2 mówi czy używany program korzysta z FPU

Rejestry 32 bitowe s0 – S31 możemy połączyć w 64 bitowe D0 – D15

Floating point status and control register (FPSCR). Some bits define some of the floating point operation behaviors, some bits provide status information about the floating point operation results.

(There are several additional memory-mapped registers (ex. Coprocessor Access Control reg.

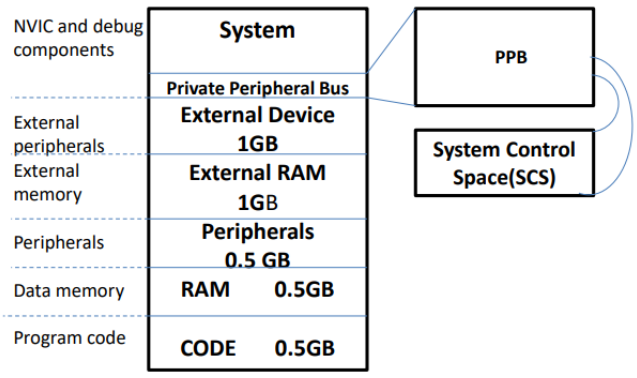
Poszczególne bity w FPSCR

Table 13.4 Bit Fields in FPSCR

Bit	Description
N	Negative flag (update by floating point comparison operations)
Z	Zero flag (update by floating point comparison operations)
C	Carry/borrow flag (update by floating point comparison operations)
V	Overflow flag (update by floating point comparison operations)
AHP	Alternate half precision control bit: 0 – IEEE half-precision format (default) 1 – Alternative half-precision format, see section 13.1.3
DN	Default NaN (Not a Number) mode control bit: 0 – NaN operands propagate through to the output of a floating point operation (default) 1 – Any operation involving one or more NaN(s) returns the default NaN
FZ	Flush-to-zero model control bit: 0 – Flush-to-zero mode disabled (default) (IEEE 754 standard compliant) 1 – Flush-to-zero mode enabled; denormalized values (tiny values with exponent equal 0) are flushed 0
RMode	Rounding Mode Control field; the specified rounding mode is used by almost all floating point instructions: 00 – Round to Nearest (RN) mode (default) 01 – Round towards Plus Infinity (RP) mode 10 – Round towards Minus Infinity (RM) mode 11 – Round towards Zero (RZ) mode
IDC	Input Denormal cumulative exception bit; set to 1 when floating point exception occurred, clear by writing 0 to this bit
IXC	Inexact cumulative exception bit; set to 1 when floating point exception occurred, clear by writing 0 to this bit
UFC	Underflow cumulative exception bit; set to 1 when floating point exception occurred, clear by writing 0 to this bit
OFC	Overflow cumulative exception bit; set to 1 when floating point exception occurred, clear by writing 0 to this bit
DZC	Division by Zero cumulative exception bit; set to 1 when floating point exception occurred, clear by writing 0 to this bit
IOC	Invalid Operation cumulative exception bit; set to 1 when floating point exception occurred, clear by writing 0 to this bit

Memory system features:

- 4GB pamięci
- Podzielona na regiony
- System zawiera NVIC i komponenty debugera
- PPB dodatkowe urządzenia dodane do procesora przez ich producentów
- 1 GB na zewnętrzne urządzenia peryferyjne
- 1 GB na zewnętrzną pamięć RAM
- 0,5 GB na urządzenia peryferyjne
- 0,5 GB na pamięć
- 0,5 GB na program

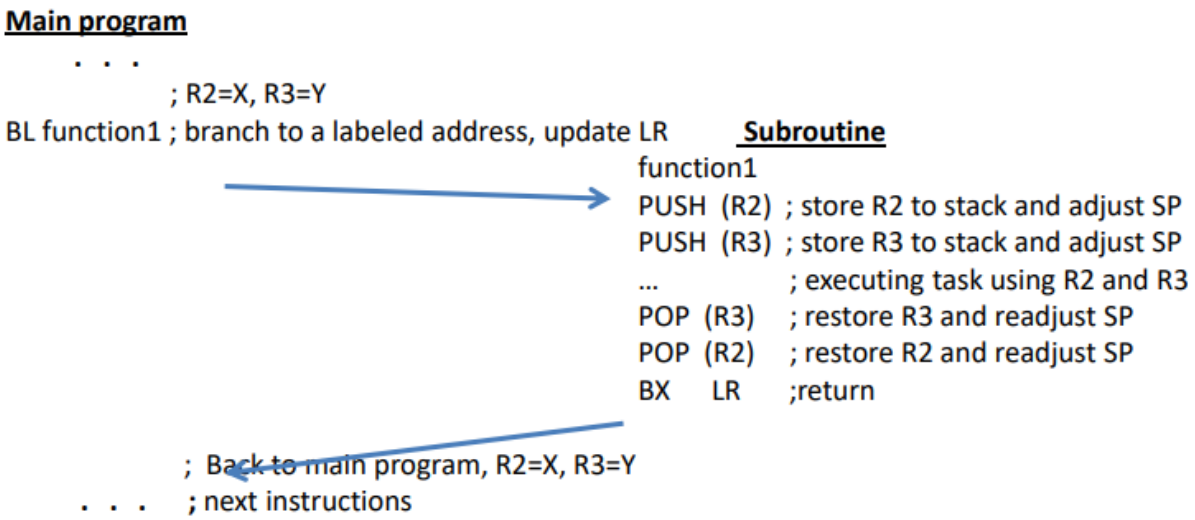


STOS to rodzaj pamięć LIFO(Ostatnie wchodzi pierwsze wychodzi) wykorzystywany do:

- Zapamiętywania status rejestrów
- Przy przerwaniu do przekazania informacji do podprogramu
- Zapisywania lokalnych zmiennych
- Tymczasowego przechowywania danych

Po resetowaniu procesora SP (Stack Pointer) wskazuje na koniec pamięci zarezerwowanej dla Stosu.

Rozkaz PUSH zmniejsza SP i zapisuje do niego jakąś wartość a POP odwrotnie.



Memory protection unit (MPU) o tym nie mówił ale dałem po angielsku w razie w

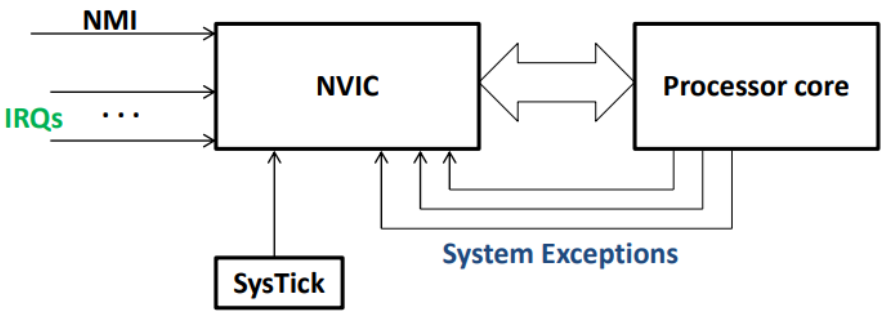
The MPU is optional in Cortex-M3 and M4. In the majority of applications, the MPU is not used. The MPU can be used to protect memory regions by means of defining access permissions in privileged and unprivileged access states.

Exceptions and interrupt

Pewnego typu exceptions są interrupts.

Te exceptions które przychodzą przez urządzenia peryferialne lub zewnętrzne to są interrupts

NVIC panuje nad interrupt Request IRQs oraz niemaskowanymi przerwaniami NMI (not mask interrupts)



Exceptions numbers 1 – 15 are classified as **system exceptions**.
Exceptions numbers 16 - 255 are for **interrupts**.
The NVIC can support up to 240 interrupt inputs.

Nested vectored interrupt controller (NVIC)

Flexible exception and interrupt management

- Każde z wejść IRQs może być włączone lub nie.
- Przerwanie jest impuls który ma długość conajmniej jednego okresu zegara. Pulsed interrupt request.
- Może też być - Level triggered interrupt request.

Nested exception/interrupt support

Każde przerwanie ma jakiś priorytet. Dla Interrupts można programować ich poziom. Kiedy występuje przerwanie, NVIC porówna priorytety przerwań z aktualnym I jeśli ma wyższy priorytet aktualne zadanie zostanie zatrzymana. Niektóre rejestry zostaną zachowane w Stosie, a processor zacznie egzekwować nowy ISR.

Vectored exception/interrupt entry

Istnieje tabela z wektorami przerwań.

Interrupt masking

Przy pomocy rejestru PRIMASK możemy wyłączyć wszystkie przerwania poza HardFault i NMI.

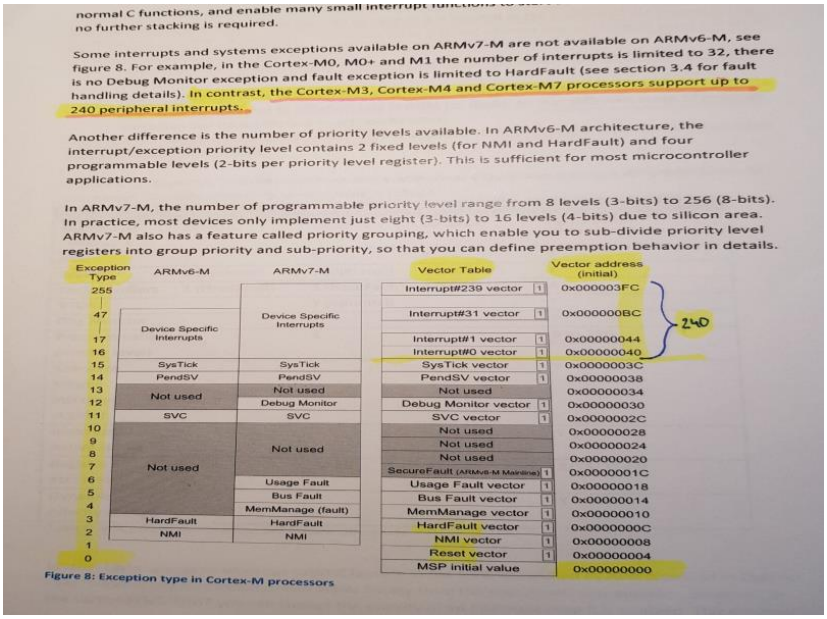
Przy pomocy rejestru BASEPRI możemy wybrać maskę dla przerwań (exceptions i interrupts) które są poniżej pewnego poziomu priorytetu/uprzywilejowania.

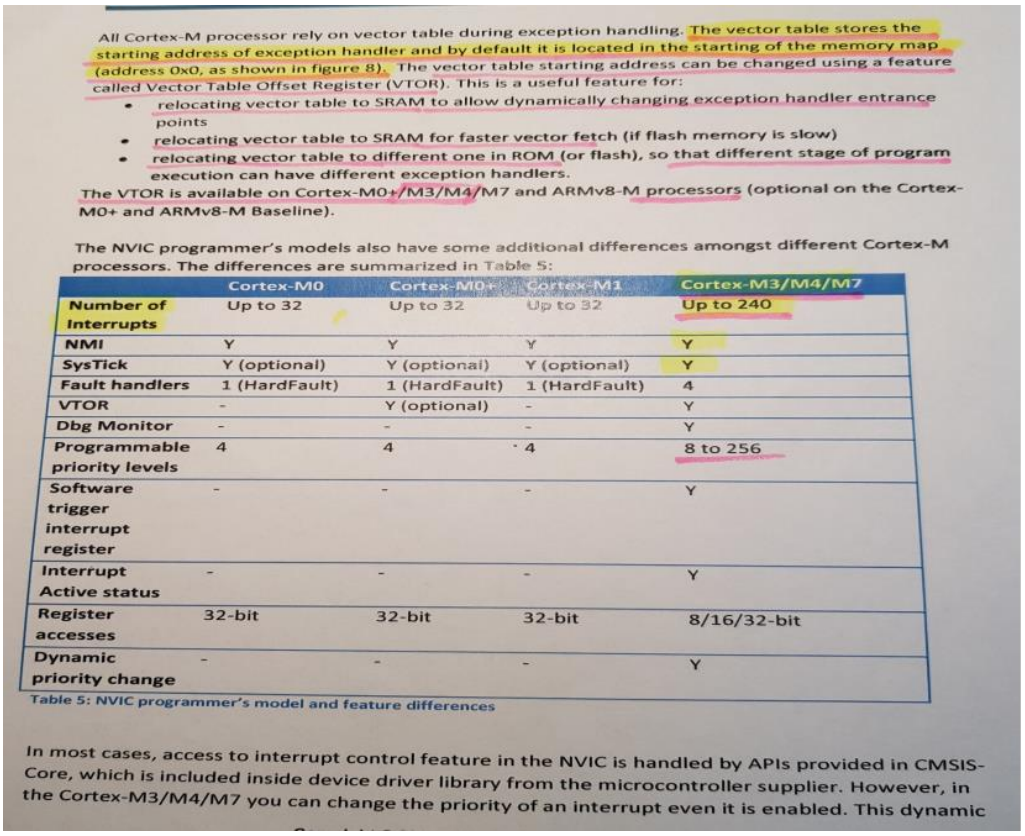
Vector table

Tabela zawiera adresy odpowiednich programów obsługujących przerwania

Tabela zaczyna się znajdujących się w Rejestrze Vector Table Offest Register

Po resecie VTOR wynosi 0. Dla NMI vector(type 2) jest na miejscu 2x4=0x00000008. Adres 0x00000000 jest używany dla MSP – main stack pointer.





All Cortex-M processor rely on vector table during exception handling. The vector table stores the starting address of exception handler and by default it is located in the starting of the memory map (address 0x0, as shown in figure 8). The vector table starting address can be changed using a feature called Vector Table Offset Register (VTOR). This is a useful feature for:

- relocating vector table to SRAM to allow dynamically changing exception handler entrance points
- relocating vector table to SRAM for faster vector fetch (if flash memory is slow)
- relocating vector table to different one in ROM (or flash), so that different stage of program execution can have different exception handlers.

The VTOR is available on Cortex-M0+/M3/M4/M7 and ARMv8-M processors (optional on the Cortex-M0+ and ARMv8-M Baseline).

The NVIC programmer's models also have some additional differences amongst different Cortex-M processors. The differences are summarized in Table 5:

	Cortex-M0	Cortex-M0+	Cortex-M1	Cortex-M3/M4/M7
Number of Interrupts	Up to 32	Up to 32	Up to 32	Up to 240
NMI	Y	Y	Y	Y
SysTick	Y (optional)	Y (optional)	Y (optional)	Y
Fault handlers	1 (HardFault)	1 (HardFault)	1 (HardFault)	4
VTOR	-	Y (optional)	-	Y
Dbg Monitor	-	-	-	Y
Programmable priority levels	4	4	4	8 to 256
Software trigger interrupt register	-	-	-	Y
Interrupt Active status	-	-	-	Y
Register accesses	32-bit	32-bit	32-bit	8/16/32-bit
Dynamic priority change	-	-	-	Y

Table 5: NVIC programmer's model and feature differences

In most cases, access to interrupt control feature in the NVIC is handled by APIs provided in CMSIS-Core, which is included inside device driver library from the microcontroller supplier. However, in the Cortex-M3/M4/M7 you can change the priority of an interrupt even it is enabled. This dynamic

Zbiór rozkazów

Rozkazy należą do zbioru Thumb-2 wykorzystujące 16 i 32 bitowe instrukcje

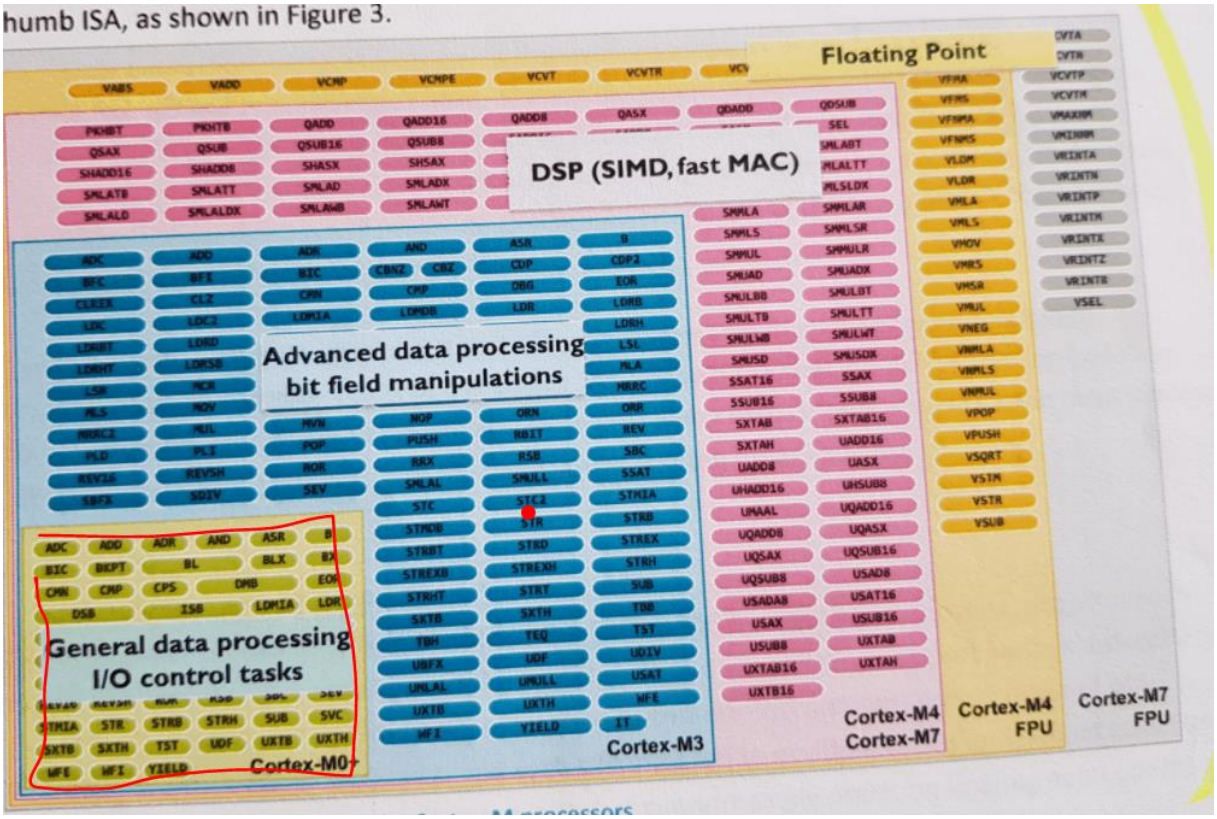
Programy zasadniczo piszemy w C, ale żeby zoptymalizować to można napisać fragmenty go w Assemblerze.

Format rozkazu to

Etykieta

Wyrażenie symboliczne Operator1, Operator2, .. ;komentarz

Po etykiecie nie ma dwukropka.



Krótka obwódka to rozkaz 16 bitowy długa obwódka to 32 bitowe

SIMD- Single instruction multiple data jeden rozkaz dotyczy kilku danych

Przykładowe Komendy

```
MOVS R0, #0x14   ; Set R0 value to 0x14 (hexadecimal) with APRS (flags) update
MOV  R1, #'B'    ; Set R1 = ASCII char. B
MOV  R4, R0      ; Copy value from R0 to R4
MOV  R3, #0x34   ; Set R3 value to 0x34

VMOV   R0, S0   ;Copy floating point register S0 to general purpose reg R0
VMOV.F32  S0, #1.0   ; Move single-precision value into floating pulse register S0
```

Rozkazy dotyczące FPU zaczynają się od V

OPERATION	DESCRIPTION	ASSEMBLER
Move (Move data within the processor)	Register	**MOV Rd, <op2>** ; the <op2> field can be ;replaced with one of the following options: ; A simple register specifier, for example Rm. ; An immediate shifted register, for example Rm, LSL #4. ; A register shifted register, for example Rm, LSL Rs. ; An immediate value, for example #0xE000E000.
	16-bit immediate	**MOVW Rd, #<imm>**
	Immediate into top	**MOVT Rd, #<imm>**
	To PC	**MOV PC, Rm**
Add	Add	**ADD Rd, Rn, <op2> ;** ADD Rd, Rn, Rm; Rd=Rn+Rm
	Add to PC	**ADD PC, PC, Rm**
	Add with carry	**ADC Rd, Rn, <op2>** ADC Rd, #immed; Rd=Rd+#immed+carry
	Form address	**ADR Rd, <label>**
Sub		**SUB Rd, Rn, Rm** ; Rd=Rn-Rm **SUB Rd, Rn, #immed**; Rd=Rn - #immed **SBC Rd, Rn, #immed** ; Rd=Rn - #immed- borrow . . .
Multiply	Multiply	**MUL Rd, Rn, Rm**
	Multiply accumulate	**MLA Rd, Rn, Rm**
	Multiply subtract	**MLS Rd, Rn, Rm**
	Long signed	**SMULL RdLo, RdHi, Rn, Rm**
	Long unsigned	**UMULL RdLo, RdHi, Rn, Rm**
	Long signed accumulate	**SMLAL RdLo, RdHi, Rn, Rm**
	Long unsigned accumulate	**UMLAL RdLo, RdHi, Rn, Rm**
Divide	Signed	**SDIV Rd, Rn, Rm**
	Unsigned	**UDIV Rd, Rn, Rm**
Saturate	Signed	**SSAT Rd, #<imm>, <op2>**
	Unsigned	**USAT Rd, #<imm>, <op2>**
Compare	Compare	**CMP Rn, <op2>**
	Negative	**CMN Rn, <op2>**
Logical	AND	**AND Rd, Rn, <op2>**
	Exclusive OR	**EOR Rd, Rn, <op2>**
	OR	**ORR Rd, Rn, <op2>**
	OR NOT	**ORN Rd, Rn, <op2>**
	Bit clear	**BIC Rd, Rn, <op2>**
	Move NOT	**MVN Rd, <op2>**
	AND test	**TST Rn, <op2>**
	Exclusive OR test	**TEQ Rn, <op1>**

Saturate- Nasycanie

Shift	Logical shift left	**LSL Rd, Rn, #<imm>**
	Logical shift left	**LSL Rd, Rn, Rs**
	Logical shift right	**LSR Rd, Rn, #<imm>**
	Logical shift right	**LSR Rd, Rn, Rs**
	Arithmetic shift right	**ASR Rd, Rn, #<imm>**
	Arithmetic shift right	**ASR Rd, Rn, Rs**
Rotate	Rotate right	**ROR Rd, Rn, #<imm>**
	Rotate right	**ROR Rd, Rn, Rs**
	With extension	**RRX Rd, Rn**
Count	Leading zeroes	**CLZ Rd, Rn**
Load	Word	**LDR Rd, [Rn, <op2>]**
	To PC	**LDR PC, [Rn, <op2>]**
	Halfword	**LDRH Rd, [Rn, <op2>]**
	Byte	**LDRB Rd, [Rn, <op2>]**
	Signed halfword	LDRSH Rd, [Rn, <op2>]
	Signed byte	**LDRSB Rd, [Rn, <op2>]**
	User word	**LDRT Rd, [Rn, #<imm>]**
	User halfword	**LDRHT Rd, [Rn, #<imm>]**
	User byte	**LDRBT Rd, [Rn, #<imm>]**
	User signed halfword	**LDRSHT Rd, [Rn, #<imm>]**
	User signed byte	**LDRSBT Rd, [Rn, #<imm>]**
	PC relative	**LDR Rd,[PC, #<imm>]**
	Doubleword	**LDRD Rd, Rd, [Rn, #<imm>]**
	Multiple	**LDM Rn, {<reglist>}**
	Multiple including PC	**LDM Rn, {<reglist>, PC}**
Store	Word	**STR Rd, [Rn, <op2>]**
	Halfword	**STRH Rd, [Rn, <op2>]**
	Byte	**STRB Rd, [Rn, <op2>]**
	Signed halfword	**STRSH Rd, [Rn, <op2>]**
	Signed byte	**STRSB Rd, [Rn, <op2>]**
	User word	**STRT Rd, [Rn, #<imm>]**
	User halfword	**STRHT Rd, [Rn, #<imm>]**
	User byte	**STRBT Rd, [Rn, #<imm>]**
	User signed halfword	**STRSHT Rd, [Rn, #<imm>]**
	User signed byte	**STRSBT Rd, [Rn, #<imm>]**
	Doubleword	**STRD Rd, Rd, [Rn, #<imm>]**
	Multiple	**STM Rn, {<reglist>}**
Push	Push	**PUSH {<reglist>}**
	Push with link register	**PUSH {<reglist>, LR}**
Pop	Pop	**POP {<reglist>}**
	Pop and return	**POP {<reglist>, PC}**

Semaphore	Load exclusive	**LDREX Rd, [Rn, #<imm>]**
	Load exclusive half	**LDREXH Rd, [Rn]**
	Load exclusive byte	**LDREXB Rd, [Rn]**
	Store exclusive	**STREX Rd, Rt, [Rn, #<imm>]**
	Store exclusive half	**STREXH Rd, Rt, [Rn]**
	Store exclusive byte	**STREXB Rd, Rt, [Rn]**
	Clear exclusive monitor	**CLREX**
Branch	Conditional	**B<cc> <label>**
	Unconditional	**B <label>**
	With link	**BL <label>**
	With exchange	**BX Rm**
	With link and exchange	**BLX Rm**
	Branch if zero	**CBZ Rn, <label>**
	Branch if non-zero	**CBNZ Rn, <label>**
	Byte table branch	**TBB [Rn, Rm]**
	Halfword table branch	**TBH [Rn, Rm, LSL#1]**
State change	Supervisor call	**SVC #<imm>**
	If-then-else	**IT... <cond>**
	Disable interrupts	**CPSID <flags>**
	Enable interrupts	**CPSIE <flags>**
	Read special register	**MRS Rd, <specreg>**
	Write special register	**MSR <specreg>, Rn**
	Breakpoint	**BKPT #<imm>**

Branch – skoki

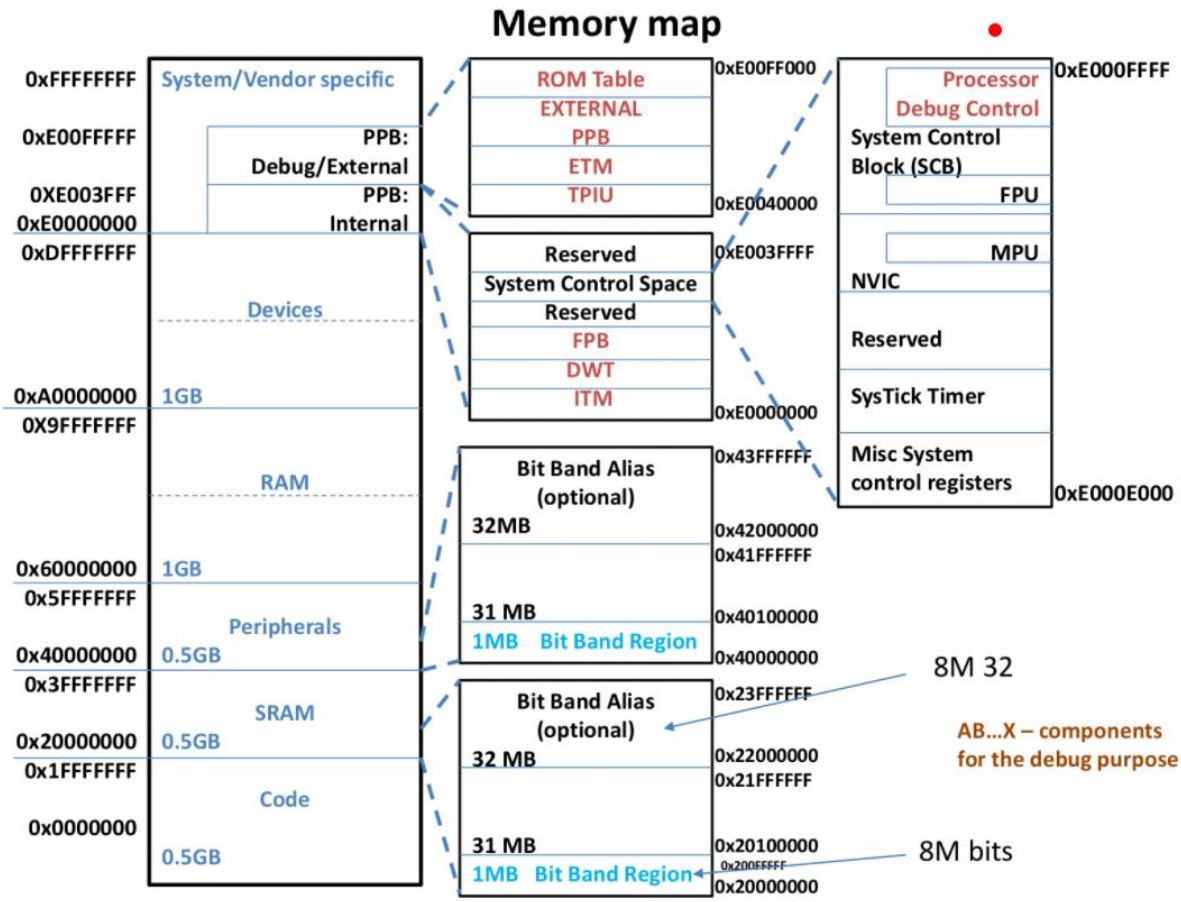
State change – zmiana stanu

Extend	Signed halfword to word	**SXTH Rd, <op2>**
	Signed byte to word	**SXTB Rd, <op2>**
	Unsigned halfword	**UXTH Rd, <op2>**
	Unsigned byte	**UXTB Rd, <op2>**
Bit field	Extract unsigned	**UBFX Rd, Rn, #<imm>, #<imm>**
	Extract signed	**SBFX Rd, Rn, #<imm>, #<imm>**
	Clear	**BFC Rd, Rn, #<imm>, #<imm>**
	Insert	**BFI Rd, Rn, #<imm>, #<imm>**
Reverse	Bytes in word	**REV Rd, Rm**
	Bytes in both halfwords	**REV16 Rd, Rm**
	Signed bottom halfword	**REVSH Rd, Rm**
	Bits in word	**RBIT Rd, Rm**
Hint	Send event	**SEV**
	Wait for event	**WFE**
	Wait for interrupt	**WFI**
	No operation	**NOP**
Barriers	Instruction synchronization	**ISB**
	Data memory	**DMB**
	Data synchronization	**DSB <flags>**

The Cortex-M4 processor implements a number of the DSP instructions.

System Pamięci

32 bitowe adresy pamięci wielkość pamięci 4GB



Części pamięci

Memory regions:
Code – A 512MB memory space primary for program code, including the default vector table. Data access is allowed.

SRAM – The SRAM region is located in the next 512MB of memory space. The first 1MB of the SRAM region is bit addressable if the bit-band feature is included. Program can be executed from this region.

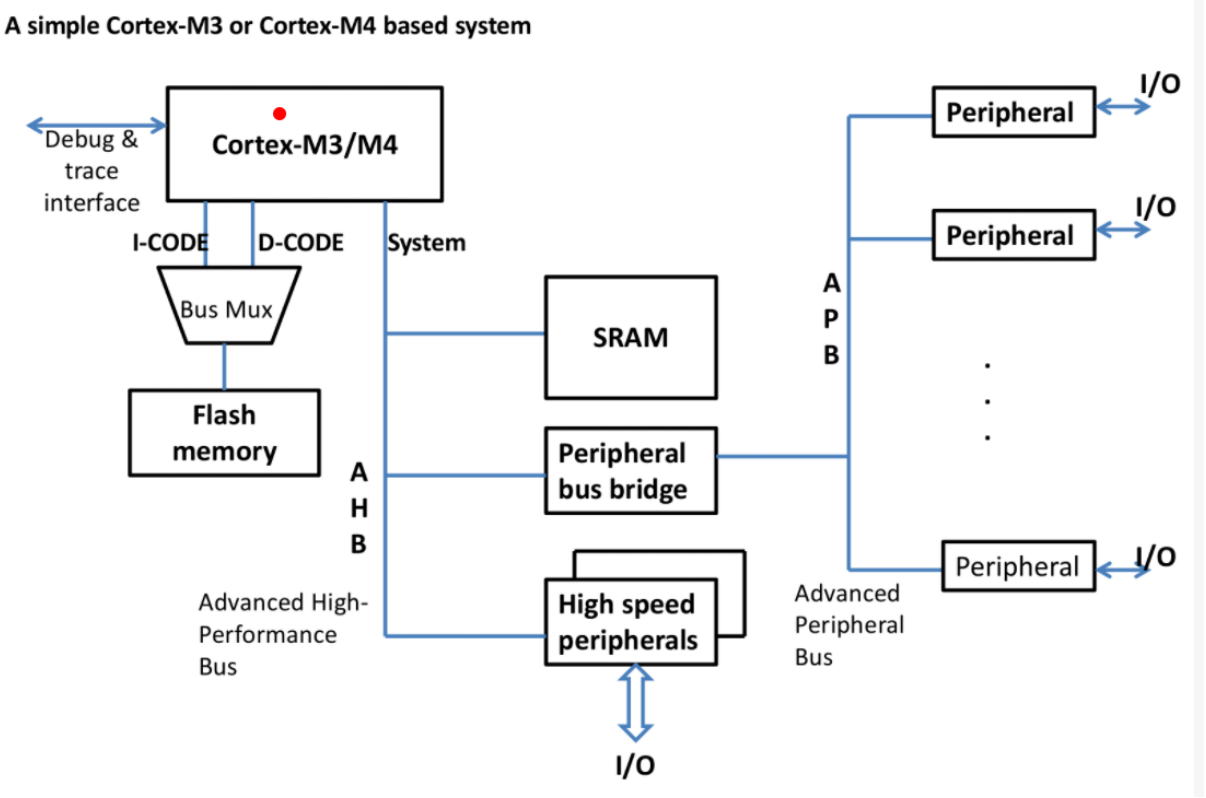
Peripherals – This region also has the size of 512MB, and is used mostly for on-chip peripherals. The first 1MB of the SRAM can be bit addressable if the bit-band feature is included.

RAM – The region contains two slots of 512MB memory space. He RAM region can be used for program code as well as data.

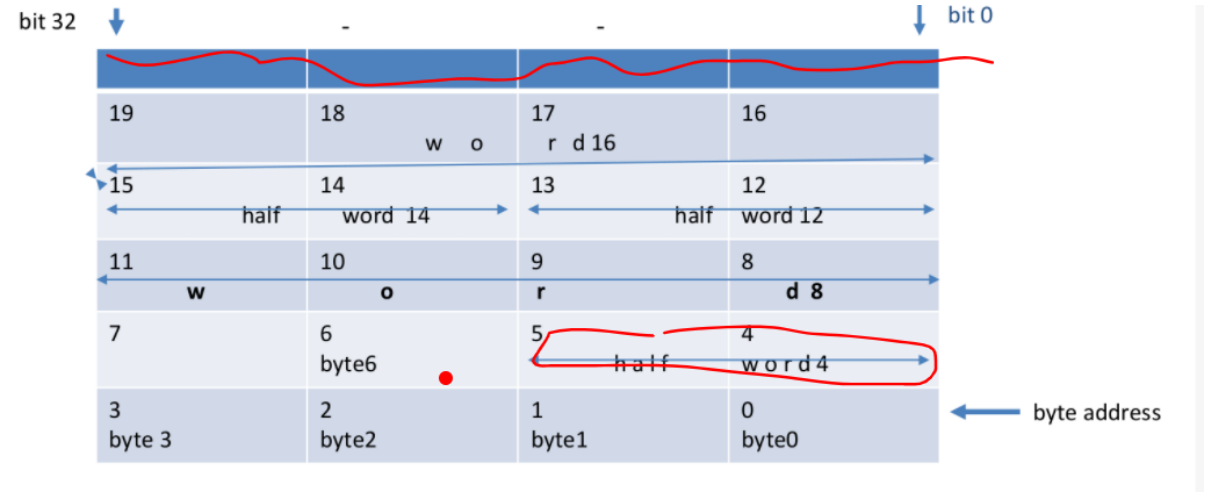
Devices – The Device region contains two slots of 512MB memory space for off-chip periferals.

System – The System region contains:
- **Internal** Private Peripheral Bus (PPB). The internal PPB is used to access system components such as NVIC, SysTick, MPU, as well as debug components inside the Cortex-M3/4 processors. This memory space can be accessed by program code running in privileged state.
- **External** PPB. The region is available for additional optional debug components added by the silicon vendors. Vendor-specific area, 0xE0100000 to 0xFFFFFFFF.

Typowe połączenie Procesora z pamięcią i urządzeniami peryferyjnymi



Każdy bajt pamięci ma swoje adresy



W little endian

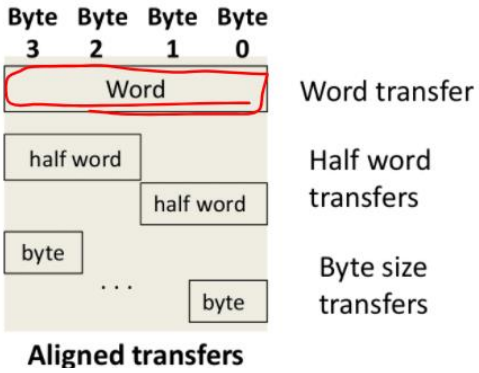
2 bajty to half word numer mniejszego bajta

4 bajty to word numer najmniejszego bajta

Memory system – aligned and unaligned data access

The memory system is 32-bit (at least from a programmer's model point of view). A data access that is 32-bit (word) or 16-bit (half-word) in size can either be aligned or unaligned.

Aligned transfer means the address value is a multiple of the size (in bytes). **A word-size aligned transfer** can be carried out to address 0x00000000, 0x00000004, …, 0x00000100, 0x00000104,…, and so on. **A half-word size aligned transfer** can be carried out to 0x00000000, 0x00000002, 0x00000004, …, and so on.



Aligned transfers

Kompilatory C nie generują unaligned data accesses. W tej sytuacji są rozbijane na kilka osobnych adresów i są wykonywane jako kilka mniejszych transferów dlatego są mniej optymalne

BIT Band Operation

Bit-band operation support allows a single load/store operation to read/write a single data bit. This is supported in two memory regions called **bit-band regions**. One of them is located **in the first 1MB of the SRAM region**, and the other is located **in the first 1MB of the peripheral region:**

- 0x20000000 - 0x200FFFFF (SRAM, 1MB)
- 0x40000000 – 0x400FFFFF (Peripherals, 1MB)

When the bit-band alias address is accessed, the address is remaped into a bit-band address.

For example to set bit 2 in word data in address 0x20000000, instead of using three instructions to read the data, set the bit, and then write back the result, this task can be carried out by a single instruction.

Without Bit-Band

```
LDR     R0, =0x20000000 ;
LDR     R1, [R0] ; Read from memory
ORR.W   R1, #0x4  ; Modify bit
STR     R1, [R0]  ; Write back result
```

With Bit-Band

```
LDR   R0, =0x22000008 ; Setup address
MOV   R1, #1          ; Setup data
STR   R1, [R0]        ; Write. The written bit data is shifted
                        to the required bit position
```

How to set a register to a 32-bit immediate data value?

```
LDR  R0, =0x12345678;  Set R0 to 0x12345678.
```

The assembler converts this instruction into a memory transfer instruction and a literal data item stored in the program image:

```
LDR  R0, [PC, #offset] ; Load (Read from memory)
. . .
DCD  0x12345678 ; Directive used for inserting data into a program
```

Another way to generate a 32-bit immediate data value:

```
MOVW   R0, #0x5678  ; Set R0 to 0x00005678
MOVT   R0, #0x1234 ; Set upper 16-bit of R0 to 0x1234.  Now R0=0x12345678
```

Write a bit without bit-band. We set bit 2 in word data in address 0x20000000.

```
LDR      R0, =0x20000000; Setup address
LDR      R1,  [R0]          ; Read 0x20000000 to register
ORR.W  R1, #0x4           ;Modify bit, i.e. set bit 2 in register. (Number 0x4= 00000100B, where bit 2 = 1 )
STR      R1,   [R0]          ; Write back result. Write register to 0x20000000
```

Within the bit-band region, each word is represented by an LSB of 32 words in the bit-band alias address range.

Write a bit with Bit-Band. We set bit 2 in word data in address 0x20000000.

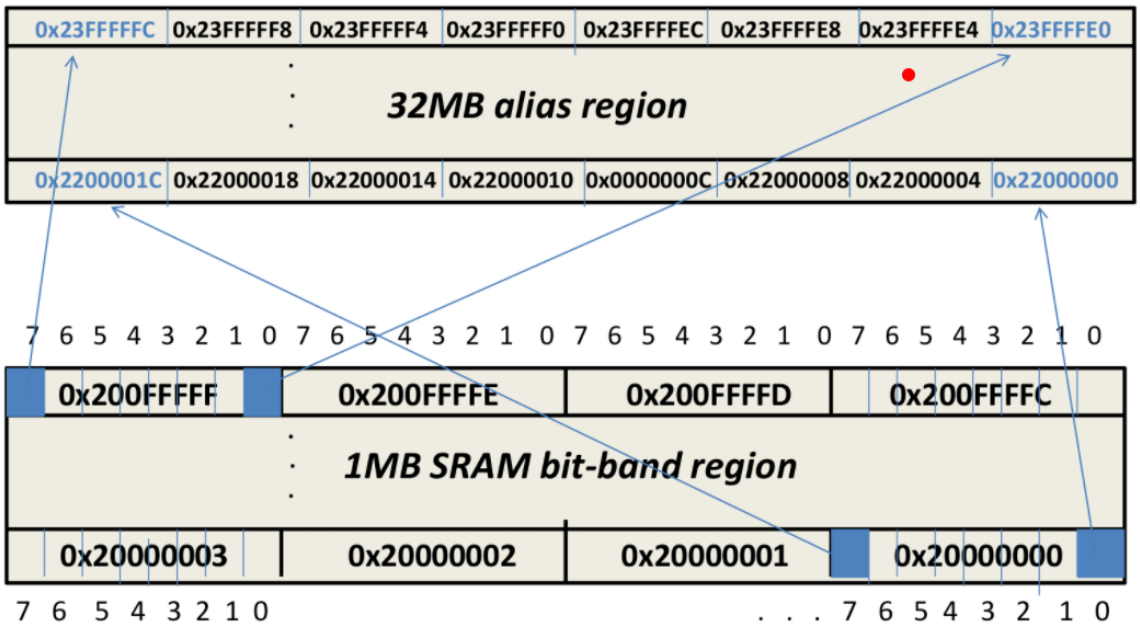
```
LDR     R0, =0x22000008     ;Setup address. Bit-band alias address of bit 2 in word data in address 0x20000000
                            ; is 0x22000008
MOV    R1, #1               ; Setup data
STR     R1,  [R0]           ; Write. Store content of R1 into memory location addressed by R0. When the bit-band alias
                            ; address is accessed, the address is remapped into a bit-band address.
            ; Writing a word from R1 in the alias region updates a single bit (bit 2) in the bit-band region.
```

Remapping of Bit-Band Addresses in SRAM region

Bit-Band Region	Aliased Equivalent
0x20000000 bit[0]	0x22000000 bit[0]
0x20000000 bit[1]	0x22000004 bit[0]
0x20000000 bit[2]	**0x22000008 bit[0]**
…	
0x20000000 bit[31]	0x2200007C bit[0]
0x20000004 bit[0]	0x22000080 bit[0]
…	
0x20000004 bit[31]	0x220000FC bit[0]
…	
0X200FFFFC bit[31]	0x23FFFFFC bit[0]

Recorded with BigBlueButton

Rejestr w bit band i odpowiednik w aliased



Memory system – bit-band operations

Read from the bit-band alias

Without bit-band

Read 0x20000000 to register

Shift bit 2 to LSB and mask other bits

With bit-band

Read from 0x22000008

Read data from 0x20000000, and extract bit 2 to register

Without bit-band

```
LDR     R0, 00x20000000; setup address
LDR     R1, [R0]          ; read
UBFX.W  R1, R1, #2, #1; Extract bit [2]
```

With bit-band

```
LDR   R0, =0x22000008; setup address
LDR   R1, [R0]
```

(Notice that a similar feature (bit-band operation) has existed for 30 years on 8-bit microcontrollers, for example on 8051.)

Example

1. Set address 0x20000000 to value 0x3355AACC

2. Read address 0x22000008. This read access is remapped into read access to 0x20000000, and bit 2 is extracted to register, i.e. the return value is 1 (bit [2] of ox3355AACC).

3. Write 0x0 to 0x22000008. This write access remapped into a READ0MODIFY-WRITE to 0x20000000. The value 0x3355AACC is read from memory, bit 2 is cleared, and a result of 0x3355AAC8 is written back to address 0x20000000.

4. Now read 0x20000000. that gives you a return value of 0x3355AAC8 (bit [2] cleared).

When you access bit-band alias address, only the LCB (bit [0]) in the data is used.
(The bit-band feature is optional on current releases of Cortex-M3 and M4 processors.)

Exceptions i priorytety

Exceptions are numbered 1-15 for system exceptions and 16 and above for external interrupt inputs.
System exceptions:

Exception Number	Exception Type	Priority	Descriptions
1	Reset	-3(highest)	Reset
2	NMI	-2	Non-maskable Interrupt (NMI), can be generated from on chip peripherals or from external sources
3	Hard Fault	-1	All fault conditions, if the corresponding fault handler is not enabled
4	MemManage Fault	Progra-mmable	MPU violation or program execution from address location with XN (eXecute Never) memory attribute
5	Bus fault	Prog.	Bus error.
6	Usage fault	Prog.	Exceptions due to program error or trying to access co-processor.
7-10	Reserved		
11	SVC	Prog.	SuperVisor Call
12	Debug Monitor	Prog.	Debug monitor (breakpoints, watchpoints)
13	Reserved		
14	PendSV	Prog.	Pendable service call. (Used by an OS in context switching)
15	SYSTICK	Prog.	Exception generated by a System Tick Timer peripheral

Interupts

Exceptions and Interrupts

List of interrupts

Exception Number	Exception Type	Priority	Descriptions
16	Interrupt #0	Programmable	Generated from on chip peripherals or from external sources
17	Interrupt #1	Programmable	
. . .			
255	Interrupt #239	Programmable	

Priorytety Czasowo

An interrupt request can be accepted by the processor if:

- The pending status is set
- The interrupt is enabled
- The priority of the interrupt is higher than the current level

